

PSOPT Application Examples Document

Victor Becerra
Email: vmbecerra@vmb1.com
www: <http://www.psopt.net>
October 2, 2026

Copyright © 2026 Victor Becerra



Contents

1	About this document	3
2	Brachistochrone: a single-phase problem with free final time	4
3	Two-phase Schwartz problem: linking phases	10
4	Catalytic cracking of gas oil: parameter estimation	18
5	Bryson's maximum range problem: an equality path constraint	27
6	Interior point constraint: a condition imposed inside the time horizon	30
7	Singular control: a problem whose optimal control is undetermined on part of the interval	33
8	Time delay problem: states that depend on their own past	37
9	Constrained pendulum: a high-index DAE, and a second parameter estimation	41
10	Integer static parameter: a discrete-valued decision variable	45
11	Lotka-Volterra fishing: an integer-valued control	47
12	An analytic test problem: integrated-residual transcription against collocation	50
13	Chance-constrained double integrator: a probabilistic path constraint by constraint tightening	53
14	Manutec r3 robot: a multibody model and <i>hp</i> -adaptive mesh refinement	55
15	Minimum time to climb: a model driven by look-up tables	60
16	Multiphase launch vehicle ascent: linkages, mass discontinuities and terminal orbital elements	63
17	A two-link robotic arm: multiple-shooting transcription	68
18	The Python interface	74
19	Robust optimal control: an arm with an uncertain payload	82

PSOPT Application Examples

Victor Becerra
Email: vmbecerra@vmb1.com

October 2, 2026

1 About this document

This document describes a number of optimal control problems that have been solved using PSOPT. It does not list everything included in the distribution. The `examples` directory has several dozen problems, but the purpose here is more focused. Each problem below was chosen to represent a type of problem or a capability of the software, so that a reader who has a problem of their own can find a similar example, see how it was set up, and then check the source file and copy the parts that apply. The classes covered are a single-phase problem with free final time, a multi-phase problem, parameter estimation, a problem with a path constraint, a problem with an interior-point constraint, a singular problem, delayed states, a high-index differential-algebraic system, an integer static parameter, an integer control, the integrated-residual transcription, a chance-constrained problem, automatic mesh refinement in both its local and its *hp*-adaptive forms, a minimum-time problem driven by tabulated data, a multi-phase launch vehicle trajectory, the multiple-shooting transcription, robust optimal control under parametric uncertainty, and the Python interface.

Each section is organised in the same way. First, there is a brief discussion that explains what the problem is, where it comes from, and what to pay attention to when solving it. Next, a summary table presents the problem in the format that PSOPT requires. This includes the objective, dynamics, boundary conditions, path constraints, bounds, constants, and the discretisation and mesh-refinement settings used. Finally, the results section includes the solver's summary, the figures, and, if available, a comparison with a published or analytical answer.

The summary tables are designed to be read independently. If a problem includes large or uninteresting data, such as tables of aerodynamic coefficients, lists of measurements, or sets of physical constants, the table will indicate where to find them instead of repeating the information. You can find the complete, runnable source code for every example in the PSOPT distribution, under `examples/<name>/`. Only the source listings for three of the simplest problems are printed here. Once you understand how these are structured, you can open the others in an editor as needed.

Most of these problems come from published sources, so the solutions found here can be checked against existing results. If a published value is available, it appears in

the summary table and is compared with what PSOPT produces. If there is no published value, either because the problem is new or its answer is an infimum instead of a minimum, this is noted in the section.

Reproducing the results

Everything numerical in this document is generated using a specific version of PSOPT. This includes the figures, the results boxes, the mesh refinement tables, the parameter statistics. The source code for the examples can be found in the PSOPT distribution under `src/`.

A reader who wants to check a number quoted in the text can fetch the following release of PSOPT from GitHub, then build PSOPT and run the corresponding example:

PSOPT release 2026-10-01, built 2026-10-01 14:38:44. Results regenerated on 01 October 2026.
--

Computing times are reported as they were measured and should be read as indicative only, since they depend on the machine; the node counts, iteration counts, error estimates and objective values do not.

2 Brachistochrone: a single-phase problem with free final time

The brachistochrone is the oldest problem in the calculus of variations: given two points in a vertical plane, find the shape of the wire down which a bead, released from rest and sliding without friction, travels between them in the least time. Johann Bernoulli posed it in 1696 and the answer, that the curve is a cycloid, was supplied within a year by Newton, Leibniz, l'Hôpital, Jacob Bernoulli and Johann Bernoulli himself. It is used here as the simplest possible complete example: one phase, three states, one control, a free final time, no path constraints, and an answer that can be written down in closed form and compared with what the software produces.

Posed as an optimal control problem [4], the control is the angle θ of the wire from the vertical, the states are the two coordinates of the bead and its speed, and the cost is the final time itself. Measuring y downwards makes the dynamics free of a sign convention: the bead accelerates at $g \cos \theta$ along the wire, and conservation of energy, which the equations imply and do not impose, gives $v = \sqrt{2gy}$ along any trajectory that starts from rest at the origin.

Two features of the formulation matter here, because they recur in every later example. The final time is free, and is made so by giving it a bound ($t_f \in [0, 10]$) and not a value; PSOPT then treats it as one more decision variable. And the terminal conditions are imposed as event constraints, with equal lower and upper bounds, which is how an equality is written throughout the interface.

Because the problem is smooth and has no active path constraint, a global pseudospectral discretisation is well suited to it, and a single Legendre grid of 40 nodes is

enough: the maximum relative local error is 4.0×10^{-9} without any mesh refinement. The computed minimum time agrees with the analytic cycloid solution to eight significant figures, which is a more useful check on an installation than any statement in this document.

Table 1: Brachistochrone problem: summary

<i>Demonstrates</i>	The smallest complete problem: single phase, free final time, terminal conditions as event constraints, global pseudospectral discretisation on a fixed mesh
<i>Reference</i>	Bryson and Ho [4]
<i>Dimensions</i>	1 phase; 3 states, 1 control, 5 events, no path constraints, no static parameters
<i>Objective</i>	$J = t_f$
<i>Dynamics</i>	$\dot{x} = v \sin \theta, \quad \dot{y} = v \cos \theta, \quad \dot{v} = g \cos \theta$
<i>Initial conditions</i>	$x(0) = y(0) = v(0) = 0, \quad t_0 = 0$
<i>Terminal conditions</i>	$x(t_f) = 2, \quad y(t_f) = 2$
<i>Bounds</i>	$0 \leq x, y, v \leq 20, \quad 0 \leq \theta \leq 2\pi, \quad 0 \leq t_f \leq 10$
<i>Constants</i>	$g = 9.8$
<i>Discretisation</i>	Legendre pseudospectral, 40 nodes, automatic differentiation, automatic scaling, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	The cycloid $x = k(\phi - \sin \phi), y = k(1 - \cos \phi), t = \sqrt{k/g} \phi$. The terminal condition $x(t_f) = y(t_f)$ gives $\phi_f = 2.412011143914$, $k = 1.145834075064$ and $t_f^* = 0.824759143157$
<i>Computed</i>	$t_f = 8.247591 \times 10^{-1}$, a relative error of 5×10^{-8} against t_f^*
<i>Source</i>	examples/brac1/brac1.cxx, listed below

Results

PSOPT results summary

=====

Problem: Brachistochrone Problem

CPU time (seconds): 2.323390e-01

NLP solver used: IPOPT

Optimal (unscaled) cost function value: 8.247591e-01

Phase 1 endpoint cost function value: 8.247591e-01

Phase 1 integrated part of the cost: 0.000000e+00

Phase 1 initial time: 0.000000e+00

Phase 1 final time: 8.247591e-01

Phase 1 maximum relative local error: 1.150780e-08

NLP solver reports: The problem has been solved!

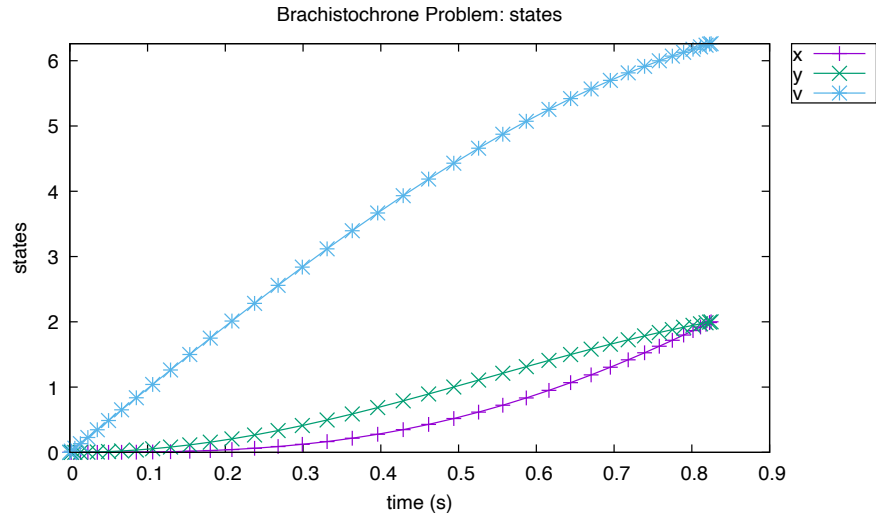


Figure 1: States for the brachistochrone problem

Source

The complete source of this example is listed below. It should be read once in full, because every later example has the same skeleton: the user functions `endpoint_cost`, `integrand_cost`, `dae`, `events` and `linkages` (with `observation_function` added for estimation problems), then the two-level problem setup, the bounds, the initial guess, the algorithm options, and the call to `psopt`.

```

////////////////////////////////////
////////////////////////////////////      brach1.cxx      ///////////////////////////////////
////////////////////////////////////
////////////////////////////////////      PSOPT Example      ///////////////////////////////////
////////////////////////////////////
////////////////////////////////////      Title:      Brachistochrone problem      ///////////////////////////////////
////////////////////////////////////      Last modified:      04 January 2009      ///////////////////////////////////
////////////////////////////////////      Reference:      Bryson and Ho (1975)      ///////////////////////////////////
////////////////////////////////////      (See PSOPT handbook for full reference)      ///////////////////////////////////
////////////////////////////////////
////////////////////////////////////      Copyright (c) Victor M. Becerra, 2009      ///////////////////////////////////
////////////////////////////////////
////////////////////////////////////      This is part of the PSOPT software library, which      ///////////////////////////////////
////////////////////////////////////      is distributed under the terms of the GNU Lesser      ///////////////////////////////////
////////////////////////////////////      General Public License (LGPL)      ///////////////////////////////////
////////////////////////////////////
#include "psopt.h"

using namespace std;

using namespace PSOPT;

////////////////////////////////////

```

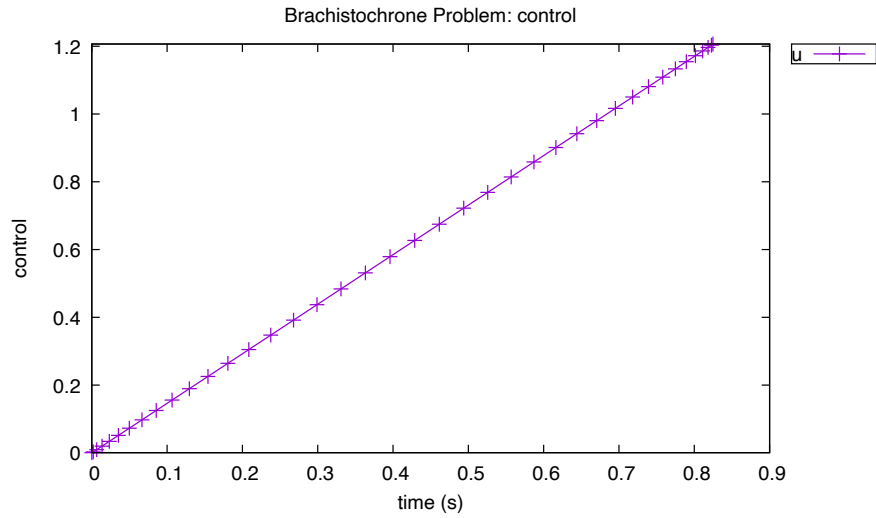


Figure 2: Control for the brachistochrone problem

```

//////////////////////////////// Define the end point (Mayer) cost function //////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////

adouble endpoint_cost(adouble* initial_states, adouble* final_states,
                      adouble* parameters, adouble& t0, adouble& tf,
                      adouble* xad, int iphase, Workspace* workspace)
{
    return tf;
}

////////////////////////////////////////////////////////////////////////////////////////////////////////
//////////////////////////////// Define the integrand (Lagrange) cost function //////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////

adouble integrand_cost(adouble* states, adouble* controls, adouble* parameters,
                      adouble& time, adouble* xad, int iphase, Workspace* workspace)
{
    return 0.0;
}

////////////////////////////////////////////////////////////////////////////////////////////////////////
//////////////////////////////// Define the DAE's //////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////

void dae(adouble* derivatives, adouble* path, adouble* states,
         adouble* controls, adouble* parameters, adouble& time,
         adouble* xad, int iphase, Workspace* workspace)
{
    adouble xdot, ydot, vdot;

    adouble x = states[ 0 ];
    adouble y = states[ 1 ];
    adouble v = states[ 2 ];

    adouble theta = controls[ 0 ];

    xdot = v*sin(theta);
    ydot = v*cos(theta);
    vdot = 9.8*cos(theta);

    derivatives[ 0 ] = xdot;
    derivatives[ 1 ] = ydot;
    derivatives[ 2 ] = vdot;
}

////////////////////////////////////////////////////////////////////////////////////////////////////////
//////////////////////////////// Define the events function //////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////

```

```

////////////////////////////////////
void events(adouble* e, adouble* initial_states, adouble* final_states,
           adouble* parameters, adouble& t0, adouble& tf, adouble* xad,
           int iphase, Workspace* workspace)
{
    adouble x0 = initial_states[ 0 ];
    adouble y0 = initial_states[ 1 ];
    adouble v0 = initial_states[ 2 ];
    adouble xf = final_states[ 0 ];
    adouble yf = final_states[ 1 ];

    e[ 0 ] = x0;
    e[ 1 ] = y0;
    e[ 2 ] = v0;
    e[ 3 ] = xf;
    e[ 4 ] = yf;
}

////////////////////////////////////
//////////////////////////////////// Define the phase linkages function ///////////////////////////////////
////////////////////////////////////

void linkages( adouble* linkages, adouble* xad, Workspace* workspace)
{
    // No linkages as this is a single phase problem
}

////////////////////////////////////
//////////////////////////////////// Define the main routine ///////////////////////////////////
////////////////////////////////////

int main(void)
{
    ///////////////////////////////////
    /////////////////////////////////// Declare key structures ///////////////////////////////////
    ///////////////////////////////////

    Alg algorithm;
    Sol solution;
    Prob problem;

    ///////////////////////////////////
    /////////////////////////////////// Register problem name ///////////////////////////////////
    ///////////////////////////////////

    problem.name          = "Brachistochrone Problem";
    problem.outfilename    = "brac1.txt";

    ///////////////////////////////////
    /////////////////////////////////// Define problem level constants & do level 1 setup ///////////////////////////////////
    ///////////////////////////////////

    problem.nphases       = 1;
    problem.nlinkages      = 0;

    psopt_level1_setup(problem);

    ///////////////////////////////////
    /////////////////////////////////// Define phase related information & do level 2 setup ///////////////////////////////////
    ///////////////////////////////////

    problem.phases(1).nstates   = 3;
    problem.phases(1).ncontrols = 1;
    problem.phases(1).nevents   = 5;
    problem.phases(1).npath     = 0;
    problem.phases(1).nodes     << 40;

    psopt_level2_setup(problem, algorithm);

    ///////////////////////////////////
    /////////////////////////////////// Enter problem bounds information ///////////////////////////////////
    ///////////////////////////////////

    problem.phases(1).bounds.lower.states << 0, 0, 0;

```

```

problem.phases(1).bounds.upper.states << 20, 20, 20;

problem.phases(1).bounds.lower.controls << 0.0;
problem.phases(1).bounds.upper.controls << 2*pi;

problem.phases(1).bounds.lower.events << 0, 0, 0, 2, 2;
problem.phases(1).bounds.upper.events << 0, 0, 0, 2, 2;

problem.phases(1).bounds.lower.StartTime = 0.0;
problem.phases(1).bounds.upper.StartTime = 0.0;

problem.phases(1).bounds.lower.EndTime = 0.0;
problem.phases(1).bounds.upper.EndTime = 10.0;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Register problem functions %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

problem.integrand_cost = &integrand_cost;
problem.endpoint_cost = &endpoint_cost;
problem.dae = &dae;
problem.events = &events;
problem.linkages = &linkages;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% set scaling factors (optional) %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

// problem.phases(1).scale.controls = 1.0*ones(1,1);
// problem.phases(1).scale.states = 1.0*ones(3,1);
// problem.phases(1).scale.events = 1.0*ones(5,1);
// problem.phases(1).scale.defects = 1.0*ones(3,1);
// problem.phases(1).scale.time = 1.0;

// problem.scale.objective = 1.0;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Define & register initial guess %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

MatrixXd x0(3,20);

x0.row(0) = linspace(0.0,1.0, 20);
x0.row(1) = linspace(0.0,1.0, 20);
x0.row(2) = linspace(0.0,1.0, 20);

problem.phases(1).guess.controls = ones(1,20);
problem.phases(1).guess.states = x0;
problem.phases(1).guess.time = linspace(0.0, 2.0, 20);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Enter algorithm options %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

algorithm.nlp_method = "IPOPT";
algorithm.scaling = "automatic";
algorithm.derivatives = "automatic";
algorithm.nlp_iter_max = 1000;
algorithm.nlp_tolerance = 1.e-6;
// algorithm.hessian = "exact";
algorithm.collocation_method = "Legendre";
// algorithm.mesh_refinement = "automatic";

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Now call PSOPT to solve the problem %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if (psopt(solution, problem, algorithm) != 0) return 1;

if (solution.error_flag) exit(0);

```

```

////////////////////////////////////
////////// Extract relevant variables from solution structure //////////
////////////////////////////////////

MatrixXd x   = solution.get_states_in_phase(1);
MatrixXd u   = solution.get_controls_in_phase(1);
MatrixXd t   = solution.get_time_in_phase(1);
MatrixXd H   = solution.get_dual_hamiltonian_in_phase(1);
MatrixXd lambda = solution.get_dual_costates_in_phase(1);

////////////////////////////////////
////////// Save solution data to files if desired //////////
////////////////////////////////////

Save(x,"x.dat");
Save(u,"u.dat");
Save(t,"t.dat");
Save(lambda,"p.dat");

////////////////////////////////////
////////// Plot some results if desired (requires gnuplot) //////////
////////////////////////////////////

plot(t,x,problem.name + ": states", "time (s)", "states", "x y v");

plot(t,u,problem.name + ": control", "time (s)", "control", "u");

plot(t,H,problem.name + ": Hamiltonian", "time (s)", "H", "H");

plot(t,lambda,problem.name + ": costates", "time (s)", "lambda", "lambda_1 lambda_2 lambda_3");

plot(t,x,problem.name + ": states", "time (s)", "states", "x y v",
      "pdf", "brac1_states.pdf");

plot(t,u,problem.name + ": control", "time (s)", "control", "u",
      "pdf", "brac1_control.pdf");

plot(t,H,problem.name + ": Hamiltonian", "time (s)", "H", "H",
      "pdf", "brac1_hamiltonian.pdf");

plot(t,lambda,problem.name + ": costates", "time (s)", "lambda", "lambda_1 lambda_2 lambda_3",
      "pdf", "brac1_costates.pdf");

}

////////////////////////////////////
////////// END OF FILE //////////
////////////////////////////////////

```

3 Two-phase Schwartz problem: linking phases

A problem is split into phases when something about it changes at a known instant: a stage separates, a constraint switches on, a model is replaced. PSOPT treats each phase as a separate optimal control problem with its own states, controls, mesh and constraints, and joins them with *linkage constraints*, which the user writes in the `linkages` function. This example, due to Schwartz and used as a test problem by Rutquist and Edvall [15], is the smallest one that shows the mechanism working.

The dynamics are the same in both phases, a damped double integrator whose damping grows with position, and the split is made at $t = 1$ because the constraints, not the model, change there. In the first phase the state must stay outside an ellipse in the (x_1, x_2) plane and the control is limited to $|u| \leq 1$; in the second the ellipse is gone and the control limit is relaxed. The cost is a terminal penalty on the distance of the state

from the origin at $t = 2.9$, so the problem is to get to the origin while avoiding the ellipse on the way.

The three linkage constraints are supplied by the helper `auto_link`, which imposes continuity of every state and of time across the junction: it is the common case, and writing it out by hand is only necessary when the states of the two phases differ in number or in meaning, as in a launch vehicle that sheds a stage.

Two things in the results deserve comment. The terminal cost comes out at 3.1×10^{-16} , that is, the origin is reached to within the tolerance of the NLP solver, which is what the published solution reports. But the maximum relative local error in the second phase is 1.3×10^{-4} , five orders of magnitude larger than in the brachistochrone, and this is not a defect of the solver: with the control limit relaxed to ± 50 the optimal control has a sharp transient at the start of the phase, reaching about -6.9 within the first few nodes, and a fixed mesh of 60 nodes cannot resolve it. This is exactly the situation that automatic mesh refinement exists for; the Manutec example of Section 14 and the minimum time to climb of Section 15 show what it does with a problem whose solution is hard on a fixed grid.

Results

```
PSOPT results summary
=====

Problem:      Two phase Schwartz problem
CPU time (seconds):  2.198800e-02
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  7.259593e-16
Phase 1 endpoint cost function value:    0.000000e+00
Phase 1 integrated part of the cost:      0.000000e+00
Phase 1 initial time:                    0.000000e+00
Phase 1 final time:                      1.000000e+00
Phase 1 maximum relative local error:    2.672110e-06
Phase 2 endpoint cost function value:    7.259593e-16
Phase 2 integrated part of the cost:      0.000000e+00
Phase 2 initial time:                    1.000000e+00
Phase 2 final time:                      2.900000e+00
Phase 2 maximum relative local error:    1.279942e-04
NLP solver reports:      The problem has been solved!
```

Source

The listing below shows how a two-phase problem differs from a single-phase one: `problem.nphases` and `problem.nlinkages` are set before `psopt_level1_setup`, the

Table 2: Two-phase Schwartz problem: summary

<i>Demonstrates</i>	Multiple phases, linkage constraints via <code>auto_link</code> , a nonlinear path constraint active in one phase only, different bounds in different phases
<i>Reference</i>	Rutquist and Edvall [15]
<i>Dimensions</i>	2 phases, 3 linkages; phase 1: 2 states, 1 control, 2 events, 1 path constraint; phase 2: 2 states, 1 control, no events, no path constraints
<i>Objective</i>	$J = 5 (x_1(t_f)^2 + x_2(t_f)^2)$, evaluated at the end of phase 2
<i>Dynamics</i> (both phases)	$\dot{x}_1 = x_2, \quad \dot{x}_2 = u - 0.1 (1 + 2x_1^2) x_2$
<i>Initial conditions</i>	$x_1(0) = x_2(0) = 1$
<i>Terminal conditions</i>	None; the terminal state is penalised, not constrained
<i>Path constraint</i> (phase 1)	$1 - 9(x_1 - 1)^2 - \left(\frac{x_2 - 0.4}{0.3} \right)^2 \leq 0$
<i>Linkages</i>	Continuity of both states and of time at $t = 1$
<i>Bounds</i>	Phase 1: $-0.8 \leq x_2, u \leq 1$; phase 2: $-10 \leq x_2, u \leq 50$; both: $-20 \leq x_1 \leq 10, x_2 \leq 10$
<i>Phase intervals</i>	$t \in [0, 1]$ and $t \in [1, 2.9]$, both fixed
<i>Discretisation</i>	Hermite-Simpson, 60 nodes per phase, automatic differentiation, automatic scaling, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	The origin is reachable, so $J^* = 0$
<i>Computed</i>	$J = 3.1 \times 10^{-16}$; maximum relative local error 2.7×10^{-6} in phase 1 and 1.3×10^{-4} in phase 2
<i>Source</i>	<code>examples/twophase_schwartz/twophase_schwartz.cxx</code> , listed below

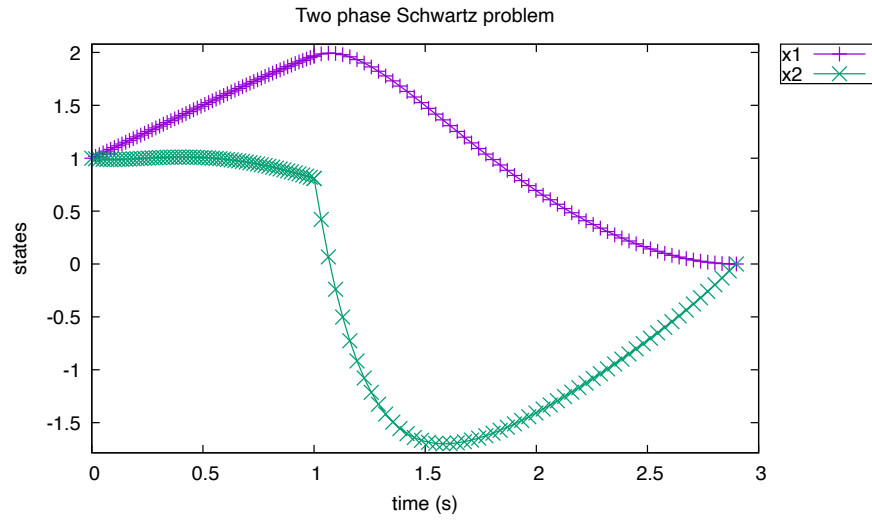


Figure 3: States for the two-phase Schwartz problem; the junction between the phases is at $t = 1$

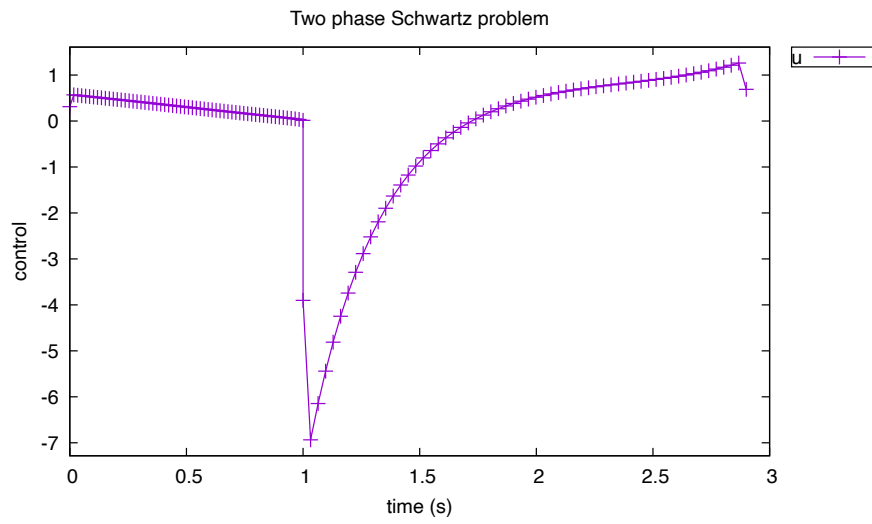


Figure 4: Control for the two-phase Schwartz problem

dimensions and bounds are given per phase, the user functions receive an `iphase` argument and branch on it, and the `linkages` function joins the phases.

```

////////////////////////////////////
//////////////////////////////////// twophase_schwartz.cxx ///////////////////////////////////
//////////////////////////////////// PSOPT Example ///////////////////////////////////
////////////////////////////////////
//////////////////////////////////// Title: Two phase Schwartz problem ///////////////////////////////////
//////////////////////////////////// Last modified: 09 January 2009 ///////////////////////////////////
//////////////////////////////////// Reference: PROPT Users Guide ///////////////////////////////////
//////////////////////////////////// (See PSOPT handbook for full reference) ///////////////////////////////////
//////////////////////////////////// Copyright (c) Victor M. Becerra, 2009 ///////////////////////////////////
//////////////////////////////////// This is part of the PSOPT software library, which ///////////////////////////////////
//////////////////////////////////// is distributed under the terms of the GNU Lesser ///////////////////////////////////
//////////////////////////////////// General Public License (LGPL) ///////////////////////////////////
////////////////////////////////////

#include "psopt.h"

using namespace std;

////////////////////////////////////
//////////////////////////////////// Define the end point (Mayer) cost function ///////////////////////////////////
////////////////////////////////////

adouble endpoint_cost(adouble* initial_states, adouble* final_states,
                    adouble* parameters, adouble& t0, adouble& tf,
                    adouble* xad, int iphase, Workspace* workspace)
{
    adouble retval;
    adouble x1f, x2f;

    // Phase 1 cost

    if ( iphase == 1 )
    {
        retval = 0.0;
    }

    // Phase 2 cost

    if( iphase == 2 ) {

        x1f = final_states[0];
        x2f = final_states[1];

        retval = 5*(x1f*x1f + x2f*x2f);
    }

    return retval;
}

////////////////////////////////////
//////////////////////////////////// Define the integrand (Lagrange) cost function ///////////////////////////////////
////////////////////////////////////

adouble integrand_cost(adouble* states, adouble* controls,
                    adouble* parameters, adouble& time, adouble* xad,
                    int iphase, Workspace* workspace)
{
    return 0.0;
}

////////////////////////////////////
//////////////////////////////////// Define the DAE's ///////////////////////////////////
////////////////////////////////////

void dae(adouble* derivatives, adouble* path, adouble* states,
        adouble* controls, adouble* parameters, adouble& time,

```

```

        adouble* xad, int iphase, Workspace* workspace)
{
    adouble x1 = states[0];
    adouble x2 = states[1];

    adouble u = controls[0];

    derivatives[0] = x2;
    derivatives[1] = u-0.1*(1+2*x1*x1)*x2;

    if(iphase==1) {
        path[0] = 1.0 - 9.0*((x1-1.0)*(x1-1.0))-((x2-0.4)*(x2-0.4))/(0.3*0.3);
    }
}

////////////////////////////////////
//////////////////////////////////// Define the events function ///////////////////////////////////
////////////////////////////////////

void events(adouble* e, adouble* initial_states, adouble* final_states,
            adouble* parameters, adouble& t0, adouble& tf, adouble* xad,
            int iphase, Workspace* workspace)
{
    adouble x1i = initial_states[0];
    adouble x2i = initial_states[1];

    if (iphase==1) {
        e[0] = x1i;
        e[1] = x2i;
    }
}

////////////////////////////////////
//////////////////////////////////// Define the phase linkages function ///////////////////////////////////
////////////////////////////////////

void linkages( adouble* linkages, adouble* xad, Workspace* workspace)
{
    int index = 0;
    auto_link( linkages, &index, xad, 1, 2, workspace );
}

////////////////////////////////////
//////////////////////////////////// Define the main routine ///////////////////////////////////
////////////////////////////////////

int main(void)
{
    //////////////////////////////////////
    ////////////////////////////////////// Declare key structures ///////////////////////////////////
    //////////////////////////////////////

    Alg algorithm;
    Sol solution;
    Prob problem;

    //////////////////////////////////////
    ////////////////////////////////////// Register problem name ///////////////////////////////////
    //////////////////////////////////////

    problem.name          = "Two phase Schwartz problem";
    problem.outfilename    = "twophsc.txt";

    //////////////////////////////////////
    ////////////////////////////////////// Define problem level constants & do level 1 setup ///////////////////////////////////
    //////////////////////////////////////

    problem.nphases       = 2;
    problem.nlinkages     = 3;

    psopt_level1_setup(problem);
}

```

```

////////////////////////////////////
//////// Define phase related information & do level 2 setup //////////
////////////////////////////////////

    problem.phases(1).nstates = 2;
    problem.phases(1).ncontrols = 1;
    problem.phases(1).nevents = 2;
    problem.phases(1).npath = 1;

    problem.phases(2).nstates = 2;
    problem.phases(2).ncontrols = 1;
    problem.phases(2).nevents = 0;
    problem.phases(2).npath = 0;

    problem.phases(1).nodes << 60;
    problem.phases(2).nodes << 60;

    psopt_level2_setup(problem, algorithm);

////////////////////////////////////
//////// Enter problem bounds information //////////
////////////////////////////////////

    double x1L = -20.0;
    double x2L_phase1 = -0.8;
    double x2L_phase2 = -10.0;
    double uL = -1.0;
    double x1U = 10.0;
    double x2U = 10.0;
    double uU = 1.0;
    double hL = -100.0;
    double hU = 0.0;

    // Phase 0 bounds

    problem.phases(1).bounds.lower.states(0) = x1L;
    problem.phases(1).bounds.lower.states(1) = x2L_phase1;

    problem.phases(1).bounds.upper.states(0) = x1U;
    problem.phases(1).bounds.upper.states(1) = x2U;

    problem.phases(1).bounds.lower.controls(0) = uL;
    problem.phases(1).bounds.upper.controls(0) = uU;

    problem.phases(1).bounds.lower.events(0) = 1.0;
    problem.phases(1).bounds.lower.events(1) = 1.0;

    problem.phases(1).bounds.upper.events(0) = 1.0;
    problem.phases(1).bounds.upper.events(1) = 1.0;

    problem.phases(1).bounds.lower.path(0) = hL;
    problem.phases(1).bounds.upper.path(0) = hU;

    problem.phases(1).bounds.lower.StartTime = 0.0;
    problem.phases(1).bounds.upper.StartTime = 0.0;

    problem.phases(1).bounds.lower.EndTime = 1.0;
    problem.phases(1).bounds.upper.EndTime = 1.0;

    // Phase 1 bounds

    problem.phases(2).bounds.lower.states(0) = x1L;
    problem.phases(2).bounds.lower.states(1) = x2L_phase2;

    problem.phases(2).bounds.upper.states(0) = x1U;
    problem.phases(2).bounds.upper.states(1) = x2U;

    problem.phases(2).bounds.lower.controls(0) = -50.0;
    problem.phases(2).bounds.upper.controls(0) = 50.0;

    problem.phases(2).bounds.lower.StartTime = 1.0;
    problem.phases(2).bounds.upper.StartTime = 1.0;

    problem.phases(2).bounds.lower.EndTime = 2.9;
    problem.phases(2).bounds.upper.EndTime = 2.9;

////////////////////////////////////
//////// Register problem functions //////////
////////////////////////////////////

```

```

////////////////////////////////////
problem.integrand_cost = &integrand_cost;
problem.endpoint_cost = &endpoint_cost;
problem.dae = &dae;
problem.events = &events;
problem.linkages = &linkages;

////////////////////////////////////
//////////////////////////////////// Define & register initial guess ///////////////////////////////////
////////////////////////////////////

int iphase;

MatrixXd u0(1,40);
MatrixXd x0(2,40);

MatrixXd time_guess0 = linspace(0.0, 1.0 , 40);
MatrixXd time_guess1 = linspace(1.0, 2.9 , 40);

iphase = 1;

u0 = zeros(1,40);
x0.row(0) = linspace(1.0,1.0, 40);
x0.row(1) = linspace(1.0,1.0, 40);

problem.phases(iphase).guess.controls = u0;
problem.phases(iphase).guess.states = x0;
problem.phases(iphase).guess.time = time_guess0;

iphase = 2;

u0 = zeros(1,40);
x0.row(0) = linspace(1.0,1.0, 40);
x0.row(1) = linspace(1.0,1.0, 40);

problem.phases(iphase).guess.controls = u0;
problem.phases(iphase).guess.states = x0;
problem.phases(iphase).guess.time = time_guess1;

////////////////////////////////////
//////////////////////////////////// Enter algorithm options ///////////////////////////////////
////////////////////////////////////

algorithm.nlp_method = "IPOPT";
algorithm.scaling = "automatic";
algorithm.derivatives = "automatic";
algorithm.collocation_method = "Hermite-Simpson";
algorithm.nlp_iter_max = 1000;
algorithm.nlp_tolerance = 1.e-6;

////////////////////////////////////
//////////////////////////////////// Now call PSOPT to solve the problem ///////////////////////////////////
////////////////////////////////////

if (psopt(solution, problem, algorithm) != 0) return 1;

////////////////////////////////////
//////////////////////////////////// Extract relevant variables from solution structure ///////////////////////////////////
////////////////////////////////////

MatrixXd xphase1 = solution.get_states_in_phase(1);
MatrixXd uphase1 = solution.get_controls_in_phase(1);
MatrixXd tphase1 = solution.get_time_in_phase(1);

MatrixXd xphase2 = solution.get_states_in_phase(2);
MatrixXd uphase2 = solution.get_controls_in_phase(2);
MatrixXd tphase2 = solution.get_time_in_phase(2);

MatrixXd x(xphase1.rows(), xphase1.cols()+xphase2.cols());

x << xphase1 , xphase2;

MatrixXd u(uphase1.rows(), uphase1.cols()+uphase2.cols());

u << uphase1 , uphase2;

```

```

MatrixXd t(tphase1.rows(), tphase1.cols()+tphase2.cols());

t << tphase1 , tphase2;

/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
// Save solution data to files if desired //
/////////////////////////////////////////////////////////////////

Save(x,"x.dat");
Save(u,"u.dat");
Save(t,"t.dat");

/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
// Plot some results if desired (requires gnuplot) //
/////////////////////////////////////////////////////////////////

plot(t,x,problem.name, "time (s)", "states", "x1 x2");

plot(t,u,problem.name, "time (s)", "control", "u");

plot(t,x,problem.name, "time (s)", "states", "x1 x2",
      "pdf", "twophsc_states.pdf");

plot(t,u,problem.name, "time (s)", "control", "u",
      "pdf", "twophsc_control.pdf");

}

/////////////////////////////////////////////////////////////////
//                               END OF FILE                               //
/////////////////////////////////////////////////////////////////

```

4 Catalytic cracking of gas oil: parameter estimation

Estimating the parameters of a dynamic model from measurements is not a different kind of problem from optimal control: it is an optimal control problem whose decision variables are static parameters and not a control history, and whose objective is a sum of squared residuals between the model's outputs and the data. PSOPT supports it directly. The user declares how many observations there are and at what times they were taken, supplies them, and writes an **observation_function** that says which combination of states, controls and parameters each measurement corresponds to; the least-squares objective is then formed internally and there is no need to write an integrand or endpoint cost at all.

The problem used here is the catalytic cracking of gas oil, from the COPS collection [5]. Gas oil (species A) cracks to gasoline (species Q) and to lighter products and coke (species S), by the reaction network $A \rightarrow Q$, $A \rightarrow S$, $Q \rightarrow S$, in which the first two steps are second order in the concentration of A and the third is first order in the concentration of Q . The three rate constants $\theta_1, \theta_2, \theta_3$ are to be estimated from 21 samples of the two concentrations. The problem is a standard test case because the data are few, the model is nonlinear in the parameters, and the third rate constant is poorly determined, which is the point of interest here.

That last point is what makes the example one to read as well as to run. The estimate $\hat{\theta}_3 \approx 1.0$ arrives with a 95 per cent confidence interval of roughly $[0.32, 1.69]$: the data constrain it to within a factor of five, and a report of the estimate alone, without

the interval, would be misleading. The correlation matrix explains why. The three parameters are strongly correlated ($r_{12} = 0.79$, $r_{13} = -0.84$, $r_{23} = -0.87$), so the data determine certain combinations of them far better than they determine any one of them; a change in θ_1 can be compensated almost exactly by changes in θ_2 and θ_3 . Setting `algorithm.parameter_statistics = "yes"` makes PSOPT compute and report the covariance, the correlations, the residual standard deviation and the confidence limits, and for an estimation problem these should be regarded as part of the answer and not as optional extras. The statistics are computed by the method of Kostina [8], from the null space of the active constraints at the solution.

The initial conditions $y(0) = (1, 0)$ are those of the published problem and are imposed here as event constraints. They are not free parameters of the fit: the experiment starts with pure gas oil and no gasoline, and this is known rather than measured.

Table 3: Catalytic cracking of gas oil: summary

<i>Demonstrates</i>	Parameter estimation: observations, sampling instants, <code>observation_function</code> , and the reporting of covariance, correlations and confidence limits
<i>Reference</i>	Dolan and Moré, COPS [5]; statistics after Kostina [8]
<i>Dimensions</i>	1 phase; 2 states, no controls, 3 static parameters, 2 events, no path constraints; 2 observed variables at 21 sampling instants
<i>Objective</i>	$J = \sum_{i=1}^{21} [(y_1(t_i) - y_{m,1}(i))^2 + (y_2(t_i) - y_{m,2}(i))^2]$, formed internally from the observations
<i>Dynamics</i>	$\dot{y}_1 = -(\theta_1 + \theta_3)y_1^2$, $\dot{y}_2 = \theta_1 y_1^2 - \theta_2 y_2$
<i>Observation function</i>	$g(x, u, p, t) = [y_1 \ y_2]^T$, unit residual weights
<i>Initial conditions</i>	$y_1(0) = 1$, $y_2(0) = 0$
<i>Bounds</i>	$0 \leq y_1, y_2 \leq 2$, $0 \leq \theta_j \leq 20$, $t \in [0, 0.95]$ fixed
<i>Data</i>	21 sampling instants and 42 measurements, listed in the source
<i>Discretisation</i>	Hermite-Simpson, 80 nodes, automatic differentiation, automatic scaling, NLP tolerance 10^{-6} , <code>parameter_statistics = "yes"</code>
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	COPS reports a residual of 5.23659×10^{-3} . Solving the same problem outside PSOPT by nonlinear least squares with an accurate ODE solver gives $\theta = (11.847, 8.344, 1.001)$
<i>Computed</i>	$J = 5.228971 \times 10^{-3}$, $\hat{\theta} = (11.848, 8.345, 1.001)$, $\hat{\sigma} = 1.158 \times 10^{-2}$
<i>Source</i>	<code>examples/cracking/cracking.cxx</code> , listed below

Results

PSOPT results summary
=====

```

Problem:      Catalytic cracking of gas oil
CPU time (seconds):  5.680900e-02
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  5.228971e-03
Phase 1 endpoint cost function value:  5.228971e-03
Phase 1 integrated part of the cost:    0.000000e+00
Phase 1 initial time:    0.000000e+00
Phase 1 final time:      9.500000e-01
Phase 1 maximum relative local error:  4.273984e-06
NLP solver reports:      The problem has been solved!

```

The parameter statistics reported by PSOPT are reproduced below. The confidence limits are the ones to read first, and the correlation matrix immediately after them: the diagonal of the covariance on its own would suggest three independently determined parameters, which these are not.

```

***** PHASE 1 PARAMETERS *****

1.184812e+01
8.345197e+00
1.000612e+00
***** PARAMETER STATISTICS *****
>>>> Parameter covariance matrix (all phases)
1.064561e-01  7.886537e-02  -9.611805e-02
7.886537e-02  9.461555e-02  -9.345354e-02
-9.611805e-02  -9.345354e-02  1.219091e-01

>>>> Rank of parameter covariance matrix: 3

>>>> Parameter correlation matrix (all phases)
1.000000e+00  7.858141e-01  -8.437264e-01
7.858141e-01  1.000000e+00  -8.701547e-01
-8.437264e-01  -8.701547e-01  1.000000e+00

>>>> Estimated residual standard deviation (sigma_hat): 1.157913e-02
>>>> Scalar observations (N_s): 42, degrees of freedom of the NLP (n_f): 3
>>>> Sum of squared weighted residuals at the solution: 5.228971e-03

>>> 95 percent statistical confidence limits on estimated parameters
Phase Parameter (Low Confidence Limit) (Value) (High Confidence Limit)
1 1  1.118806e+01  1.184812e+01  1.250817e+01
1 2  7.722930e+00  8.345197e+00  8.967465e+00
1 3  2.942715e-01  1.000612e+00  1.706952e+00

```

Source

The listing below includes the measurement vectors and the sampling instants, so it also serves as a template for the mechanics of getting data into a PSOPT estimation problem: `nobserved` and `nsamples` are set with the other phase dimensions, `observation_nodes`, `observations` and `residual_weights` are matrices, and `observation_function` is registered alongside the other user functions.

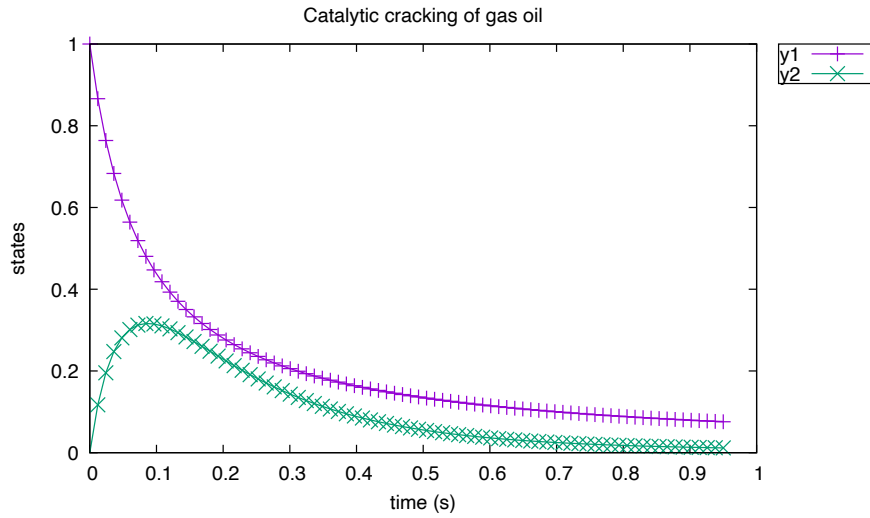


Figure 5: Fitted concentrations and the measurements for the catalytic cracking problem

```

////////////////////////////////////
////////////////////////////////////      cracking.cxx      ///////////////////////////////////
////////////////////////////////////
////////////////////////////////////      PSOPT Example      ///////////////////////////////////
////////////////////////////////////
////////////////////////////////////
//////////////////////////////////// Title: Catalytic Cracking of Gas Oil      ///////////////////////////////////
//////////////////////////////////// Last modified: 06 September 2026      ///////////////////////////////////
//////////////////////////////////// Reference: User's guide for DIRCOL      ///////////////////////////////////
//////////////////////////////////// (See PSOPT handbook for full reference) ///////////////////////////////////
//////////////////////////////////// Copyright (c) Victor M. Becerra, 2009      ///////////////////////////////////
//////////////////////////////////// This is part of the PSOPT software library, which ///////////////////////////////////
//////////////////////////////////// is distributed under the terms of the GNU Lesser ///////////////////////////////////
//////////////////////////////////// General Public License (LGPL)      ///////////////////////////////////
////////////////////////////////////
//
// Run with no arguments, this estimates the three rate constants and prints
// the 95 per cent confidence intervals PSOPT computes from the linearized
// (Wald) covariance of the estimates.
//
// Run with arguments, it additionally computes a PROFILE LIKELIHOOD for one
// of the parameters:
//
//      cracking <index> [lo] [hi] [ngrid] [nobserved]
//
//      index      1, 2 or 3: which parameter to profile
//      lo, hi     the range of that parameter to scan   (default 0 to 2.2)
//      ngrid      number of grid points                 (default 45)
//      nobserved  1 fits y1 only, 2 fits y1 and y2      (default 2)
//
// The profile is obtained by fixing the chosen parameter at each grid value,
// re-estimating the others, and recording the optimal sum of squares J(theta).
// Fixing a parameter needs no special support: its lower and upper bounds are
// set to the same number. The 95 per cent profile-likelihood interval is the
// set of values for which
//
//      J(theta) <= J* [ 1 + t^2 / (Ns - nf) ],    t = t^{0.975}_{(Ns-nf)}
//
// which is the standard F-test threshold for one parameter. Unlike the Wald
// interval it needs no assumption that the model is linear near the estimate,
// and it cannot return an interval for a parameter the data do not determine.
//
// Two runs are worth comparing:
//

```

```

//      ./cracking 3          profile theta3 with both variables observed
//      ./cracking 3 0 8 17 1  profile theta3 with only y1 observed
//
// In the second, theta1 and theta3 enter the y1 equation only through their
// sum, so they are structurally non-identifiable: the profile is flat, and
// theta1 moves to hold theta1 + theta3 fixed. The Wald interval, computed at
// the same solution, is finite, narrow, and includes negative rate constants.
// That contrast is the reason for computing a profile at all.
//
////////////////////////////////////

#include "psopt.h"

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>

using namespace std;
using namespace PSOPT;

////////////////////////////////////
//////////////////////////////////// Problem configuration //////////////////////////////////////
////////////////////////////////////

// Number of observed variables: 2 fits y1 and y2, 1 fits y1 alone. Set from
// the command line; the observation function has to see it, so it is global.
static int NOBSERVED = 2;

////////////////////////////////////
//////////////////////////////////// Define the observation function //////////////////////////////////////
////////////////////////////////////

void observation_function( adouble* observations,
                          adouble* states, adouble* controls,
                          adouble* parameters, adouble& time, int k,
                          adouble* xad, int iphase, Workspace* workspace)
{
    observations[ 0 ] = states[ 0 ];
    if ( NOBSERVED > 1 ) observations[ 1 ] = states[ 1 ];
}

////////////////////////////////////
//////////////////////////////////// Define the DAE's //////////////////////////////////////
////////////////////////////////////

void dae(adouble* derivatives, adouble* path, adouble* states,
         adouble* controls, adouble* parameters, adouble& time,
         adouble* xad, int iphase, Workspace* workspace)
{
    adouble y1 = states[0];
    adouble y2 = states[1];

    adouble theta1 = parameters[ 0 ];
    adouble theta2 = parameters[ 1 ];
    adouble theta3 = parameters[ 2 ];

    derivatives[0] = -(theta1 + theta3)*y1*y1;
    derivatives[1] = theta1*y1*y1 - theta2*y2;
}

////////////////////////////////////
//////////////////////////////////// Define the events function //////////////////////////////////////
////////////////////////////////////

void events(adouble* e, adouble* initial_states, adouble* final_states,
           adouble* parameters, adouble& t0, adouble& tf, adouble* xad,
           int iphase, Workspace* workspace)
{
    // The published problem (COPS gas-oil cracking) fixes y(0) = (1,0);
    // estimating the initial conditions as well makes this a different problem.
    e[0] = initial_states[0];
    e[1] = initial_states[1];
}

////////////////////////////////////
//////////////////////////////////// Define the phase linkages function //////////////////////////////////////
////////////////////////////////////

```

```

////////////////////////////////////
void linkages( adouble* linkages, adouble* xad, Workspace* workspace)
{
    // No linkages as this is a single phase problem
}

////////////////////////////////////
//////////////////////////////////// Problem setup, done once //////////////////////////////////
////////////////////////////////////

// Everything that does not change from one solve to the next. The profile
// below then changes exactly two numbers per point -- the lower and the upper
// bound of the parameter being profiled -- and calls psopt() again.

void setup_cracking(Prob& problem, Alg& algorithm)
{
    MatrixXd y1meas(1,21), y2meas(1,21), tmeas(1,21);
    // Measured values of y1
    y1meas << 1.0,0.8105,0.6208,0.5258,0.4345,0.3903,0.3342,0.3034, \
              0.2735,0.2405,0.2283,0.2071,0.1669,0.153,0.1339,0.1265, \
              0.12,0.099,0.087,0.077,0.069;
    // Measured values of y2
    y2meas << 0.0,0.2,0.2886,0.301,0.3215,0.3123,0.2716,0.2551,0.2258, \
              0.1959,0.1789,0.1457,0.1198,0.0909,0.0719,0.0561,0.046, \
              0.028,0.019,0.014,0.01;
    // Sampling instants
    tmeas << 0.0,0.025,0.05,0.075,0.1,0.125,0.15,0.175,0.2,0.225,0.25, \
            0.3,0.35,0.4,0.45,0.5,0.55,0.65,0.75,0.85,0.95;

    //////////////////////////////////
    ////////////////////////////////// Register problem name //////////////////////////////////
    //////////////////////////////////

    problem.name          = "Catalytic cracking of gas oil";
    problem.outfilename    = "cracking.txt";

    //////////////////////////////////
    ////////////////////////////////// Define problem level constants & do level 1 setup //////////////////////////////////
    //////////////////////////////////

    problem.nphases       = 1;
    problem.nlinkages      = 0;

    psopt_level1_setup(problem);

    //////////////////////////////////
    ////////////////////////////////// Define phase related information & do level 2 setup //////////////////////////////////
    //////////////////////////////////

    problem.phases(1).nstates    = 2;
    problem.phases(1).ncontrols  = 0;
    problem.phases(1).nevents    = 2;
    problem.phases(1).npath      = 0;
    problem.phases(1).nparameters = 3;
    problem.phases(1).nodes      << 80;
    problem.phases(1).nobserved   = NOBSERVED;
    problem.phases(1).nsamples    = 21;

    psopt_level2_setup(problem, algorithm);

    //////////////////////////////////
    ////////////////////////////////// Enter estimation information //////////////////////////////////
    //////////////////////////////////

    MatrixXd observations(NOBSERVED, 21);

    if ( NOBSERVED > 1 ) observations << y1meas, y2meas;
    else observations << y1meas;

    problem.phases(1).observation_nodes = tmeas;
    problem.phases(1).observations      = observations;
    problem.phases(1).residual_weights  = ones(NOBSERVED,21);

    //////////////////////////////////
    ////////////////////////////////// Enter problem bounds information //////////////////////////////////
    //////////////////////////////////

```

```

problem.phases(1).bounds.lower.states(0) = 0.0;
problem.phases(1).bounds.lower.states(1) = 0.0;

problem.phases(1).bounds.upper.states(0) = 2.0;
problem.phases(1).bounds.upper.states(1) = 2.0;

problem.phases(1).bounds.lower.parameters(0) = 0.0;
problem.phases(1).bounds.lower.parameters(1) = 0.0;
problem.phases(1).bounds.lower.parameters(2) = 0.0;
problem.phases(1).bounds.upper.parameters(0) = 20.0;
problem.phases(1).bounds.upper.parameters(1) = 20.0;
problem.phases(1).bounds.upper.parameters(2) = 20.0;

problem.phases(1).bounds.lower.events(0) = 1.0;
problem.phases(1).bounds.upper.events(0) = 1.0;
problem.phases(1).bounds.lower.events(1) = 0.0;
problem.phases(1).bounds.upper.events(1) = 0.0;

problem.phases(1).bounds.lower.StartTime = 0.0;
problem.phases(1).bounds.upper.StartTime = 0.0;

problem.phases(1).bounds.lower.EndTime = 0.95;
problem.phases(1).bounds.upper.EndTime = 0.95;

/////////////////////////////////////////////////////////////////
///////////////////////////////////////////////////////////////// Register problem functions ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

problem.dae = &dae;
problem.events = &events;
problem.linkages = &linkages;
problem.observation_function = &observation_function;

/////////////////////////////////////////////////////////////////
///////////////////////////////////////////////////////////////// Define & register initial guess ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

MatrixXd state_guess(2, 40);

state_guess.row(0) = linspace(1.0,0.069, 40);
state_guess.row(1) = linspace(0.30,0.01, 40);

problem.phases(1).guess.states = state_guess;
problem.phases(1).guess.time = linspace(0.0, 0.95, 40);
problem.phases(1).guess.parameters = zeros(3,1);

/////////////////////////////////////////////////////////////////
///////////////////////////////////////////////////////////////// Enter algorithm options ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

algorithm.nlp_method = "IPOPT";
algorithm.scaling = "automatic";
algorithm.derivatives = "automatic";
algorithm.collocation_method = "Hermite-Simpson";
algorithm.parameter_statistics = "yes";
algorithm.nlp_iter_max = 1000;
algorithm.nlp_tolerance = 1.e-6;
}

/////////////////////////////////////////////////////////////////
///////////////////////////////////////////////////////////////// What one solve reports ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

struct Fit {
    double J; // optimal sum of squared residuals
    double theta[3]; // estimated parameters
    double lo[3], hi[3]; // 95 per cent Wald limits, when available
    double sigma_hat; // estimated residual standard deviation
    long dof; // number of fitted quantities n_f
    long ns; // number of scalar observations N_s
    bool stats_ok;
    bool solved;
};

Fit read_fit(Sol& solution, int status)
{

```

```

Fit fit;
fit.solved = ( status == 0 && solution.error_flag == 0 );
fit.J      = solution.cost;

MatrixXd p = solution.get_parameters_in_phase(1);
for (int i = 0; i < 3; i++) fit.theta[i] = p(i,0);

fit.sigma_hat = solution.sigma_hat;
fit.dof        = solution.parameter_dof;
fit.ns         = solution.n_observations;
fit.stats_ok   = solution.parameter_statistics_ok
    && solution.parameter_confidence_low.rows() >= 3;
for (int i = 0; i < 3; i++) {
    fit.lo[i] = fit.stats_ok ? solution.parameter_confidence_low(i,0) : 0.0;
    fit.hi[i] = fit.stats_ok ? solution.parameter_confidence_high(i,0) : 0.0;
}
return fit;
}

// Fix a parameter, or release it. Fixing needs no special support: equal bounds
// remove it from the estimation and leave everything else untouched.
void fix_parameter(Prob& problem, int k, double value)
{
    problem.phases(1).bounds.lower.parameters(k) = value;
    problem.phases(1).bounds.upper.parameters(k) = value;
}

void release_parameter(Prob& problem, int k)
{
    problem.phases(1).bounds.lower.parameters(k) = 0.0;
    problem.phases(1).bounds.upper.parameters(k) = 20.0;
}

/////////////////////////////////////////////////////////////////
///////////////////////////////////////////////////////////////// Define the main routine ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

// What distinguishes one profile run's files from another's, and all of them from the
// nominal fit's. There are two profile runs in the book's regeneration -- theta3 against
// both observed states and theta3 against y1 alone -- so a single "profile" suffix would
// have them overwrite each other, which is the hazard examples/ecodriving met from the
// other direction when _w0.5 and _w0 collided after PSOPT trimmed the name at its first
// dot. These carry no dot before the extension.
// Writes into the caller's buffer rather than returning a pointer to a static one, so
// that two uses in the same expression cannot quietly become the same string.
static void profile_tag(char* tag, size_t n, int index, int nobserve)
{
    if (nobserve == 2) snprintf(tag, n, "_theta%d", index);
    else                snprintf(tag, n, "_theta%d_y%d", index, nobserve);
}

int main(int argc, char* argv[])
{
    int    index = (argc > 1) ? atoi(argv[1]) : 0;    // 0 means: no profile
    double lo    = (argc > 2) ? atof(argv[2]) : 0.0;
    double hi    = (argc > 3) ? atof(argv[3]) : 2.2;
    int    ngrid = (argc > 4) ? atoi(argv[4]) : 45;
    if (argc > 5) NOBSERVED = atoi(argv[5]);

    if (index < 0 || index > 3 || ngrid < 2 || NOBSERVED < 1 || NOBSERVED > 2) {
        printf("usage: %s <index 1..3> [lo] [hi] [ngrid] [nobserve 1|2]\n",
            argv[0]);
        return 1;
    }

    const bool profiling = (index > 0);

    Alg algorithm;
    Prob problem;
    setup_cracking(problem, algorithm);
    algorithm.print_level = profiling ? 0 : 1;

    // A profiling run must not write where the nominal fit writes. Every solve of the
    // profile -- and there are ngrid of them, plus the nominal one above -- reopens
    // problem.outfilename and the psolt_solution_<stem>.txt that PSOPT derives from it,
    // and with print_level 0 it leaves them empty. So an invocation of this example in
    // profile mode used to truncate cracking.txt and psolt_solution_cracking.txt to
    // nothing, destroying the fit's own summary -- which is the only file carrying the
    // parameter confidence limits, and so the reproducer for the book's table of them.
    // The book's regeneration runs the fit and then two profiles, in that order, and the
    // copy afterwards therefore found an empty file, declined to copy it (rightly: the

```

```

// guard exists to keep empty files out), and left the previous one in place. It had
// been stale for two days and through three regenerations that all reported success.
//
// This is the same rule examples/lts_costates learned in patch 145 and
// examples/ecodriving in patch 147: a sweep must not write where the nominal run
// writes. The profile is a sweep. It has its own names now.
if (profiling) {
    char tag[32], outname[64], probname[96];
    profile_tag(tag, sizeof tag, index, NOBSERVED);
    snprintf(outname, sizeof outname, "cracking_profile%s.txt", tag);
    snprintf(probname, sizeof probname,
        "Catalytic cracking of gas oil (profile likelihood%s)", tag);
    problem.outfilename = outname;
    problem.name = probname;
}

// The unconstrained fit, with all three parameters estimated. Its solution
// is kept in its own Sol for the whole run, because it is the starting
// point of both halves of the profile below.
Sol nominal;
Fit nom = read_fit(nominal, psopt(nominal, problem, algorithm));

printf("\n Estimated parameters\n");
for (int i = 0; i < 3; i++) printf("    theta%d = %12.6f\n", i+1, nom.theta[i]);

if (!profiling) {
    // Exactly the behaviour this example has always had.
    MatrixXd x = nominal.get_states_in_phase(1);
    MatrixXd t = nominal.get_time_in_phase(1);
    Save(x, "x.dat");
    Save(t, "t.dat");
    plot(t, x, "Catalytic cracking of gas oil", "time (s)", "states", "y1 y2");
    plot(t, x, "Catalytic cracking of gas oil", "time (s)", "states", "y1 y2",
        "pdf", "cracking_states.pdf");
    return 0;
}

////////////////////////////////////
//////////////////////////////////// Profile likelihood //////////////////////////////////
////////////////////////////////////

const int    k = index - 1;
const long   nu = nom.ns - nom.dof; // residual degrees of freedom
const double tq = inverse_twotailed_t_cdf(0.95, (int) nu);
const double thr = 1.0 + tq*tq/(double) nu; // threshold on J/J*

printf("\n Profile likelihood for theta%d\n", index);
printf("  observed variables : %d\n", NOBSERVED);
printf("    J*                : %.10e\n", nom.J);
printf("  sigma_hat           : %.6e\n", nom.sigma_hat);
printf("  Ns, nf, Ns-nf        : %ld, %ld, %ld\n", nom.ns, nom.dof, nu);
printf("  t*0.975_%ld          : %.4f\n", nu, tq);
printf("  threshold J/J*       : %.6f\n", thr);
if (nom.stats_ok)
    printf("    Wald 95%% interval : [%.6f, %.6f]\n", nom.lo[k], nom.hi[k]);
else
    printf("    Wald 95%% interval : not available\n");

MatrixXd grid(1, ngrid), ratio(1, ngrid);
vector<Fit> fits(ngrid);
for (int j = 0; j < ngrid; j++) grid(0,j) = lo + (hi - lo)*j/(double)(ngrid - 1);

// The grid point nearest the unconstrained estimate, from which the profile
// is traced outwards in both directions.
int jstart = 0;
for (int j = 1; j < ngrid; j++)
    if (fabs(grid(0,j) - nom.theta[k]) < fabs(grid(0,jstart) - nom.theta[k]))
        jstart = j;

// The profile is traced by CONTINUATION: each solve starts from the
// solution of the one before it, and each half starts from the
// unconstrained fit. Solving every point from the same cold guess instead
// does not fail -- it converges, reports success, and occasionally returns
// a different local minimum, which appears on a plot of the profile as
// structure in the likelihood that is not there. Two of seventeen points
// did exactly that on the y1-only run below, on a profile that is
// otherwise flat to two parts in 100,000.
Sol solution;

set_guess_from_solution(problem, nominal);
for (int j = jstart; j < ngrid; j++) {
    fix_parameter(problem, k, grid(0,j));

```

```

fits[j] = read_fit(solution, psopt(solution, problem, algorithm));
set_guess_from_solution(problem, solution);
}

set_guess_from_solution(problem, nominal);
for (int j = jstart - 1; j >= 0; j--) {
    fix_parameter(problem, k, grid(0,j));
    fits[j] = read_fit(solution, psopt(solution, problem, algorithm));
    set_guess_from_solution(problem, solution);
}
release_parameter(problem, k);

char dtag[32], fname[64];
profile_tag(dtag, sizeof dtag, index, NOBSERVED);
snprintf(fname, sizeof fname, "cracking_profile%s.dat", dtag);
FILE* fh = fopen(fname, "w");
fprintf(fh, "# profile likelihood, catalytic cracking of gas oil\n");
fprintf(fh, "# nobserve = %d, J* = %.10g, threshold J/J* = %.10g\n",
        NOBSERVED, nom.J, thr);
fprintf(fh, "# theta%d J J/J* theta1 theta2 theta3\n", index);

printf("\n%12s %18s %12s %10s %10s %10s\n",
        "theta", "J", "J/J*", "theta1", "theta2", "theta3");
for (int j = 0; j < ngrid; j++) {
    const Fit& f = fits[j];
    ratio(0,j) = f.J/nom.J;
    printf("%12.6f %18.10e %12.6f %10.5f %10.5f %10.5f\n",
            grid(0,j), f.J, f.J/nom.J, f.theta[0], f.theta[1], f.theta[2],
            f.solved ? "" : " NOT SOLVED");
    fprintf(fh, "%.10g %.10g %.10g %.10g %.10g %.10g\n",
            grid(0,j), f.J, f.J/nom.J, f.theta[0], f.theta[1], f.theta[2]);
}
fclose(fh);

// Where the profile crosses the threshold, by linear interpolation. A side
// that never crosses is reported as open: that is the signal that the data
// do not bound the parameter on that side, and it is exactly the case a
// Wald interval cannot express.
int jmin = 0;
for (int j = 1; j < ngrid; j++) if (ratio(0,j) < ratio(0,jmin)) jmin = j;
double plo = 0.0, phi = 0.0; bool haslo = false, hashi = false;
for (int j = 0; j < jmin; j++)
    if ((ratio(0,j) - thr)*(ratio(0,j+1) - thr) < 0.0) {
        double s = (thr - ratio(0,j))/(ratio(0,j+1) - ratio(0,j));
        plo = grid(0,j) + s*(grid(0,j+1) - grid(0,j)); haslo = true;
    }
for (int j = jmin; j < ngrid-1; j++)
    if ((ratio(0,j) - thr)*(ratio(0,j+1) - thr) < 0.0) {
        double s = (thr - ratio(0,j))/(ratio(0,j+1) - ratio(0,j));
        phi = grid(0,j) + s*(grid(0,j+1) - grid(0,j)); hashi = true;
    }

printf("\n Profile 95%% interval for theta%d: ", index);
if (haslo) printf("[%12.4f, ", plo); else printf("(open below %12.4f, ", lo);
if (hashi) printf("%12.4f]\n", phi); else printf("(open above %12.4f)\n", hi);
if (!haslo || !hashi)
    printf(" The profile does not cross the threshold on at least one side\n"
           " over the range scanned: the data do not bound theta%d there,\n"
           " whatever the Wald interval above may say.\n", index);
printf(" wrote %s\n", fname);

return 0;
}

////////////////////////////////////
//////////////////////////////////// END OF FILE //////////////////////////////////
////////////////////////////////////

```

5 Bryson's maximum range problem: an equality path constraint

A particle moves in a vertical plane under a constant tangential acceleration a and gravity g . Its position is (x, y) , its speed is v , and the direction in which it travels is set

by a two-component control (u_1, u_2) that must be a unit vector at every instant. The time available is fixed, the particle starts from rest at the origin, and it must arrive at a prescribed height; subject to that, it is to travel as far as possible in the x direction. The problem is from Bryson and Ho [4] and it is small enough to be read in a minute, which is why it is used here to introduce the one feature that the brachistochrone of Section 2 did not have: a constraint that must hold along the whole trajectory and not only at its ends.

That constraint is $u_1^2 + u_2^2 = 1$. In PSOPT a path constraint is a component of the `path` array filled in by the `dae` function, and it is made an equality in exactly the way an event is: by giving its lower and upper bound the same value. Here both are set to unity. Nothing else in the formulation confines the controls in any useful way: their box bounds are ± 10 , ten times larger than any value they can take. The path constraint is therefore the only thing that gives the two controls their meaning as a steering direction, and it is active at every collocation node by construction. Note that the supplied initial guess sets both controls to zero, which violates the constraint outright. That is permitted: the guess has to be a starting point for the NLP solver, not a feasible trajectory, and the interior-point method restores feasibility as it converges.

The solution has no switching structure to look for. Because the right-hand sides do not depend on x or y , the costates of those two states are constant, and the stationarity condition with respect to (u_1, u_2) , taken together with the path constraint, makes the control the unit vector parallel to $(\lambda_x v, \lambda_y v - g \lambda_v)$. The computed controls behave accordingly and turn smoothly and monotonically: u_2 begins at -1 , so the particle dives to build up speed, passes through zero shortly after $t = 1$, and ends positive as the trajectory is pulled back up to meet $y(t_f) = 0.1$; u_1 rises from zero to unity at the point where the motion is momentarily horizontal, then falls back. The height dips to about -0.37 before recovering and the speed peaks near 1.37 . The maximum range obtained is $x(t_f) = 1.712314$.

The accuracy deserves a plain word. The run uses trapezoidal collocation on a fixed mesh of 50 nodes with mesh refinement disabled, and the maximum relative local error is 2.08×10^{-5} , four orders of magnitude worse than the pseudospectral result of Section 2. That is expected and not alarming: the trapezoidal rule is second-order accurate, so its error falls only quadratically with node count, and the NLP tolerance here is 10^{-4} rather than the usual 10^{-6} . Enabling automatic mesh refinement, which the source file leaves commented out, is the remedy if more digits are wanted; the example is kept as it is because it shows what a plain fixed trapezoidal mesh actually delivers.

Results

PSOPT results summary

=====

Problem: Bryson Maximum Range Problem
CPU time (seconds): 1.622500e-02

Table 4: Bryson’s maximum range problem: summary

<i>Demonstrates</i>	An equality path constraint, imposed by setting equal lower and upper bounds on a component of the <code>path</code> array; trapezoidal collocation on a fixed mesh
<i>Reference</i>	Bryson and Ho [4]
<i>Dimensions</i>	1 phase; 3 states, 2 controls, 4 events, 1 path constraint, no static parameters
<i>Objective</i>	$J = -x(t_f)$ (maximise the range $x(t_f)$)
<i>Dynamics</i>	$\dot{x} = vu_1, \quad \dot{y} = vu_2, \quad \dot{v} = a - gu_2$
<i>Initial conditions</i>	$x(0) = y(0) = v(0) = 0, \quad t_0 = 0$
<i>Terminal conditions</i>	$y(t_f) = 0.1, \quad t_f = 2$ fixed
<i>Path constraints</i>	$u_1^2 + u_2^2 = 1$, an equality imposed with coincident bounds
<i>Bounds</i>	$-10 \leq x, y, v \leq 10, \quad -10 \leq u_1, u_2 \leq 10, \quad t_0 = 0, t_f = 2$
<i>Constants</i>	$g = 1, \quad a = 0.5g$
<i>Discretisation</i>	Trapezoidal, 50 nodes, automatic differentiation, automatic scaling, state-based defect scaling, limited-memory Hessian, NLP tolerance 10^{-4} , ODE tolerance 10^{-6}
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	None quoted. The maximum principle fixes the form of the optimal steering law, but no published numerical value of the maximum range for these data is asserted here
<i>Computed</i>	$J = -1.712314$, that is $x(t_f) = 1.712314$, in 0.06 s of CPU time, with a maximum relative local error of 2.082623×10^{-5}
<i>Source</i>	<code>examples/bryson_max_range/bryson_max_range.cxx</code>

```
NLP solver used:      IPOPT
```

```
Optimal (unscaled) cost function value:  -1.712317e+00
```

```
Phase 1 endpoint cost function value:  -1.712317e+00
```

```
Phase 1 integrated part of the cost:    0.000000e+00
```

```
Phase 1 initial time:      0.000000e+00
```

```
Phase 1 final time:       2.000000e+00
```

```
Phase 1 maximum relative local error:  1.995979e-05
```

```
NLP solver reports:       The problem has been solved!
```

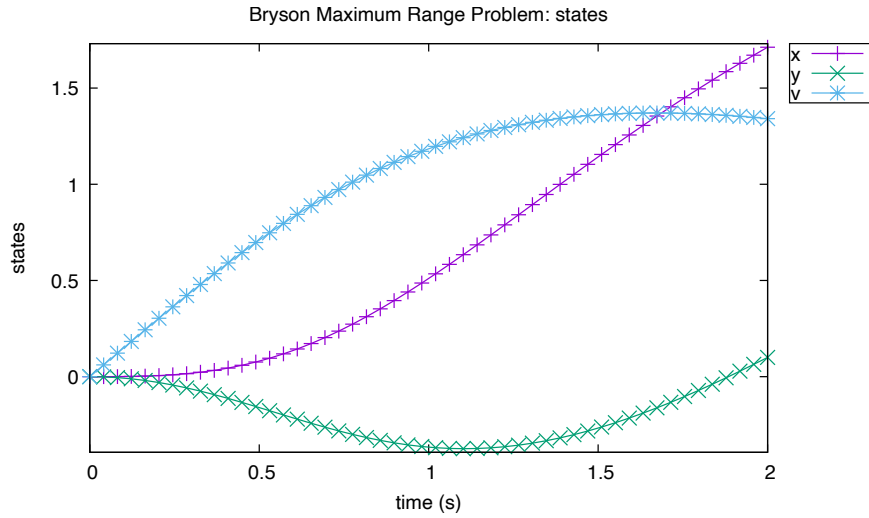


Figure 6: States for Bryson’s maximum range problem

6 Interior point constraint: a condition imposed inside the time horizon

Boundary conditions at the two ends of a trajectory are easy to state: they are event constraints, and every example in this document uses them. A condition imposed at an instant *strictly inside* the horizon, such as a spacecraft that must pass through a given point, a reactor that must reach a given temperature part way through a batch, or a vehicle that must be at a waypoint at a given time, has no such direct expression, because the interface offers events only at t_0 and t_f . The technique shown here is the standard remedy, and it is the reusable idea of this section: cut the horizon at the instant in question, treat the two pieces as separate phases, and impose the interior condition as an event at the end of the first phase. What was interior to the original problem is terminal to the new first phase, and can therefore be written down.

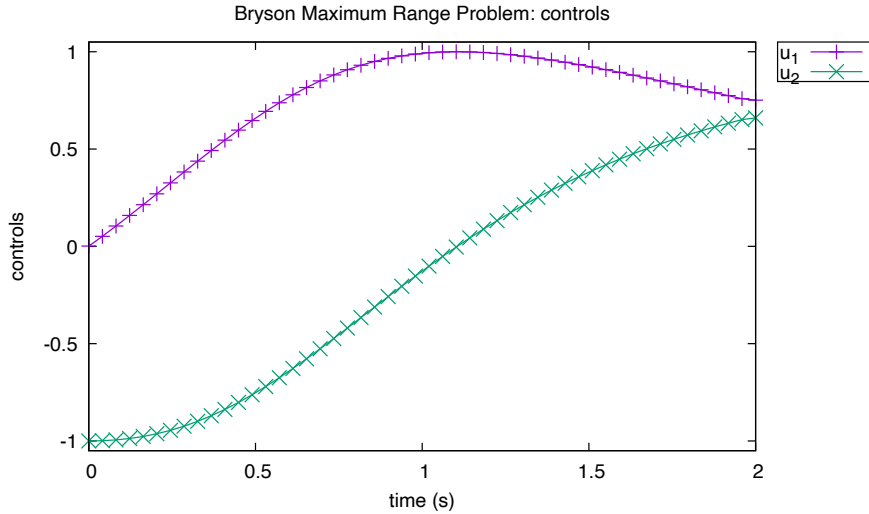


Figure 7: Controls for Bryson’s maximum range problem. The two components trace out a unit vector: the equality path constraint holds along the whole trajectory

The problem itself is deliberately trivial so that the construction is visible. It is taken from the MISER3 manual [7]: minimise $\int_0^1 (x^2 + u^2) dt$ subject to $\dot{x} = u$ with $x(0) = 1$ and $x(1) = 0.75$, and additionally $x(0.75) = 0.9$. The split is at $t = 0.75$; phase 1 runs on $[0, 0.75]$ and carries two events, $x(0) = 1$ and $x(0.75) = 0.9$, and phase 2 runs on $[0.75, 1]$ and carries one, $x(1) = 0.75$. Both phase boundary times are fixed by giving them equal lower and upper bounds. The two phases are joined by `auto_link`, which generates the two linkage constraints that make the state and the time continuous across the junction; the state continuity is what makes the phase-1 event bind on phase 2 as well, so the interior condition need be written once only.

The splitting does more than tidy the notation. The optimal control is genuinely discontinuous at $t = 0.75$: the Euler-Lagrange equation for this problem is $\ddot{x} = x$ on each side of the junction, and the two solutions meet with $u(0.75^-) = 0.2009$ and $u(0.75^+) = -0.7057$. A single-phase discretisation would be trying to represent a jump with a polynomial, which is exactly the situation a phase boundary exists to avoid. The interior condition also binds hard: without it the optimal trajectory would pass through $x(0.75) = 0.7398$, and forcing it up to 0.9 raises the cost by 0.1453, from 0.7752 to 0.9205. Because x must return to 0.9 from below, dipping to a minimum of 0.8773 at $t = 0.523$, the state is not monotone on phase 1, which is visible in Figure 8.

Being linear-quadratic, the problem has a closed-form answer, and the agreement is exact to every digit PSOPT prints: 9.205314×10^{-1} against an analytic 0.9205314414730, with the two phase contributions matching individually as well. Twenty Legendre nodes per phase suffice, the maximum relative local error is below 2×10^{-9} in both phases, no mesh refinement was needed, and the run takes 0.11 s. Neither the state bound $|x| \leq 1$ nor the control bound $|u| \leq 1$ is active anywhere. The same construction with the

junction time left free instead of fixed at 0.75 handles the more common case in which the condition must be met at an instant that is itself unknown: a phase boundary is free exactly when its `StartTime` and `EndTime` bounds are given a range and not a value, as the final phase of the launch problem of Section 16 does.

Table 5: Interior point constraint problem: summary

<i>Demonstrates</i>	Imposing a constraint at an interior instant by splitting the horizon into two phases and writing the constraint as a terminal event of the first phase; automatic phase linkages
<i>Reference</i>	MISER3 manual, Jennings et al. [7]
<i>Dimensions</i>	2 phases; 1 state and 1 control per phase, no static parameters; 2 events in phase 1 and 1 in phase 2; 2 linkage constraints; no path constraints
<i>Objective</i>	$J = \int_0^1 (x^2 + u^2) dt$, accumulated over both phases
<i>Dynamics</i>	$\dot{x} = u$ in both phases
<i>Initial conditions</i>	$x(0) = 1, \quad t_0 = 0$
<i>Terminal conditions</i>	$x(1) = 0.75, \quad t_f = 1$
<i>Interior conditions</i>	$x(0.75) = 0.9$, imposed as the second event of phase 1, whose end time is fixed at 0.75
<i>Linkages</i>	<code>auto_link</code> between phases 1 and 2: continuity of x and of t at $t = 0.75$
<i>Bounds</i>	$-1 \leq x \leq 1, \quad -1 \leq u \leq 1$ in both phases; neither is active at the solution
<i>Discretisation</i>	Legendre pseudospectral, standard differentiation matrix, 20 nodes per phase, automatic differentiation, automatic scaling, limited-memory Hessian, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	The Euler-Lagrange equation is $\ddot{x} = x$, so on a segment of length T joining $x = \alpha$ to $x = \beta$ the optimal cost is $[(\alpha^2 + \beta^2) \cosh T - 2\alpha\beta] / \sinh T$. Summing the segments $(1 \rightarrow 0.9, T = 0.75)$ and $(0.9 \rightarrow 0.75, T = 0.25)$ gives $J^* = 0.6607876554 + 0.2597437861 = 0.9205314415$
<i>Computed</i>	$J = 9.205314 \times 10^{-1}$, with phase contributions 6.607877×10^{-1} and 2.597438×10^{-1} : agreement with J^* to all seven figures printed
<i>Source</i>	<code>examples/interior_point/interior_point.cxx</code>

Results

PSOPT results summary
=====

```

Problem:      Problem with interior point constraint
CPU time (seconds):  4.334800e-02
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  9.205314e-01
Phase 1 endpoint cost function value:  0.000000e+00
Phase 1 integrated part of the cost:  6.607877e-01
Phase 1 initial time:  0.000000e+00
Phase 1 final time:  7.500000e-01
Phase 1 maximum relative local error:  1.045275e-09
Phase 2 endpoint cost function value:  0.000000e+00
Phase 2 integrated part of the cost:  2.597438e-01
Phase 2 initial time:  7.500000e-01
Phase 2 final time:  1.000000e+00
Phase 2 maximum relative local error:  2.514966e-10
NLP solver reports:    The problem has been solved!

```

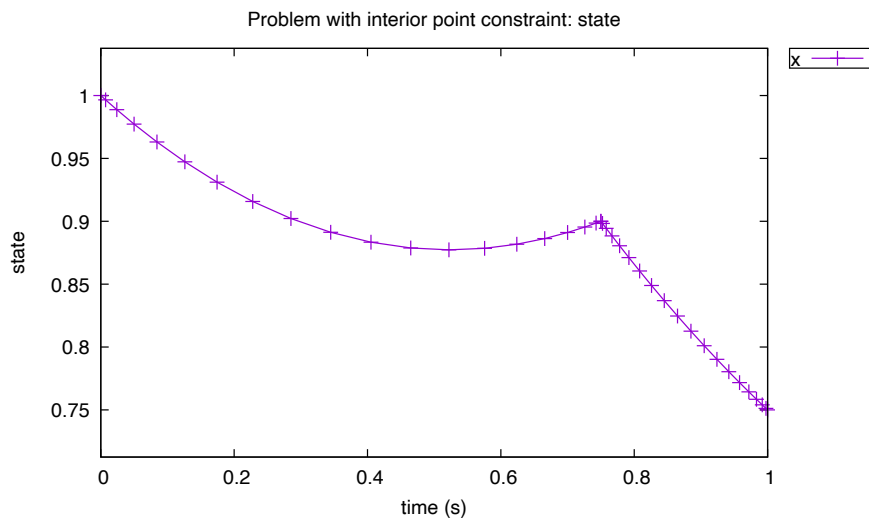


Figure 8: State for the interior point constraint problem. The trajectory is forced through $x = 0.9$ at $t = 0.75$ and dips below that value beforehand

7 Singular control: a problem whose optimal control is undetermined on part of the interval

A control problem is called singular when the Hamiltonian depends linearly on the control over some interval of time, so that the stationarity condition of the minimum principle degenerates: the coefficient multiplying the control, the switching function,

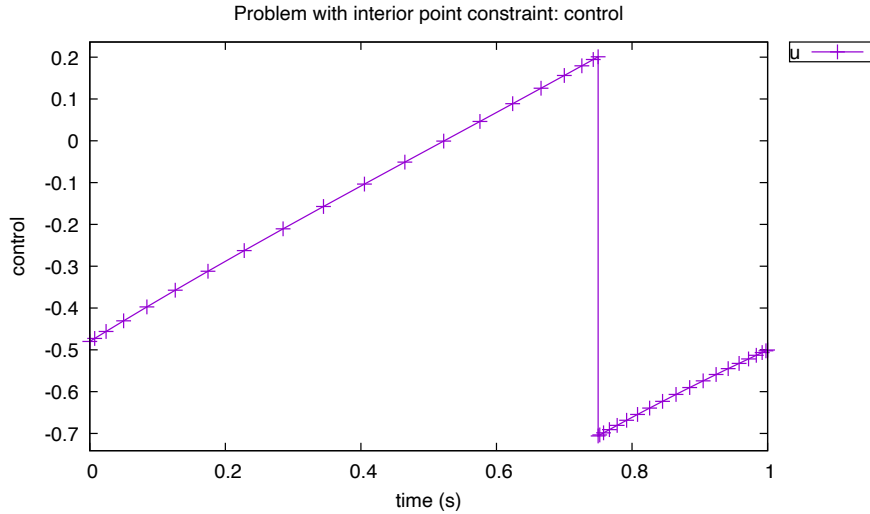


Figure 9: Control for the interior point constraint problem. The jump at $t = 0.75$ is the phase boundary

vanishes identically there and $\partial H/\partial u = 0$ is satisfied for every admissible value of u . The condition therefore says nothing about what the control should be, and the singular control has to be recovered by differentiating the switching function along the trajectory until the control reappears. Away from such an interval the same problem is bang-bang, with the control resting on one of its bounds. Singular arcs are the standard difficult case for any direct method, which is why one example of the kind is included here.

The example is a third-order problem on the fixed interval $t \in [0, 1]$, taken from the singular-control literature [9, 15]. Its Hamiltonian contains the group $(\lambda_3 - \lambda_2 x_3)u$, and the only other dependence on the control comes from the quadratic penalty in the integrand, which contributes a quartic term in u whose coefficient is $5 \times 10^{-6}x_3^2$, small enough that over most of the interval the problem behaves as if it were exactly singular. Note that time enters the cost and the second differential equation explicitly, as the independent variable and not as an extra state: the example carries three states, one control, three events and no path constraints.

Direct collocation responds to this in a characteristic way. The discretised cost is almost insensitive to the control values on a singular portion, so the reduced Hessian of the NLP is nearly singular in those variables and neighbouring nodes can trade control values against one another at almost no penalty. States and objective, which depend on the control through integrals, come out accurately while the control itself is polluted by node-to-node ringing. The computed control here sits on its upper bound $u = 10$ until $t \approx 0.06$, dips to 7.43 near $t = 0.14$ and returns to the bound, falls to the lower bound at $t \approx 0.25$ and stays there until $t \approx 0.72$, and then rises to 10 before coming off the bound onto a smooth interior arc that decreases to about 5.19 at $t \approx 0.94$ and recovers to 6.58 at the final time. The arcs at 10 and at 0 are constrained; the smooth interior

arc at the end is the singular one. Two blemishes are visible in Figure 11: a single node at $t = 0.265$ takes the value 2.28 that neither of its neighbours supports, and along the zero segment the control alternates between about 2×10^{-6} and 2×10^{-5} from one node to the next. Both are artefacts of the flat direction just described, not features of the solution.

The states, by contrast, are clean. The third state is driven through zero at $t \approx 0.28$ and held to within 5×10^{-3} of it until $t \approx 0.72$, which is what makes the coefficient of u in the second differential equation vanish there, and the trajectory ends at $x = (-0.1923, -0.0869, 1.7815)$ with a cost of 1.192620×10^{-1} . The maximum relative local error over the single 100-node Legendre mesh is 7.30×10^{-5} , with a mean of 1.74×10^{-5} and a minimum of 1.23×10^{-7} : four orders of magnitude worse than a smooth problem such as the brachistochrone of Section 2, and the discontinuous control is the reason. Note that the estimate is measuring the residual of the transcription's own state and control polynomials, so the ringing on the singular arc, which is a flat direction of the discrete problem and not an inconsistency with the dynamics, contributes very little to it: the figure is small next to how wrong the control looks. Mesh refinement is switched off in this example, so what is reported is the error of the mesh that was asked for. Refinement would place nodes around the junctions and improve the error there, but it would not by itself remove the ringing on the singular arc, which comes from the structure of the problem, not from the resolution of the grid.

Results

```
PSOPT results summary
=====

Problem:      Singular control 5
CPU time (seconds):  3.317337e+00
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  1.192620e-01
Phase 1 endpoint cost function value:    0.000000e+00
Phase 1 integrated part of the cost:      1.192620e-01
Phase 1 initial time:  0.000000e+00
Phase 1 final time:    1.000000e+00
Phase 1 maximum relative local error:    7.298287e-05
NLP solver reports:    The problem has been solved!
```

The states are shown in Figure 10 and the control in Figure 11. The control figure is the one to look at: the curve joins the computed value at every collocation node, so the node-to-node ringing appears in it directly and does not have to be inferred.

Table 6: Singular control problem: summary

<i>Demonstrates</i>	A problem with a singular arc: how a direct collocation method behaves when the optimal control is not determined pointwise by the first-order conditions
<i>Reference</i>	Luus [9]; also used as a test problem elsewhere [15]
<i>Dimensions</i>	1 phase; 3 states, 1 control, 3 events, no path constraints, no static parameters
<i>Objective</i>	$J = \int_0^1 \left[x_1^2 + x_2^2 + 0.0005 (x_2 + 16t - 8 - 0.1 x_3 u^2)^2 \right] dt$
<i>Dynamics</i>	$\dot{x}_1 = x_2, \quad \dot{x}_2 = -x_3 u + 16t - 8, \quad \dot{x}_3 = u$
<i>Initial conditions</i>	$x_1(0) = 0, \quad x_2(0) = -1, \quad x_3(0) = -\sqrt{5}$
<i>Terminal conditions</i>	None; the final state is free
<i>Bounds</i>	$0 \leq u(t) \leq 10$; the states are unbounded; $t_0 = 0$ and $t_f = 1$ are both fixed
<i>Discretisation</i>	Legendre pseudospectral with the standard differentiation matrix, 100 nodes, automatic differentiation, automatic scaling, Jacobian-based defect scaling, NLP tolerance 10^{-7}
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	No closed-form solution, and no published objective value is quoted here
<i>Computed</i>	$J = 1.192620 \times 10^{-1}$, $x(t_f) = (-0.1923, -0.0869, 1.7815)$, maximum relative local error 7.30×10^{-5}
<i>Source</i>	examples/singular5/singular5.cxx

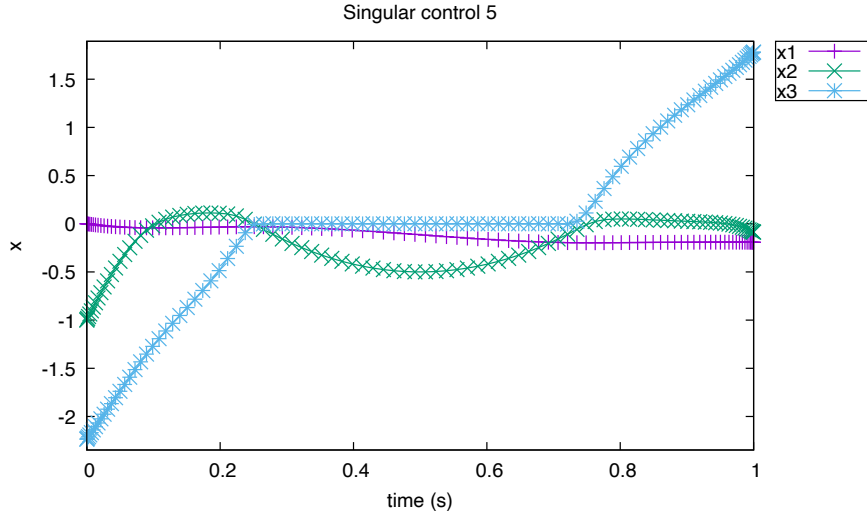


Figure 10: States for the singular control problem

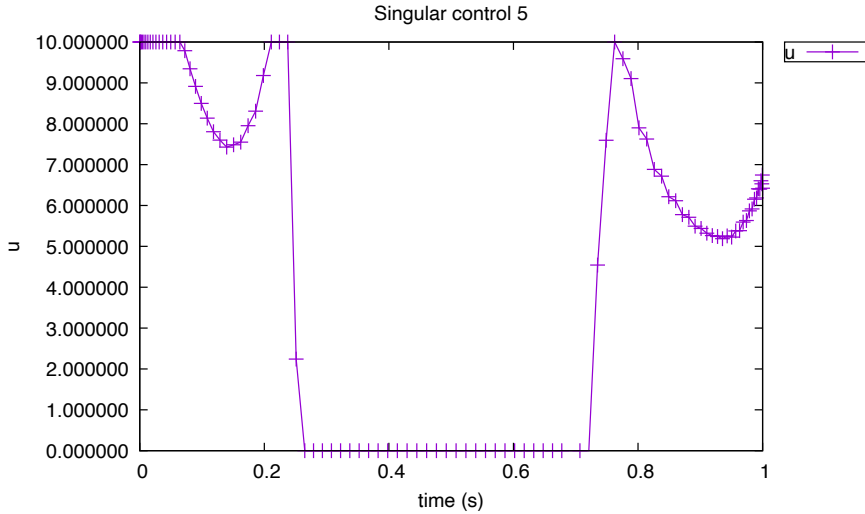


Figure 11: Control for the singular control problem: the arcs at $u = 10$ and $u = 0$ lie on the bounds, and the smooth interior arc after $t \approx 0.78$ is the singular one

8 Time delay problem: states that depend on their own past

A delay differential equation makes the derivative at time t depend on the state at an earlier time $t - \tau$, so the trajectory is no longer determined by its value at a single instant: it needs a history over an interval. Delays of this kind are unavoidable wherever transport, measurement or communication takes time: a reactant travelling down a pipe, a sensor with a filter, a control signal sent over a network. The example here, taken from Luus [9], is a linear second-order system in which both states appear delayed by $\tau = 0.25$, driven over a fixed horizon of five seconds to minimise a quadratic cost. The cost is accumulated in a third state so that the objective is a pure endpoint cost, which also makes the running cost readable off the state trajectory.

PSOPT evaluates delayed quantities by interpolating the discretised trajectory it already has. The user calls `get_delayed_state` or `get_delayed_control` inside the `dae` function, giving a state or control index and a delay, and receives an `adouble` that participates in the derivative tape like any other. The indices count from zero, as they do everywhere else in the interface; an out-of-range index is caught, but an index that is merely wrong reads the wrong variable silently. For delayed times earlier than t_0 the value has to come from somewhere outside the discretisation. A history function can be registered as `problem.initial_history`; this example does not register one, so PSOPT falls back on its default constant history $\phi(t) = x(t_0)$ and $\psi(t) = u(t_0)$. Here that means the delayed terms contribute the fixed amount $-2x_1(0) - x_2(0) = -3$ over the whole of the first quarter second. The initial time is fixed at zero, which matters more than it looks: the comparison that decides whether a delayed time falls before t_0 is recorded on

the tape, so a phase with delays and a free t_0 earns a warning from PSOPT and needs numerical derivatives.

The price of the delay is paid in accuracy and in time. The solution has a kink wherever the history discontinuity has propagated forward, that is at every multiple of τ , and across such a point a Hermite-Simpson interval behaves like a second-order scheme and not a fourth-order one. The example therefore starts from 60 nodes instead of the 30 a smooth problem of this size would want (a note in the source records that 30 nodes leaves a maximum relative local error of 1.1×10^{-3}), and leaves the rest to mesh refinement. Two iterations were needed (Table 8): a trapezoidal pass on the initial 60 nodes reaching 1.37×10^{-3} , just short of the 10^{-3} tolerance, and then Hermite-Simpson on 67 nodes at 3.11×10^{-5} . The run takes 37 seconds, which is a great deal for a three-state linear problem, and the reason is visible in the Jacobian: because every delayed value is an interpolation across the trajectory, each collocation constraint touches nodes far from itself and the sparsity ratio is 0.19 at the first iteration, where a local method without delays would give a narrow band.

The discretised problem is a convex quadratic program, since the dynamics of x_1 and x_2 are linear and the cost is quadratic, so it can be solved to high accuracy by other means. An independent solution carried out for this document, on uniform meshes of 200, 400 and 800 intervals, gives 2.529328, 2.526627 and 2.525950, extrapolating to $J^* = 2.52572$. The computed value 2.525754 differs from that by about fourteen parts in a million.

Results

```
PSOPT results summary
=====

Problem:      Time delay problem 1
CPU time (seconds):  1.579740e+01
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  2.525754e+00
Phase 1 endpoint cost function value:    2.525754e+00
Phase 1 integrated part of the cost:      0.000000e+00
Phase 1 initial time:                    0.000000e+00
Phase 1 final time:                      5.000000e+00
Phase 1 maximum relative local error:    3.113100e-05
NLP solver reports:      The problem has been solved!
```

Table 7: Time delay problem: summary

<i>Demonstrates</i>	Delayed states in the dynamics: <code>get_delayed_state</code> with zero-based indices, the default constant initial history, and the mesh refinement needed when the history discontinuity propagates
<i>Reference</i>	Luus [9]
<i>Dimensions</i>	1 phase; 3 states, 1 control, 3 events, no path constraints, no static parameters
<i>Objective</i>	$J = x_3(t_f)$
<i>Dynamics</i>	$\dot{x}_1 = x_2(t), \quad \dot{x}_2 = -10x_1(t) - 5x_2(t) - 2x_1(t - \tau) - x_2(t - \tau) + u(t),$ $\dot{x}_3 = \frac{1}{2} (10x_1^2(t) + x_2^2(t) + u^2(t))$
<i>Initial conditions</i>	$x_1(0) = 1, \quad x_2(0) = 1, \quad x_3(0) = 0$; history $\phi(t) = x(0)$ for $t < 0$, the PSOPT default
<i>Terminal conditions</i>	None; $t_f = 5$ fixed
<i>Bounds</i>	$-100 \leq x_1, x_2, x_3 \leq 100, \quad -100 \leq u \leq 100, \quad t_0 = 0, t_f = 5$
<i>Constants</i>	$\tau = 0.25$
<i>Discretisation</i>	Hermite-Simpson, 60 nodes initially, automatic differentiation, automatic scaling, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	Automatic, local variant, ODE tolerance 10^{-3} ; 2 iterations, 60 to 67 nodes, first pass trapezoidal (Table 8)
<i>Known solution</i>	No published value is quoted by the source. An independent solution of the same problem for this document, on uniform meshes of 200, 400 and 800 intervals, gives 2.529328, 2.526627 and 2.525950, extrapolating to $J^* = 2.52572$
<i>Computed</i>	$J = 2.525754$, a relative difference of 1.4×10^{-5} against J^* ; maximum relative local error 3.113100×10^{-5}
<i>Source</i>	<code>examples/delay1/delay1.cxx</code>

Table 8: Mesh refinement statistics: Time delay problem 1

Iter	DM	M	NV	NC	OE	CE	JE	HE	RHS	$\epsilon_{\max}$	CPU _a
1	TRP	60	242	184	28	28	26	0	3332	1.370e-03	7.590e+00
2	H-S	67	336	205	11	11	11	0	2926	3.113e-05	7.277e+00
CPU _b	-	-	-	-	-	-	-	-	-	-	9.300e-01
-	-	-	-	-	39	39	37	0	6258	-	1.580e+01

Key: Iter=iteration number, DM= discretization method, M=number of nodes, NV=number of variables, NC=number of constraints, OE=objective evaluations, CE = constraint evaluations, JE = Jacobian evaluations, HE = Hessian evaluations, RHS = ODE right hand side evaluations, $\epsilon_{\max}$ = maximum relative ODE error, CPU_a = CPU time in seconds spent by NLP algorithm, CPU_b = additional CPU time in seconds spent by PSOPT

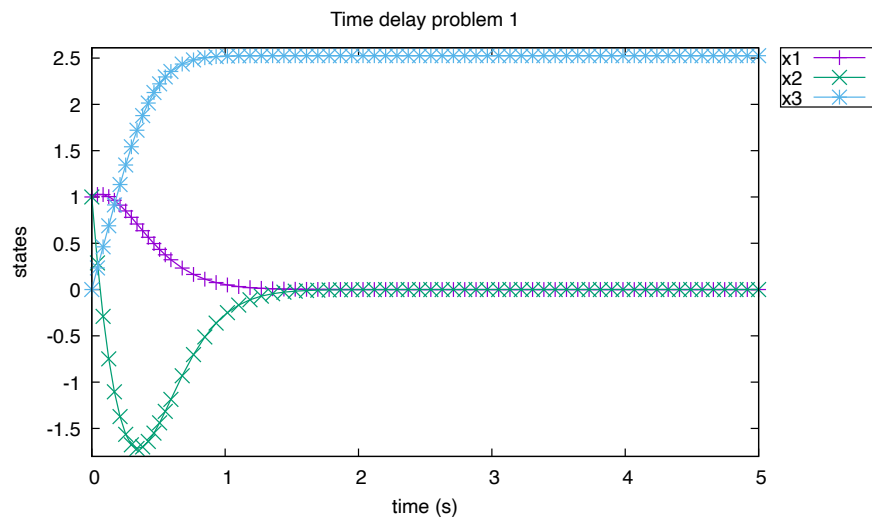


Figure 12: States for the time delay problem

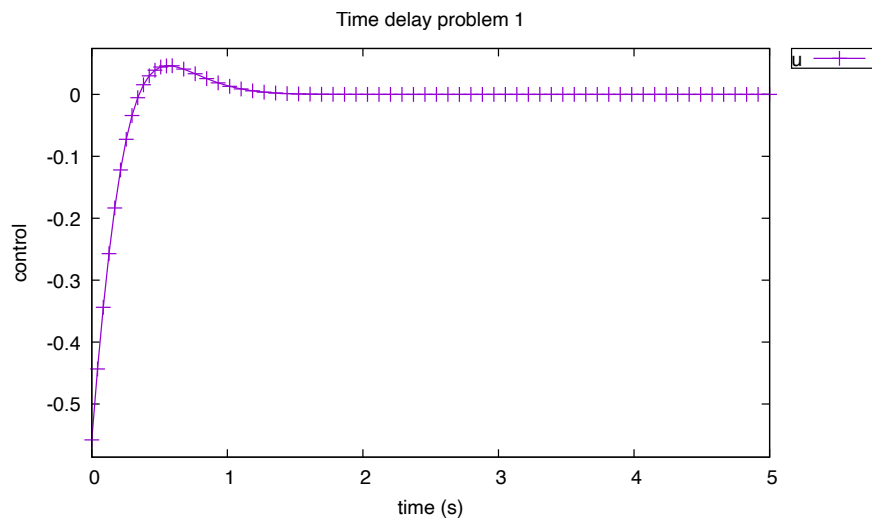


Figure 13: Control for the time delay problem

9 Constrained pendulum: a high-index DAE, and a second parameter estimation

Write the equations of a point pendulum in Cartesian coordinates and the result is a differential-algebraic equation of index three. The bob is at (x_1, x_2) with velocity (x_3, x_4) ; the rod pulls along its own direction with a magnitude not known in advance, which enters the equations of motion as a multiplier $\lambda(t)$. The differential equations therefore carry an unknown algebraic variable, and the equation that ought to determine it, $L^2 - x_1^2 - x_2^2 = 0$, does not contain it. This example takes the version posed by Schittkowski [19], in which L is not known either and has to be estimated from twenty samples of the two position coordinates, so the section stands both for high-index DAEs and for a second instance of the estimation machinery introduced in Section 4.

The index is the number of times the algebraic equation has to be differentiated before the multiplier appears. Once gives the velocity-level condition $x_1x_3 + x_2x_4 = 0$; twice gives the acceleration level, $x_3^2 + x_4^2 + \lambda(x_1^2 + x_2^2) = 0$; only a third differentiation produces a differential equation for λ itself. That is what makes the system awkward for a great many solvers. Integration-based methods (BDF codes, implicit Runge-Kutta schemes, and any shooting transcription built on them) degrade as the step size falls, because the multiplier is recovered through a divided difference taken three times and its error grows like an inverse power of the step. They also need initial values consistent with the two hidden constraints as well as with the position constraint, an unpleasant demand here, since the initial state is exactly what is not known. The usual remedy, index reduction by substituting a differentiated constraint, buys stability at the cost of drift off the circle $x_1^2 + x_2^2 = L^2$, which then has to be stabilised.

PSOPT sidesteps all of this in a way worth taking away. The algebraic variable is declared as a control, and the algebraic equation is declared as a path constraint with equal lower and upper bounds. Nothing is integrated: the state polynomial and the algebraic variable are unknowns of the same nonlinear program, and the constraint is imposed as an equality at every collocation node, so it cannot drift. The multiplier is fixed by the algebra of the constraint and not by differentiating it, and no consistent initialisation is needed: this example declares no event constraints at all, and the initial state is decided by the data along with everything else. Note also that the two hidden constraints are never written down. Enforcing the position constraint at all thirty nodes of a single global polynomial is enough to imply them to the accuracy of the discretisation.

The fit is essentially exact. On one fixed Legendre grid of 30 nodes the sum of squared residuals is 2.42×10^{-18} , the maximum relative local error is 9.7×10^{-9} , the estimated length is $\hat{L} = 1.000000$ against the exact value $L = 1$, and the computed $\lambda(t)$ is flat at -1 to the six figures written to the solution file. Mesh refinement was not needed and did not iterate. The parameter statistics should be read with that in mind: the estimated residual standard deviation is 2.6×10^{-10} and the 95 per cent confidence interval on $\hat{L}$ collapses to the estimate itself when printed to seven figures. That is not evidence that twenty samples fix a pendulum length to ten decimal places; the supplied data are noise-free samples of the analytic solution $x_1 = \sin(t + \frac{1}{2})$, $x_2 = \cos(t + \frac{1}{2})$,

reproduced to the eight figures held in the data file. At this residual the limits measure the arithmetic rather than the experiment. That is the opposite of the wide intervals in Section 4, and a reminder that an interval is only as informative as the noise behind it.

Table 9: Constrained pendulum (DAE index 3): summary

<i>Demonstrates</i>	An index-3 differential-algebraic system solved without index reduction: the algebraic state declared as a control and the algebraic equation as an equality path constraint; parameter estimation with a fitted initial state
<i>Reference</i>	Schittkowski [19]; statistics after Kostina [8]
<i>Dimensions</i>	1 phase; 4 differential states, 1 control (the algebraic state λ), 1 static parameter, no events, 1 path constraint; 2 observed variables at 20 sampling instants
<i>Objective</i>	$J = \sum_{k=1}^{20} [(y_1(t_k) - \hat{y}_1(t_k))^2 + (y_2(t_k) - \hat{y}_2(t_k))^2]$, formed internally from the observations
<i>Dynamics</i>	$\dot{x}_1 = x_3, \quad \dot{x}_2 = x_4, \quad \dot{x}_3 = \lambda x_1, \quad \dot{x}_4 = \lambda x_2$
<i>Observation function</i>	$g(x, u, p, t) = [x_1 \ x_2]^T$, unit residual weights
<i>Path constraints</i>	$0 = L^2 - x_1^2 - x_2^2$, imposed at every node with equal lower and upper bounds
<i>Bounds</i>	$-2 \leq x_j \leq 2, \quad -10 \leq \lambda \leq 10, \quad 0 \leq L \leq 5, \quad t \in [0.5, 10]$ fixed
<i>Data</i>	20 sampling instants at $t = 0.5, 1.0, \dots, 10.0$ and 40 measurements, read from <code>examples/dae_i3/dae_i3.dat</code>
<i>Discretisation</i>	Legendre pseudospectral, 30 nodes, standard differentiation matrix, automatic differentiation, automatic scaling, limited-memory Hessian, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	$L = 1$ and $\lambda(t) = -1$ exactly [19]
<i>Computed</i>	$\hat{L} = 1.000000, \quad \lambda(t) = -1$ throughout, $J = 2.421548 \times 10^{-18}$, $\hat{\sigma} = 2.558266 \times 10^{-10}$, maximum relative local error 9.671447×10^{-9}
<i>Source</i>	<code>examples/dae_i3/dae_i3.cxx</code>

Results

```
PSOPT results summary
=====
```

```
Problem:      DAE Index 3
CPU time (seconds):  5.865070e-01
NLP solver used:    IPOPT
```

```
Optimal (unscaled) cost function value:  2.627410e-18
```

```

Phase 1 endpoint cost function value: 2.627410e-18
Phase 1 integrated part of the cost: 0.000000e+00
Phase 1 initial time: 5.000000e-01
Phase 1 final time: 1.000000e+01
Phase 1 maximum relative local error: 8.458726e-09
NLP solver reports: The problem has been solved!

```

The parameter statistics are reproduced below. The covariance is 3.27×10^{-21} and the reported degrees of freedom are 40 scalar observations against 3 free directions in the nonlinear program.

```

***** PHASE 1 PARAMETERS *****
1.000000e+00
***** PARAMETER STATISTICS *****
>>>> Parameter covariance matrix (all phases)
3.550554e-21
>>>> Rank of parameter covariance matrix: 1
>>>> Parameter correlation matrix (all phases)
1.000000e+00
>>>> Estimated residual standard deviation (sigma_hat): 2.664791e-10
>>>> Scalar observations (N_s): 40, degrees of freedom of the NLP (n_f): 3
>>>> Sum of squared weighted residuals at the solution: 2.627410e-18
>>> 95 percent statistical confidence limits on estimated parameters
Phase Parameter (Low Confidence Limit) (Value) (High Confidence Limit)
1 1 1.000000e+00 1.000000e+00 1.000000e+00

```

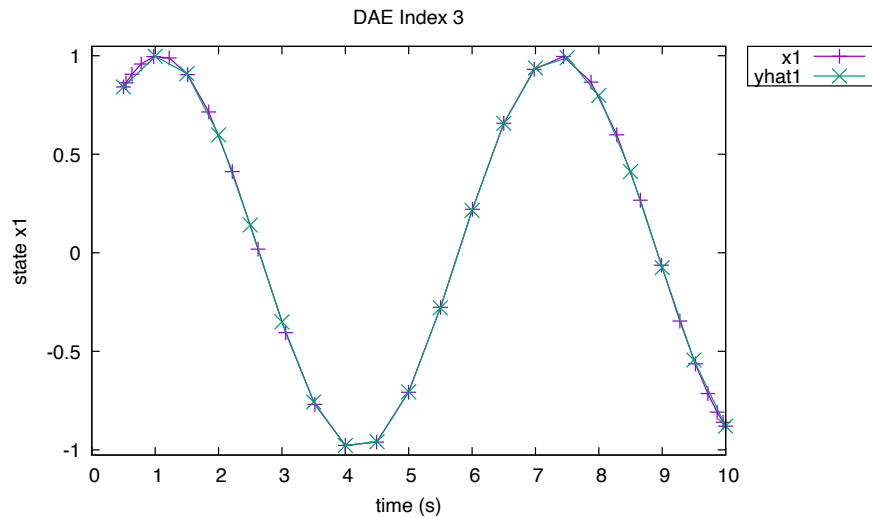


Figure 14: Fitted state x_1 and the measurements for the constrained pendulum

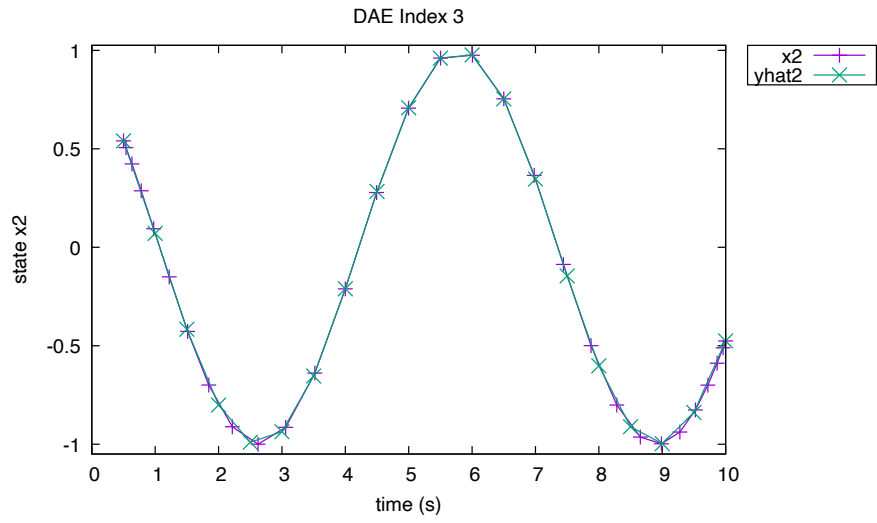


Figure 15: Fitted state x_2 and the measurements for the constrained pendulum

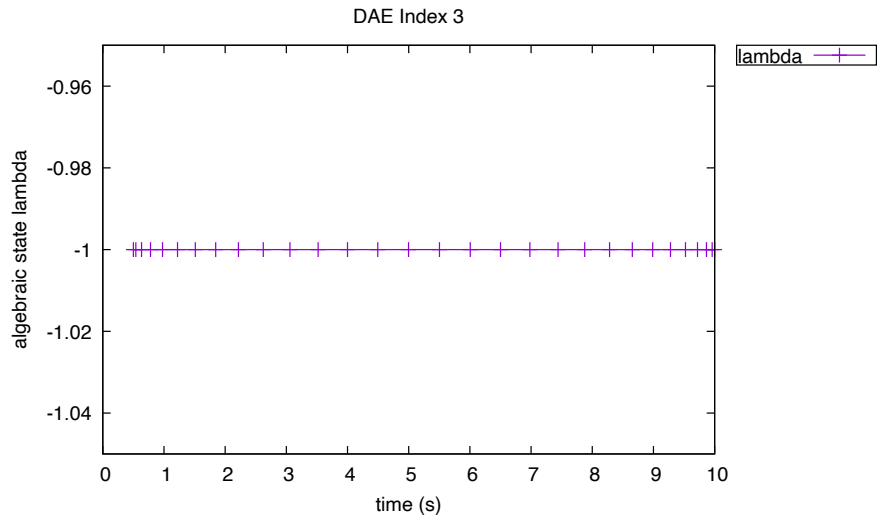


Figure 16: The algebraic state $\lambda(t)$, computed as a control

10 Integer static parameter: a discrete-valued decision variable

Static parameters that are continuous decision variables appear in several of the problems in this document; the catalytic cracking example (4) is one, where they are the quantities being estimated. Some problems instead have a static parameter that may take only a finite set of values, such as a gear ratio, the number of units in a plant or a discrete valve setting, and rounding whatever the continuous solver returns is not in general the same thing as optimising over the admissible set. PSOPT handles the discrete case with a dedicated driver, and the example here is the smallest problem that exercises it, deliberately constructed so that the right answer can be written down by hand.

The system is a double integrator on $t \in [0, 1]$ whose acceleration is not a control but the static parameter p : $\dot{s} = v$, $\dot{v} = p$, started from rest at the origin, so that $v(1) = p$ exactly. There is no control variable at all and no integrand cost. The whole problem is the Mayer term $J = (v(1) - c)^2 = (p - c)^2$ with $c = 2.3$, and the only decision left is which member of the admissible set $\{0, 1, 2, 3\}$ the parameter should be. The nearest admissible value to c is $p = 2$, giving $J = 0.09$, and the example checks the computed answer against that closed form and prints **PASS** or **FAIL**.

The parameter is made discrete by a single call to `declare_integer_parameter`, which takes the problem, the phase, the index of the parameter and the row vector of admissible values; the problem is then handed to `psopt_solve_integer` instead of to `psopt`. That driver enumerates the Cartesian product of every declared admissible set, and for each combination pins the parameter by setting its lower and upper bound to the same value and calling the ordinary `psopt` solver. The best cleanly converged trial is remembered, re-solved once into the caller's `solution`, and the user's original parameter bounds are put back. Five solves therefore take place here: four enumerated trials and the final re-solve. The core solver is untouched by any of this: it sees only a parameter whose bounds happen to coincide. A guard, `algorithm.max_integer_combinations` (default 4096), stops the enumeration before it is attempted if the product of the set sizes is too large, which states plainly where this method stops being usable: it is exact in the small-discrete-space regime and nowhere else.

Where relaxation and rounding enter deserves a precise statement, because they do enter, though not as the mechanism that produces the answer. The bounds the user declares, $0 \leq p \leq 3$, are the convex hull of the admissible set; they must contain it, since each enumerated value is imposed inside that box, and they are what the parameter reverts to after the solve. Rounding appears in `reconstruct_integer_parameter`, which reads the parameter back out of the solved phase and snaps it to the nearest admissible value, a safety net against arithmetic noise and not a decision, since the enumeration has already pinned the value exactly. The naive alternative, solving once over the relaxed box and rounding, would happen to succeed on this problem: the cost is convex in p , the relaxed minimiser is $p = 2.3$, and the nearest admissible value is 2. That coincidence is a property of a scalar convex cost, not a guarantee; enumeration gives the same answer without needing it.

Table 10: Integer static parameter: summary

<i>Demonstrates</i>	A discrete-valued static parameter: <code>declare_integer_parameter</code> , solution by enumeration with <code>psopt_solve_integer</code> , and read-back with <code>reconstruct_integer_parameter</code>
<i>Reference</i>	None. The problem is a constructed analytical test, written for this purpose and not taken from the literature
<i>Dimensions</i>	1 phase; 2 states, no controls, 1 static parameter (integer, 4 admissible values), 2 events, no path constraints
<i>Objective</i>	$J = (v(t_f) - c)^2 = (p - c)^2$, $c = 2.3$ (Mayer term only)
<i>Dynamics</i>	$\dot{s} = v$, $\dot{v} = p$
<i>Initial conditions</i>	$s(0) = 0$, $v(0) = 0$
<i>Path constraints</i>	None
<i>Bounds</i>	$p \in \{0, 1, 2, 3\}$, relaxed box $0 \leq p \leq 3$; $-20 \leq s, v \leq 20$; $t \in [0, 1]$ fixed
<i>Constants</i>	$c = 2.3$
<i>Discretisation</i>	Legendre pseudospectral (the default), 20 nodes, automatic differentiation, automatic scaling, NLP tolerance 10^{-8} , <code>print_level = 0</code> so that the enumerated solves are silent
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	Closed form: the admissible value nearest c , that is $p^* = 2$ and $J^* = (2 - 2.3)^2 = 0.09$
<i>Computed</i>	$p = 2.000000$ (admissible index 2), $J = 0.090000$, solver return code 0; the example's own check passes
<i>Source</i>	<code>examples/int_static_linear/int_static_linear.cxx</code>

Results

The example runs with `print_level` set to zero, so the four enumerated solves produce no output of their own and there are no trajectory plots to show: a two-state trajectory determined entirely by a scalar would say nothing that the selected value does not. The report below is the one the example prints itself, and it is the whole of the result.

```
=====
Integer static parameter (target c = 2.300000)
=====

Admissible values : {0, 1, 2, 3}
Solver return code: 0

Selected p      = 2.000000 (admissible index 2)
Optimal cost J  = 0.090000

Closed form     -> p = 2.000000, J = 0.090000
=====
Result: PASS
=====
```

11 Lotka-Volterra fishing: an integer-valued control

A fishing fleet operates on a prey-predator system and may only be sent out or kept in port: the control is binary. From an initial population pair $(0.5, 0.7)$ the fleet must be scheduled over a fixed horizon of twelve time units to steer the system to the coexistence steady state $(1, 1)$, at least cost in integrated squared deviation from it. This is the Lotka-Volterra fishing problem, the standard small benchmark of mixed-integer optimal control, due to Sager [16] and distributed through mintOC [17]. It stands here for problems in which a decision variable is a switch rather than a dial. Its static counterpart, a parameter drawn from a finite set, is the subject of section 10.

The dynamics are written once, with the fishing effort w appearing as an ordinary control, and the single call `declare_integer_control` then names that control and its admissible set $\{0, 1\}$. Everything else happens inside PSOPT. The named control is removed from the control vector and replaced by two weight variables $\omega_0, \omega_1 \in [0, 1]$; the user's right-hand side is evaluated once at $w = 0$ and once at $w = 1$ and the two results are combined as $\omega_0 f(x, 0) + \omega_1 f(x, 1)$; and one equality path constraint, $\omega_0 + \omega_1 = 1$, is appended. This is outer convexification, and its virtue is that the relaxation it produces is tight in the sense that matters: its optimum is the infimum of the original binary problem. What comes back from the NLP is a fractional $\omega(t)$, and `reconstruct_integer_control` turns it into an admissible schedule by sum-up rounding, the accumulator sweep of Sager, Bock and Diehl [18], which activates on each interval whichever mode is most owed relative to its relaxed integral. Their theorem bounds the resulting state deviation by a constant times the mesh spacing h , so the rounded trajectory, and with it the objective, approaches the relaxed one as $h \rightarrow 0$.

The decisive point about this problem is that no optimal integer control exists. The relaxed optimum $\Phi^* = 1.34408$ reported by mintOC is an infimum that is approached

but never attained. The reason is that the relaxed solution is singular over part of the horizon: there $\omega(t)$ is strictly between zero and one, meaning the fleet is best operated at a fractional rate, and no $\{0,1\}$ -valued function equals a fractional one on a set of positive measure. By the approximation theorem the value can be got arbitrarily close by switching fast enough, which is why the switch count in Table 12 climbs with the mesh: 4 switches at 40 nodes, 78 at 640. Each of those schedules is feasible, each better than the last, but the sequence has no best member. This is a property of the problem and not a defect of the method, and what is reported here is stated accordingly: a lower bound from the relaxation, and a sequence of implementable controls descending towards it.

The relaxed objective falls from 1.348102 at 40 nodes to 1.344118 at 640, converging to Φ^* from above as the trapezoidal discretisation error shrinks; at the finest mesh the maximum relative local error is 2.4×10^{-4} . The rounded objective behaves less tidily. It is 1.351850 at 40 nodes but 1.431325 at 80, a gap that is ten times worse on the finer mesh. Nothing has gone wrong: near a singular arc the accumulator can be tipped either way by a small change in the weights, a switch lands one whole interval away from where it should, and the theory offers only a one-sided $O(h)$ bound, not monotonicity. The fine end is clean, the gap falling from 6.0×10^{-4} at 320 nodes to 2.0×10^{-4} at 640, a factor of 2.98 for a halving of h , which is faster than linearly and which the bound permits.

Results

The summary below is that of the last and finest solve of the sweep, the relaxed problem on 640 nodes.

```
PSOPT results summary
=====

Problem:      Lotka-Volterra fishing (integer-control declaration API)
CPU time (seconds):  1.765824e+00
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  1.344118e+00
Phase 1 endpoint cost function value:  1.344118e+00
Phase 1 integrated part of the cost:    0.000000e+00
Phase 1 initial time:    0.000000e+00
Phase 1 final time:      1.200000e+01
Phase 1 maximum relative local error:  2.442577e-04
NLP solver reports:      The problem has been solved!
```

Table 12 collects the sweep. The relaxed objective in the second column is the optimum of the relaxation, a lower bound up to discretisation error on what any binary schedule can achieve; the third column is the objective of the rounded schedule, obtained

Table 11: Lotka-Volterra fishing problem: summary

<i>Demonstrates</i>	An integer-valued control: <code>declare_integer_control</code> , outer convexification performed internally, and recovery of a binary schedule by sum-up rounding
<i>Reference</i>	Sager [16]; mintOC benchmark library [17]; rounding after Sager, Bock and Diehl [18]
<i>Dimensions</i>	1 phase; 3 states, 1 control, 3 events, no user path constraints, no static parameters. The declaration expands the phase internally to 2 weight controls and 1 SOS1 equality path constraint
<i>Objective</i>	$J = x_2(t_f) = \int_0^{t_f} [(x_0 - 1)^2 + (x_1 - 1)^2] dt$
<i>Dynamics</i>	$\dot{x}_0 = x_0 - x_0x_1 - c_0x_0w, \quad \dot{x}_1 = -x_1 + x_0x_1 - c_1x_1w, \quad \dot{x}_2 = (x_0 - 1)^2 + (x_1 - 1)^2, \quad \text{with } w(t) \in \{0, 1\}$
<i>Initial conditions</i>	$x_0(0) = 0.5, \quad x_1(0) = 0.7, \quad x_2(0) = 0$
<i>Path constraints</i>	None written by the user; the convexification adds $\omega_0 + \omega_1 = 1$
<i>Bounds</i>	$0 \leq x_0, x_1 \leq 2, \quad 0 \leq x_2 \leq 100, \quad t_0 = 0, \quad t_f = 12$ both fixed. Bounds on a declared integer control are ignored: its admissible set governs
<i>Constants</i>	$c_0 = 0.4, \quad c_1 = 0.2$
<i>Discretisation</i>	Trapezoidal, meshes of 40, 80, 160, 320 and 640 nodes solved in turn, automatic differentiation, automatic scaling, NLP tolerance 10^{-7}
<i>Mesh refinement</i>	None (fixed mesh); the sweep over meshes is driven by the example itself
<i>Known solution</i>	No optimal integer control exists. The relaxed optimum $\Phi^* = 1.34408$ quoted by mintOC [17] is the infimum of the binary problem, approached but not attained
<i>Computed</i>	At 640 nodes the relaxed optimum is 1.344118; the rounded binary schedule gives 1.344282, that is 2.024×10^{-4} above Φ^* , with 78 switches
<i>Source</i>	<code>examples/lotka/lotka.cxx</code>

by simulating it with a fixed-step Runge-Kutta integrator from the given initial state, and is therefore an achievable upper bound. The fourth column is the third minus Φ^* .

Table 12: Lotka-Volterra fishing: the mesh sweep

nodes	relaxed Φ	integer Φ	gap to Φ^*	switches
40	1.348102	1.351850	7.770e-03	4
80	1.344991	1.431325	8.724e-02	10
160	1.344232	1.354100	1.002e-02	20
320	1.344125	1.344684	6.035e-04	40
640	1.344118	1.344282	2.024e-04	78

12 An analytic test problem: integrated-residual transcription against collocation

This is a small scalar problem with a closed-form solution, introduced by Neuenhofen and Kerrigan [10] to compare direct transcription methods. One state, one control, a fixed horizon $[0, \pi/2]$, and an integrand $y^2 + u \cos t$ that is quadratic in the state and linear in the control. The control bound $-1.5 \leq u \leq 1$ never becomes active: the optimal control runs from $u^*(0) = -1$ to $u^*(\pi/2) = 0.125$. The unconstrained minimum principle therefore holds throughout and the solution can be written down: $\lambda^* = -\cos t$, $y^*(t) = -\sin t/(2 - \cos t)$, $u^*(t) = 1/2 - 1.5/(\cos t - 2)^2$, with $J^* = -0.256996962561$. Every number a solver produces on this problem can therefore be compared against the truth and not against another solver, which is why it is used here to exercise PSOPT's integrated-residual transcription.

Collocation forces the dynamic residual to zero at a finite set of points and says nothing whatever about the intervals between them. The discretised problem is consequently a *relaxation* of the continuous one, not a restriction of it, and its optimal value may lie below the true optimum. That is exactly what happens here: on 41 nodes, Hermite-Simpson returns $J = -0.257079348996$, which is 8.24×10^{-5} below J^* , a value no admissible trajectory can attain. The control that produces it is wrong by 1.68 in the maximum norm over the nodes, which is more than the entire range of u^* ; this is the familiar node-to-node ringing.

Neither of the two quantities one would naturally consult gives any hint of it. The objective is below the optimum, which looks like success. And PSOPT's relative local error estimate reads 3.35×10^{-5} , which looks like an accurate solution. The estimate is not malfunctioning: it measures how nearly the transcription's own state and control polynomials satisfy the differential equation, and that pair, ringing control and all, satisfies it perfectly well. What has gone wrong is not feasibility. It is that Simpson's rule cannot integrate $u(t) \cos t$ when u oscillates from one node to the next, so the cost the nonlinear program minimises is not quite the cost the user wrote; the same trajectory's polynomials, integrated accurately, give a cost 1.9×10^{-5} above J^* , on the admissible

side. A residual that is small and a quadrature that is wrong look identical from the outside, which is the reason this example is here. The solve is also badly conditioned near its ringing solution and only certifies optimality at an NLP tolerance of 10^{-5} .

Setting `transcription_method` to `"integrated-residual"` changes what is being constrained. PSOPT now evaluates the residual by Gauss-Legendre quadrature at `ir_residual_nodes` points in each interval, placed deliberately away from the mesh nodes, so that inter-node error is visible to the nonlinear program. The runs here use the well-conditioned residual-box form, `ir_objective` set to `"cost"` with a bound `ir_residual_bound` $= \delta$, which minimises the cost subject to $|r| \leq \delta$ at every quadrature point, with $\delta = h^p$ (h the node spacing, p the order of the local representation). This is still a relaxation, but a declared one, and the accounting is remarkably clean: over the flexible-order sequence the shortfall below J^* is very nearly proportional to δ , the ratio $|J - J^*|/\delta$ taking the values 40.6, 40.7, 40.6 and 44.9 as δ falls through four decades. The declared bound thus buys an estimate of how far below the truth the reported cost can be, which collocation does not provide. With the Nie-Kerrigan representation [12, 11] at `ir_local_order` = 5, on the same 41 nodes as the collocation run, $J = -0.257001152538$: two orders of magnitude closer to J^* , a control error of 1.54×10^{-3} , and a relative local error of 1.17×10^{-7} .

Two things should be said against this. First, the default cubic-Hermite residual box on 41 nodes returns $J = -0.304114780310$, further from J^* than the collocation number is. That is the relaxation behaving as advertised and not failing: $\delta = h^2 = 1.5 \times 10^{-3}$ is a loose allowance, and since the cost depends on the control only through $u \cos t$, the optimiser spends the allowance where the weight $\cos t$ is smallest, near $t = \pi/2$, where a large control excursion is nearly free. Second, δ cannot be driven below what the NLP can resolve. The p -sequence stops at order 5 because the next step would ask for $\delta = h^6 \approx 10^{-9}$ against an NLP tolerance of 10^{-7} .

Results

The example performs six solves of the same problem and prints its own comparison against the analytic solution, reproduced below; it runs with `print_level` set to zero and writes no options or summary file of its own, so the table is the whole of the result. The last two columns are the ones to compare: $\max |u - u^*|$ is the largest control error at a node, and `relLocErr` is PSOPT's estimate of the relative local error.

X1 -- Neuenhofen-Kerrigan analytic example (J* = -0.256996962561)					
method	nodes	J	J-J*	max u-u*	relLocErr
collocation (Hermite-Simpson)	41	-0.257075361040	7.84e-05	1.58e+00	3.34e-05
integrated-residual (cubic-H)	41	-0.304114775838	4.71e-02	1.54e+00	8.69e-04
Nie-Kerrigan order 5	41	-0.257001152564	4.19e-06	7.68e-04	1.17e-07
Nie-Kerrigan flexible-order p-convergence (8 elements, order-aware box):					
order 2	17	-0.647620085342	3.91e-01	5.19e-01	1.61e-02
order 3	25	-0.268360104258	1.14e-02	2.64e-02	4.93e-04
order 4	33	-0.257232675528	2.36e-04	4.75e-03	7.56e-06
order 5	41	-0.257001152564	4.19e-06	7.68e-04	1.17e-07

Table 13: Neuenhofen-Kerrigan analytic problem: summary

<i>Demonstrates</i>	Integrated-residual transcription: the residual box (<code>ir_objective = "cost"</code> with <code>ir_residual_bound</code>), the Nie-Kerrigan flexible-order representation, and the fact that collocation can report a cost below the true optimum
<i>Reference</i>	Neuenhofen and Kerrigan [10]; the integrated-residual methods of Nie and Kerrigan [12, 11]
<i>Dimensions</i>	1 phase; 1 state, 1 control, 1 event, no path constraints, no static parameters
<i>Objective</i>	$J = \int_0^{\pi/2} [y(t)^2 + u(t) \cos t] dt$
<i>Dynamics</i>	$\dot{y} = \frac{1}{2}y^2 + u$
<i>Initial conditions</i>	$y(0) = 0$
<i>Terminal conditions</i>	None; $y(t_f)$ is free
<i>Bounds</i>	$-10 \leq y \leq 10$, $-1.5 \leq u \leq 1$ (inactive at the solution), $t_0 = 0$ and $t_f = \pi/2$ both fixed
<i>Discretisation</i>	Hermite-Simpson, automatic differentiation, automatic scaling, IPOPT; 41 nodes for the three-way comparison and $8p + 1$ nodes for the p -sequence; <code>ir_residual_scaling = "state"</code> , <code>ir_residual_bound = h^p</code> , <code>ir_residual_nodes = 4</code> for the cubic-Hermite runs and $2p$ for the flexible-order runs; NLP tolerance 10^{-5} for the two 41-node comparison runs and 10^{-7} for the Nie-Kerrigan runs
<i>Mesh refinement</i>	None (fixed mesh); the mesh and the local order are varied by hand between solves
<i>Known solution</i>	$y^*(t) = -\sin t/(2 - \cos t)$, $u^*(t) = 1/2 - 1.5/(\cos t - 2)^2$, $\lambda^*(t) = -\cos t$, $J^* = -0.256996962561$, stated by the source and verified there by quadrature to about 10^{-14}
<i>Computed</i>	On 41 nodes: collocation $J = -0.257079348996$ (8.24×10^{-5} below J^* , control error 1.68); cubic-Hermite residual box $J = -0.304114780310$; Nie-Kerrigan order 5 $J = -0.257001152538$ (4.19×10^{-6} from J^* , control error 1.54×10^{-3} , relative local error 1.17×10^{-7})
<i>Source</i>	<code>examples/x1/x1_example.cxx</code>

```

Collocation reports J below J* -- not because the trajectory is infeasible
between the nodes (its relative local error is ~3e-5) but because Simpson's
rule cannot integrate u(t)cos(t) for a control that rings at the mesh scale --
and the control itself is wrong by more than the range of u*;
integrated-residual is feasibility-honest; flexible-order p-refinement converges
to the analytic optimum.

```

13 Chance-constrained double integrator: a probabilistic path constraint by constraint tightening

A chance constraint asks that a path constraint hold with at least a stated probability, $P\{v(t) \leq V_{\max}\} \geq 1 - \epsilon$, instead of with certainty. The oldest and cheapest way to impose one is to convert it into a deterministic constraint on the nominal trajectory: if the estimate of $v(t)$ is Gaussian with standard deviation σ , the requirement is met exactly when the nominal value satisfies $v(t) + k\sigma \leq V_{\max}$ with $k = \Phi^{-1}(1 - \epsilon)$, so the original limit is tightened by the back-off $k\sigma$. That is all this example does, and what it does not do needs saying plainly. It does not propagate uncertainty: σ is a constant put in by hand purely to illustrate the mechanism, the Gaussian assumption is asserted and not derived, and the $1 - \epsilon$ guarantee obtained is pointwise in time, not over the whole trajectory. The comments in the source say what a full treatment would require: carry $\sigma(t)$ as extra states governed by a Lyapunov differential equation driven by the linearised dynamics and the process noise, and evaluate the back-off from those states. PSOPT is perfectly able to solve that larger problem, since the extra covariance states are ordinary states and the tightened constraint is an ordinary path constraint, but it is not the problem solved here. The problem is a constructed illustration and not taken from the literature.

The nominal problem is the textbook minimum-energy move of a double integrator: $\dot{x} = v$, $\dot{v} = u$, from $(x, v) = (0, 0)$ to $(1, 0)$ in one unit of time, minimising $\int_0^1 u^2 dt$. Its unconstrained solution is $u = 6 - 12t$, $v = 6t(1 - t)$, $x = 3t^2 - 2t^3$, with cost 12 and a velocity that peaks at 1.5 half way through. The numbers in the example are chosen so that this peak is inadmissible: with $V_{\max} = 1.30$, $\sigma = 0.03$ and $k = 1.6449$ (that is, $\epsilon = 5$ per cent) the tightened limit is $1.30 - 1.6449 \times 0.03 = 1.250653$, well below 1.5, so the constraint binds over an interval in the middle of the move. The computed cost is 13.87380, some 16 per cent more energy than the unconstrained move, which is the price of the probabilistic margin. A little over half of that price is charged by the nominal limit alone: enforcing $v \leq 1.30$ without any back-off would cost 13.0193, so the $k\sigma$ term accounts for about 45 per cent of the increase.

The constrained problem is simple enough to solve by hand, which makes it a useful check. The solution is a boundary arc on which v sits exactly on the tightened limit V_t and $u = 0$, joined to entry and exit arcs on which u is linear in time; matching the distance travelled gives an entry time $t_1 = \frac{3}{2}(1 - 1/V_t) = 0.30063$, a peak magnitude $2V_t/t_1 = 8.32$ for the control, and a cost $16V_t^3/(9(V_t - 1)) = 13.87443$. The computed value is below this by 4.5×10^{-5} in relative terms, that is, very slightly better than

optimal, which is what one expects when an active path constraint is imposed only at the collocation nodes and may be exceeded a little between them.

The maximum relative local error, 8.9×10^{-10} on a fixed grid of 40 Legendre nodes, is worth a moment's thought, because it is smaller than the error in the answer. The estimate measures how nearly the transcription's own state and control polynomials satisfy the differential equations, and on this problem that pair is very nearly consistent: it is a smooth solution of the dynamics, and the dynamics here are linear. What it is not is the true optimal control, which has corners where the boundary arc is entered and left and which no single global polynomial can represent; the cost is 4.5×10^{-5} below the analytic optimum for exactly that reason. The two quantities answer different questions, and the discretisation error answers only the first of them. Local collocation with automatic mesh refinement, as used in Sections 8 and 15, would place nodes near the junctions and bring the cost closer, which is the quantity that has room to improve.

Table 14: Chance-constrained double integrator: summary

<i>Demonstrates</i>	A probabilistic path constraint imposed by deterministic tightening: the nominal bound is reduced by the back-off $k\sigma$ with $k = \Phi^{-1}(1-\epsilon)$
<i>Reference</i>	None. The problem is a constructed illustration
<i>Dimensions</i>	1 phase; 2 states, 1 control, 4 events, 1 path constraint, no static parameters
<i>Objective</i>	$J = \int_0^1 u^2 dt$
<i>Dynamics</i>	$\dot{x} = v, \quad \dot{v} = u$
<i>Initial conditions</i>	$x(0) = 0, \quad v(0) = 0, \quad t_0 = 0$
<i>Terminal conditions</i>	$x(t_f) = 1, \quad v(t_f) = 0, \quad t_f = 1$
<i>Path constraints</i>	$v + k\sigma - V_{\max} \leq 0$, the deterministic equivalent of $P\{v \leq V_{\max}\} \geq 1 - \epsilon$ for a Gaussian velocity estimate of standard deviation σ
<i>Bounds</i>	$-5 \leq x, v \leq 5, \quad -20 \leq u \leq 20, \quad t_0 = 0, t_f = 1$ both fixed
<i>Constants</i>	$V_{\max} = 1.30, \quad \sigma = 0.03$ (constant and illustrative; not propagated), $k = \Phi^{-1}(0.95) = 1.6449$, giving $\epsilon = 5$ per cent and a tightened limit $V_t = 1.250653$
<i>Discretisation</i>	Legendre pseudospectral, 40 nodes, automatic differentiation, automatic scaling, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	None (fixed mesh)
<i>Known solution</i>	No published value. Analytically, the unconstrained move costs 12 with $v_{\max} = 1.5$; with the boundary arc active the cost is $16V_t^3/(9(V_t - 1)) = 13.87443$, the boundary arc runs from $t_1 = \frac{3}{2}(1 - 1/V_t) = 0.30063$ to $1 - t_1$, and $ u $ peaks at $2V_t/t_1 = 8.32$
<i>Computed</i>	$J = 1.387380 \times 10^{-1}$, below the analytic value by 4.5×10^{-5} in relative terms; maximum relative local error 8.851978×10^{-10}
<i>Source</i>	<code>examples/chance_constraint/chance_constraint.cxx</code>

Results

```

PSOPT results summary
=====

Problem:      Chance-constrained double integrator
CPU time (seconds):  1.477650e-01
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  1.387380e+01
Phase 1 endpoint cost function value:    0.000000e+00
Phase 1 integrated part of the cost:      1.387380e+01
Phase 1 initial time:                    0.000000e+00
Phase 1 final time:                      1.000000e+00
Phase 1 maximum relative local error:    9.161908e-10
NLP solver reports:      The problem has been solved!

```

The run is short: 51 IPOPT iterations and 0.473 seconds in total, of which 0.390 seconds is the NLP solve. The residual difference from the analytic optimum is the discretisation of the junctions and not a failure of convergence; IPOPT reports the problem solved, and the objective is unchanged in all eight printed digits, 1.3873797×10^1 , over the final twenty iterations.

14 Manutec r3 robot: a multibody model and *hp*-adaptive mesh refinement

Most of the plants in this document are written out in a few lines of algebra. This one is not. The dynamics used here are the DLR model 2 of the Manutec r3 industrial robot, due to Otter and Turk [13] and documented as a benchmark by Franke and Otter [6]; von Stryk and Schlemmer [20] used it as an optimal control test case. It covers the first three links, with the equation of motion

$$\mathbf{M}(\mathbf{q}(t)) \ddot{\mathbf{q}}(t) = \mathbf{V}(\mathbf{q}(t), \dot{\mathbf{q}}(t)) + \mathbf{G}(\mathbf{q}(t)) + \mathbf{D} \mathbf{u}(t)$$

where $\mathbf{q} = [q_1 \ q_2 \ q_3]^T$ are the relative joint angles, $\mathbf{u}$ are the three motor voltages, $\mathbf{M}$ is the symmetric inertia matrix, $\mathbf{V}$ collects the Coriolis and centrifugal torques, $\mathbf{G}$ the gravitational torques, and $\mathbf{D} = \text{diag}(-126, 252, 72)$ N m/V converts voltages into generalised forces. None of these terms is a textbook idealisation: they carry the measured link masses, lengths, centre of mass offsets and inertia tensors of the real machine, and they are evaluated by a routine translated from the DLR simulation code and shipped in full with the example¹, which the user function `dae` does no more than

¹Dr Martin Otter of DLR, Germany, has kindly authorised the author to publish a translated form of subroutine R3M2SI as part of the PSOPT distribution.

call. A mechanical model of realistic complexity can therefore be handed to PSOPT as a black box, with automatic differentiation taking derivatives through it and no Jacobian written by hand.

The manoeuvre posed here is a point to point transfer in fixed time: move from rest at $\mathbf{q}(0) = (0, -1.5, 0)$ to rest at $\mathbf{q}(t_f) = (1, -1.95, 1)$ radians in $t_f = 0.53$ s while minimising the integral of $\mathbf{u}^T \mathbf{u}$, subject to the machine's joint angle, joint rate and voltage limits. The horizon is tight enough that the first motor cannot be driven gently: the computed u_1 sits on its lower limit of -7.5 V for roughly the first 0.035 s, sweeps through zero and returns to $+7.5$ V at the final time, while u_2 and u_3 stay comfortably inside their bounds (Figure 19). The joint rate limits are approached but never reached, $\dot{q}_1$ peaking just under its 3 rad/s bound.

That saturated arc is what makes the example the natural place to look at *hp*-adaptive refinement. Setting `algorithm.mesh_refinement = "automatic"` with a global Legendre discretisation hands the mesh to PSOPT's *hp* driver, which measures the decay of the Legendre coefficients on each interval and raises the polynomial degree where the solution looks smooth, splitting an interval only where raising the degree has stopped paying or the degree cap (here $N_{\max} = 12$) has been reached; growth is limited to forty per cent of the node count per iteration.

As shipped, however, the driver has nothing to do. The 20 nodes given in the source already meet the requested ODE tolerance of 10^{-5} , at 1.34×10^{-6} , so the refinement stops after one pass and the remaining entries of the source's node list are never used. To see the mechanism work, set `algorithm.ode_tolerance` to 10^{-9} : the driver then chooses 21, 25, 35 and finally 46 nodes, and the error estimate falls monotonically through 1.34×10^{-6} , 5.11×10^{-7} , 6.88×10^{-8} , 2.63×10^{-8} and 3.51×10^{-9} : three orders of magnitude for rather more than twice the nodes, which is what *p*-refinement on a smooth solution should look like.

The objective is the reassuring part of that experiment. It reads 2.040442×10^1 on the first mesh and 2.040446×10^1 on the last, two parts in a million apart, so the first mesh was converged and the estimate was right to stop. Contrast the minimum time to climb problem of §15, where a low order scheme is held fixed and accuracy is bought purely by adding nodes where the error is largest.

Results

```
PSOPT results summary
=====
```

```
Problem:      Manutec r3 robot problem
CPU time (seconds):  1.581370e-01
NLP solver used:    IPOPT
```

```
Optimal (unscaled) cost function value:  2.040442e+01
Phase 1 endpoint cost function value:  0.000000e+00
```

Table 15: Manutec r3 robot: summary

<i>Demonstrates</i>	A realistic multibody model used as a black box within <code>dae</code> , and automatic <i>hp</i> -adaptive mesh refinement of a global pseudospectral discretisation
<i>Reference</i>	DLR model 2 of Otter and Turk [13]; benchmark description by Franke and Otter [6]; optimal control of the same robot by von Stryk and Schlemmer [20]
<i>Dimensions</i>	1 phase; 6 states, 3 controls, 12 events, no path constraints, no static parameters
<i>Objective</i>	$J = \int_0^{t_f} \mathbf{u}(t)^T \mathbf{u}(t) dt$ with t_f fixed (OBJ_OPTION 3; the source also offers minimum time and regularised minimum time variants)
<i>Dynamics</i>	$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \mathbf{V}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{G}(\mathbf{q}) + \mathbf{D}\mathbf{u}$, written as six first order equations in $\mathbf{x} = [\mathbf{q}^T \dot{\mathbf{q}}^T]^T$. The inertia, Coriolis and gravity terms are evaluated by the translated DLR routine <code>r3m2si</code> in the example source
<i>Initial conditions</i>	$\mathbf{q}(0) = (0, -1.5, 0) \text{ rad}, \quad \dot{\mathbf{q}}(0) = \mathbf{0}$
<i>Terminal conditions</i>	$\mathbf{q}(t_f) = (1, -1.95, 1) \text{ rad}, \quad \dot{\mathbf{q}}(t_f) = \mathbf{0}$
<i>Bounds</i>	$ q_i \leq (2.97, 2.01, 2.86) \text{ rad}, \quad \dot{q}_i \leq (3.0, 1.5, 5.2) \text{ rad/s}, \quad u_i \leq 7.5 \text{ V}, \quad t_0 = 0, \quad t_f = 0.53 \text{ s}$
<i>Constants</i>	$g = 9.81 \text{ m/s}^2$, zero payload, and the DLR link and motor data (masses, lengths, centres of mass, inertia tensors, gear ratios, voltage-force constants) listed at the head of <code>main</code> in the source
<i>Discretisation</i>	Legendre pseudospectral with the standard differentiation matrix, 20 nodes initially, automatic differentiation, automatic scaling, state-based defect scaling, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	Automatic, <i>hp</i> (<i>ph</i>) variant for global pseudospectral methods; ODE tolerance 10^{-5} , at most 5 iterations, growth capped at 0.4, interval orders in [3, 12]; the tolerance is met on the initial 20 nodes, so no refinement takes place
<i>Known solution</i>	None: neither the example source nor this document quotes a published value of the minimum energy cost for these boundary conditions and this horizon
<i>Computed</i>	$J = 2.040442 \times 10^1$, with a maximum relative local error of 1.336018×10^{-6} against a requested 10^{-5} , in 0.45 s of CPU time
<i>Source</i>	<code>examples/manutec/manutec.cxx</code>

Table 16: Mesh refinement statistics: Manutec r3 robot problem

Iter	DM	M	NV	NC	OE	CE	JE	HE	RHS	$\epsilon_{\max}$	CPU _a
1	LGL-ST	20	182	133	32	32	30	0	640	1.336e-06	1.331e-01
CPU _b	-	-	-	-	-	-	-	-	-	-	2.509e-02
-	-	-	-	-	32	32	30	0	640	-	1.581e-01

Key: Iter=iteration number, DM= discretization method, M=number of nodes, NV=number of variables, NC=number of constraints, OE=objective evaluations, CE = constraint evaluations, JE = Jacobian evaluations, HE = Hessian evaluations, RHS = ODE right hand side evaluations, $\epsilon_{\max}$ = maximum relative ODE error, CPU_a = CPU time in seconds spent by NLP algorithm, CPU_b = additional CPU time in seconds spent by PSOPT

```

Phase 1 integrated part of the cost:  2.040442e+01
Phase 1 initial time:      0.000000e+00
Phase 1 final time:       5.300000e-01
Phase 1 maximum relative local error:  1.336034e-06
NLP solver reports:       The problem has been solved!

```

The single-row refinement table is in Table 16. The solve is cheap: 860 evaluations of the right-hand side and 0.45 s in total, of which 0.04 s was spent by PSOPT outside the NLP solver. The Hessian evaluation count is zero because IPOPT was run in limited-memory mode.

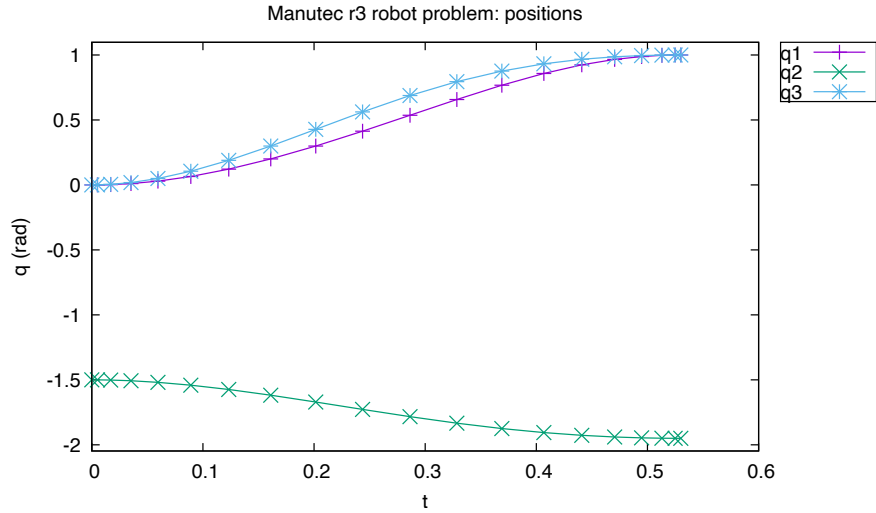


Figure 17: Joint angles q_1 , q_2 and q_3 for the Manutec r3 minimum energy problem

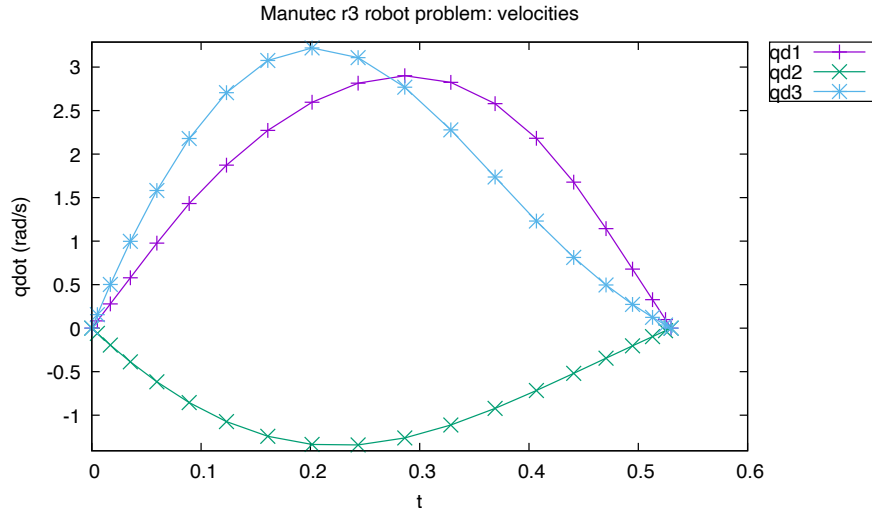


Figure 18: Joint rates $\dot{q}_1$, $\dot{q}_2$ and $\dot{q}_3$ for the Manutec r3 minimum energy problem

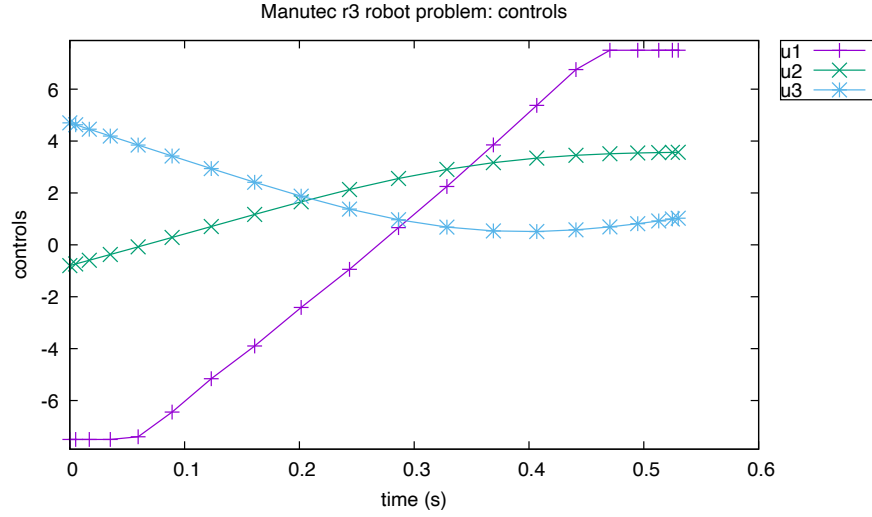


Figure 19: Motor voltages u_1 , u_2 and u_3 for the Manutec r3 minimum energy problem. The first control is saturated at -7.5 V over the initial arc and returns to $+7.5$ V at the final time

15 Minimum time to climb: a model driven by look-up tables

A supersonic aircraft starts at sea level at 424.26 ft/s and must reach 65 600 ft at 968.148 ft/s in as little time as possible, with the flight path angle zero at both ends and the angle of attack as the only control. The cost is the final time itself and nothing else, so the entire problem is carried by the dynamics; the brachistochrone of §2 is the analytic minimum-time problem of this document and this is the practical one. It is a long-standing benchmark for trajectory optimisation software: it descends from the energy-state analysis of supersonic climb by Bryson, Desai and Hoffman [3], is treated at length by Betts [2], and is taken here in the form given in the GPOPS user’s manual [14].

What distinguishes it from the other examples is that the model is not written in closed form. The lift-curve slope $c_{L\alpha}$, the zero-lift drag coefficient c_{D0} and the induced drag factor η are each obtained by one-dimensional spline interpolation of nine tabulated values against Mach number, and the thrust $T(M, h)$ by two-dimensional spline interpolation of a 10×10 table against Mach number and altitude. Air density and the speed of sound come from the 1976 standard atmosphere, through the library’s `include/us76.h`, whose lower branch is the closed-form solution of the standard’s layer equations written without any comparison that a derivative tape could freeze. The right-hand side is therefore only as smooth as the spline interpolation of the aerodynamic tables makes it, and that shows in the results: the maximum relative local error starts at 3.6×10^{-2} and reaches only 2.7×10^{-4} over three mesh refinement iterations, where a problem with an analytic right-hand side of the same size would have reached 10^{-9} (compare §2).

A model of this kind raises a difficulty that is easy to miss and needs stating plainly, because it applies to any look-up table used under automatic differentiation. Selecting an atmospheric layer, or a spline interval, means comparing a quantity that depends on the states against a table of breakpoints. An ordinary C++ comparison on a taped quantity is resolved once, when the derivative tape is recorded, and is then frozen: the layer or the interval stays at whatever it was at the point of taping, and the model silently becomes wrong as soon as the trajectory moves out of it.

There are two ways out, and this example has used both. The first is to record the comparison on the tape, with the conditional operations PSOPT provides: every layer’s formula is evaluated and `psopt_cond_lt` selects among them, so the choice is re-made at every evaluation. That works, and it is what PSOPT’s interpolation routines do for their spline intervals, but it has a sharp edge: an unselected layer’s extrapolated temperature goes negative (the tropospheric lapse rate reaches 0 K near 44 km) and a negative base in the pressure power law puts a NaN on the tape even in the arm that is discarded, so the local temperature has to be floored.

The second is to have no comparison to record. The atmosphere now comes from the library’s `include/us76.h`, which writes the temperature as the sum of the lapse rates against smooth positive parts and the pressure as the closed-form solution of hydrostatic equilibrium through those same layers: one expression, no branches, and a kilometre

clear of a layer boundary it reproduces the layer equations to two parts in 10^5 . Where the selection can be removed instead of taped, removing it is the better answer, and it leaves one implementation of the standard atmosphere in the distribution instead of three. The altitude is still passed in as the taped variable and not through `.value()`, which would leave the density and the speed of sound with no sensitivity to altitude at all.

The computed minimum time is 321.01 s, and the trajectory has the shape expected of a supersonic climb: about 30 s accelerating in level flight, a climb to roughly 31 000 ft, a shallow dive back to about 26 000 ft in which the aircraft accelerates through the transonic drag rise, and then a zoom climb during which the speed falls from its peak near 1 700 ft/s to the required 968.148 ft/s. The angle of attack stays well inside its $\pm 20^\circ$ bounds, between 5.8° and -6.6° , so no bound is active.

Results

```
PSOPT results summary
=====

Problem:      Minimum time to climb for a supersonic aircraft
CPU time (seconds):  3.592644e+00
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  3.210142e+02
Phase 1 endpoint cost function value:  3.210142e+02
Phase 1 integrated part of the cost:    0.000000e+00
Phase 1 initial time:      0.000000e+00
Phase 1 final time:       3.210142e+02
Phase 1 maximum relative local error:  2.709090e-04
NLP solver reports:      The problem has been solved!
```

The mesh refinement history is in Table 18. It is the document’s example of local, or *h*-type, refinement: the discretisation is a fixed low-order collocation scheme and accuracy is bought by adding nodes where the estimated error is largest and, once, by raising the order of the scheme from trapezoidal to Hermite-Simpson after the second iteration. This is the opposite of the *hp*-adaptive strategy used for the Manutec robot in §14, where the polynomial degree within each interval is the variable of first resort and the number of intervals changes only when raising the degree stops paying.

The refinement stopped on its tolerance, at 58 nodes and a maximum relative local error of 2.7×10^{-4} against a requested 10^{-3} , having taken 30 765 evaluations of the right-hand side and 8.3 s. Two of the three iterations bought very little, taking 3.6×10^{-2} to 1.5×10^{-2} for twelve more nodes, and the factor of fifty at the third came from the change of scheme and not from the mesh. The slow decrease under *h*-refinement alone is

Table 17: Minimum time to climb: summary

<i>Demonstrates</i>	A minimum-time problem whose model is driven by tabulated data: 1-D and 2-D spline interpolation, tape-safe interval selection under automatic differentiation, and local (h) mesh refinement with a change of collocation method
<i>Reference</i>	GPOPS user's manual [14]; also in Betts [2]
<i>Dimensions</i>	1 phase; 4 states, 1 control, 7 events, no path constraints, no static parameters
<i>Objective</i>	$J = t_f$
<i>Dynamics</i>	$\dot{h} = v \sin \gamma, \quad \dot{w} = -T(M, h)/I_{sp},$ $\dot{v} = \frac{1}{m} [T(M, h) \cos \alpha - D] - \frac{\mu}{(R_e + h)^2} \sin \gamma,$ $\dot{\gamma} = \frac{1}{mv} [T(M, h) \sin \alpha + L] + \cos \gamma \left[\frac{v}{R_e + h} - \frac{\mu}{v(R_e + h)^2} \right]$ with $m = w/g_0$, $M = v/c$, $c = 20.0468\sqrt{\theta}$ m/s, $D = \frac{1}{2}C_D S \rho v^2$, $L = \frac{1}{2}C_L S \rho v^2$, $C_L = c_{L\alpha}(M) \alpha$, $C_D = c_{D0}(M) + \eta(M) c_{L\alpha}(M) \alpha^2$, and $\rho(h)$, $\theta(h)$ from the 1976 standard atmosphere
<i>Initial conditions</i>	$h(0) = 0$ ft, $v(0) = 424.26$ ft/s, $\gamma(0) = 0$, $w(0) = 42\,000$ lb, $t_0 = 0$
<i>Terminal conditions</i>	$h(t_f) = 65\,600$ ft, $v(t_f) = 968.148$ ft/s, $\gamma(t_f) = 0$; $w(t_f)$ free
<i>Bounds</i>	$0 \leq h \leq 69\,000$ ft, $1 \leq v \leq 2000$ ft/s, $ \gamma \leq 89^\circ$, $0 \leq w \leq 45\,000$ lb, $ \alpha \leq 20^\circ$, $200 \leq t_f \leq 500$ s
<i>Constants</i>	$S = 530$ ft ² , $I_{sp} = 1600$ s, $g_0 = 32.174$ ft/s ² , $R_e = 2.09029 \times 10^7$ ft, $\mu = 0.14076539 \times 10^{17}$ ft ³ /s ² ; sea-level standard density 1.22521 kg/m ³ and temperature 288.15 K
<i>Data</i>	Tabulated aerodynamic and propulsion data, listed in the source and not reproduced here: $c_{L\alpha}$, c_{D0} and η at nine Mach numbers from 0 to 1.8, and thrust at ten Mach numbers by ten altitudes from 0 to 70 000 ft; and the eight-layer table of the 1976 standard atmosphere
<i>Discretisation</i>	Trapezoidal, 30 nodes initially, automatic differentiation, automatic scaling, Jacobian-based defect scaling, limited-memory Hessian, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	Automatic, local (h) variant; ODE tolerance 10^{-3} , at most 4 iterations, order switched to Hermite-Simpson after the second; 30 nodes grow to 58
<i>Known solution</i>	None: neither the example source nor this document quotes a published value of the minimum time
<i>Computed</i>	$t_f = 3.210142 \times 10^2$ s, with a maximum relative local error of 2.709068×10^{-4} and 8.3 s of CPU time
<i>Source</i>	<code>examples/climb/climb.cxx</code>

Table 18: Mesh refinement statistics: Minimum time to climb for a supersonic aircraft

Iter	DM	M	NV	NC	OE	CE	JE	HE	RHS	$\epsilon_{\max}$	CPU _a
1	TRP	30	152	128	48	48	48	0	2832	3.553e-02	4.746e-01
2	TRP	42	212	176	42	43	42	0	3569	1.471e-02	5.740e-01
3	H-S	58	349	240	87	88	73	0	20182	2.709e-04	2.445e+00
CPU _b	-	-	-	-	-	-	-	-	-	-	9.889e-02
-	-	-	-	-	177	179	163	0	26583	-	3.593e+00

Key: Iter=iteration number, DM= discretization method, M=number of nodes, NV=number of variables, NC=number of constraints, OE=objective evaluations, CE = constraint evaluations, JE = Jacobian evaluations, HE = Hessian evaluations, RHS = ODE right hand side evaluations, $\epsilon_{\max}$ = maximum relative ODE error, CPU_a = CPU time in seconds spent by NLP algorithm, CPU_b = additional CPU time in seconds spent by PSOPT

the interpolation showing through, as discussed above, and not a defect of the refinement algorithm.

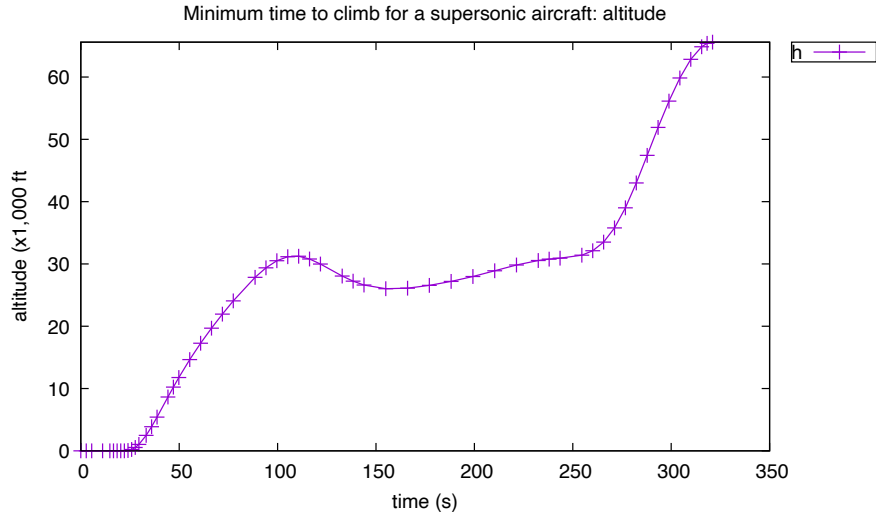


Figure 20: Altitude for the minimum time to climb problem

16 Multiphase launch vehicle ascent: linkages, mass discontinuities and terminal orbital elements

This is the largest application in the document: the ascent of a launch vehicle from Cape Canaveral into a geosynchronous transfer orbit, taken from the GPOPS documentation [14] and from Benson's thesis [1]. The vehicle carries nine strap-on solid boosters, a liquid first stage, a second stage and a payload of 4164 kg, for a lift-off mass of 301 454 kg.

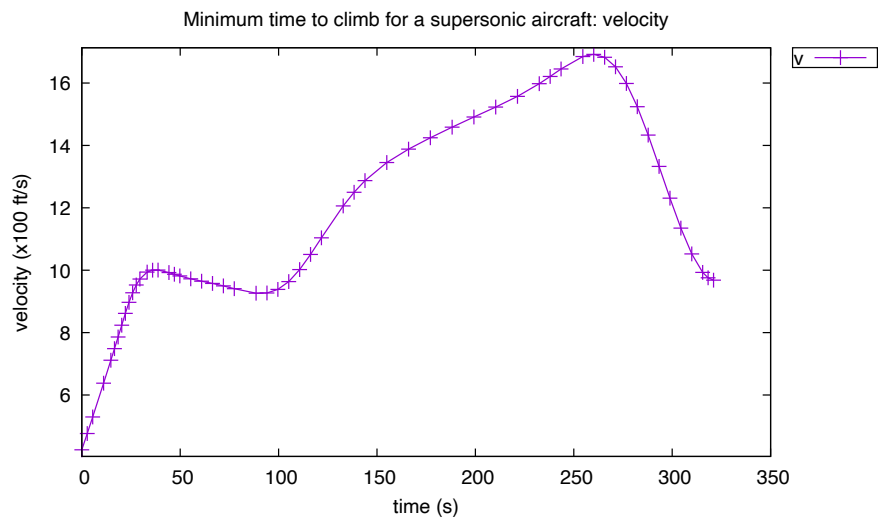


Figure 21: Velocity for the minimum time to climb problem

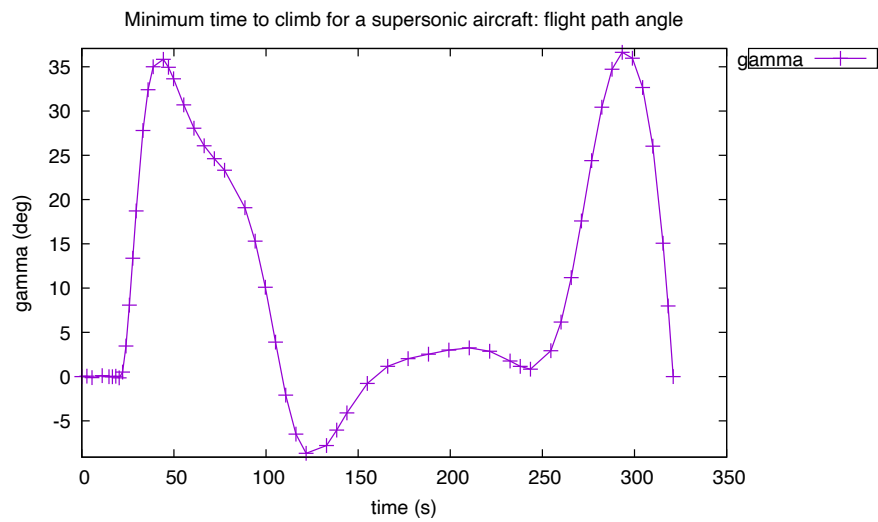


Figure 22: Flight path angle for the minimum time to climb problem

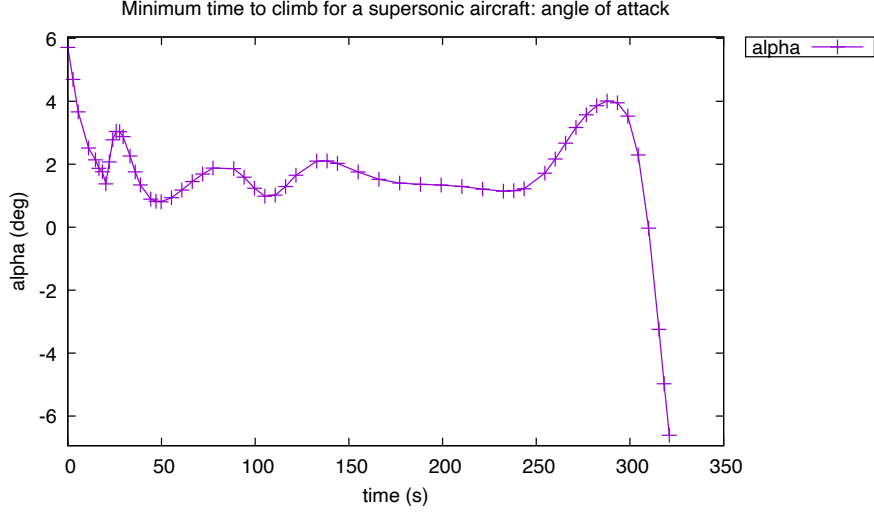


Figure 23: Angle of attack for the minimum time to climb problem

Six boosters are lit at lift-off and burn for 75.2 s; the remaining three are lit as the first six are jettisoned and burn until 150.4 s; the first stage then continues alone until 261 s; the second stage flies the rest of the ascent. Those four intervals are the four phases. Each has seven states (Earth-centred inertial position and velocity, and mass) and three controls, the components of the thrust direction, constrained by the path condition $\|\mathbf{u}\|^2 = 1$. The first three phase durations are fixed by the burn times; the final time of phase 4 is free within $[261, 961]$ s, and the objective is to maximise the mass delivered, posed as the minimisation of $-m^{(4)}(t_f)$.

The linkage machinery introduced in 3 does real work here. Each of the three junctions is handled by a call to `auto_link`, which emits eight constraints: one continuity condition for each of the seven states, $x_k^{(p)}(t_f) - x_k^{(p+1)}(t_0) = 0$, followed by continuity of time, $t_f^{(p)} = t_0^{(p+1)}$. Three junctions give the 24 linkages declared in the setup. The mass, however, is not continuous, so immediately after each call the example reaches back into the linkage vector and modifies the single entry corresponding to the mass, which is the seventh state and the penultimate entry written by that call, by subtracting the jettisoned dry mass. The three junctions therefore impose $m^{(p)}(t_f) - m^{(p+1)}(t_0) = \Delta m^{(p)}$ with $\Delta m^{(1)} = 13\,680$ kg (six booster casings of 2280 kg each), $\Delta m^{(2)} = 6840$ kg (the other three casings) and $\Delta m^{(3)} = 8830$ kg (the first-stage dry mass), while position and velocity remain continuous across all three. The pattern generalises: write the continuity conditions automatically, then perturb the rows that are not continuity conditions.

The terminal conditions are stated on the classical orbital elements of the target orbit and not on the Cartesian state. The `events` function of phase 4 converts the final position and velocity to $(a, e, i, \Omega, \omega, \nu)$ and constrains the first five with equal lower and upper bounds. This is the natural formulation because the mission requirement is an orbit, not a point: the true anomaly ν is deliberately left free, so the optimiser may

arrive anywhere on the target ellipse, and five conditions on six Cartesian coordinates would otherwise have to be chosen arbitrarily. The price is that the conversion must be differentiable. The quadrant corrections that would normally be written as sign tests on Ω , ω and ν are instead formed with `smooth_heaviside` of width 0.1, so that the automatic differentiation sees a smooth function of the final state.

The computed final mass is 7529.661 kg. Of this, 4164 kg is payload and 2480 kg is second-stage dry mass, so roughly 886 kg of the second stage's 16 820 kg of propellant is left unburnt; consistently, the ascent ends at 924.14 s, some 663 s into a stage with 700 s of burn time. The orbit is reached with margin and the lower mass bound of phase 4 is not active. The problem is solved on the two grids prescribed by the node vectors, 65 nodes and then 79, in 4.0 s of CPU time. The maximum relative local error falls from 1.85×10^{-6} on the coarse grid to 1.32×10^{-6} on the fine one, and on both it is concentrated in phase 4, the long free-final-time coast and burn to orbit; the three ascent phases sit between 2 and 3.4 parts in ten million throughout. Every phase is comfortably below 10^{-5} on both grids.

Results

PSOPT results summary

=====

```

Problem:      Multiphase vehicle launch
CPU time (seconds):  1.647166e+00
NLP solver used:    IPOPT

Optimal (unscaled) cost function value:  -7.529661e+03
Phase 1 endpoint cost function value:    0.000000e+00
Phase 1 integrated part of the cost:      0.000000e+00
Phase 1 initial time:  0.000000e+00
Phase 1 final time:    7.520000e+01
Phase 1 maximum relative local error:    2.029004e-07
Phase 2 endpoint cost function value:    0.000000e+00
Phase 2 integrated part of the cost:      0.000000e+00
Phase 2 initial time:    7.520000e+01
Phase 2 final time:     1.504000e+02
Phase 2 maximum relative local error:    2.190005e-07
Phase 3 endpoint cost function value:    0.000000e+00
Phase 3 integrated part of the cost:      0.000000e+00
Phase 3 initial time:     1.504000e+02
Phase 3 final time:      2.610000e+02
Phase 3 maximum relative local error:    3.158167e-07
Phase 4 endpoint cost function value:    -7.529661e+03
Phase 4 integrated part of the cost:      0.000000e+00
Phase 4 initial time:      2.610000e+02
Phase 4 final time:       9.241413e+02

```

Table 19: Multiphase launch vehicle ascent: summary

<i>Demonstrates</i>	A four-phase problem in which the linkages carry a state discontinuity: automatic state and time continuity from <code>auto_link</code> , with the mass condition modified to account for jettisoned hardware; terminal conditions written on orbital elements; free final time in the last phase
<i>Reference</i>	GPOPS manual, Rao et al. [14]; Benson [1]
<i>Dimensions</i>	4 phases; 7 states and 3 controls per phase; 7 events in phase 1 and 5 in phase 4, none in phases 2 and 3; 1 path constraint per phase; 24 linkage constraints; no static parameters
<i>Objective</i>	$J = -m^{(4)}(t_f)$, the negative of the mass delivered to the transfer orbit
<i>Dynamics</i>	$\dot{\mathbf{r}} = \mathbf{v}$, $\dot{\mathbf{v}} = -\frac{\mu}{\ \mathbf{r}\ ^3}\mathbf{r} + \frac{T}{m}\mathbf{u} + \frac{\mathbf{D}}{m}$, $\dot{m} = -\frac{T}{g_0 I_{sp}}$, with $\mathbf{D} = -\frac{1}{2}C_D A_{ref} \rho \ \mathbf{v}_{rel}\ \mathbf{v}_{rel}$, $\mathbf{v}_{rel} = \mathbf{v} - \boldsymbol{\omega} \times \mathbf{r}$ and $\rho = \rho_0 \exp[-(\ \mathbf{r}\ - R_e)/h_0]$. The thrust T and specific impulse I_{sp} take a different value in each phase
<i>Initial conditions</i>	$\mathbf{r}(t_0) = [R_e \cos \lambda_0 \ 0 \ R_e \sin \lambda_0]^T$ with $\lambda_0 = 28.5^\circ$, $\mathbf{v}(t_0) = \boldsymbol{\omega} \times \mathbf{r}(t_0)$ (at rest relative to the rotating Earth), $m(t_0) = 301\,454$ kg, $t_0 = 0$
<i>Terminal conditions</i>	On the orbital elements of the final state of phase 4: $a_f = 24\,361.14$ km, $e_f = 0.7308$, $i_f = 28.5^\circ$, $\Omega_f = 269.8^\circ$, $\omega_f = 130.5^\circ$; the true anomaly is free. The apogee radius $a_f(1 + e_f)$ is 42 164 km
<i>Path constraints</i>	$\ \mathbf{u}\ ^2 = 1$ in every phase, imposed with equal lower and upper bounds
<i>Linkages</i>	At each junction $p = 1, 2, 3$: $\mathbf{r}^{(p)}(t_f) = \mathbf{r}^{(p+1)}(t_0)$, $\mathbf{v}^{(p)}(t_f) = \mathbf{v}^{(p+1)}(t_0)$, $t_f^{(p)} = t_0^{(p+1)}$, $m^{(p)}(t_f) - m^{(p+1)}(t_0) = \Delta m^{(p)}$ with $\Delta m = (13\,680, 6840, 8830)$ kg
<i>Bounds</i>	$ r_j \leq 2R_e$, $ v_j \leq 10^4$ m/s, $-1 \leq u_j \leq 1$, mass bounded in each phase between the fully loaded and fully burnt values; $t_f^{(1)} = 75.2$, $t_f^{(2)} = 150.4$, $t_f^{(3)} = 261.0$ s fixed, $t_f^{(4)} \in [261, 961]$ s
<i>Constants</i>	Earth and atmosphere (μ , R_e , ω , ρ_0 , h_0 , g_0), vehicle geometry and drag (C_D , A_{ref}), and for each of the three propulsion elements a thrust, burn time, total mass and propellant mass from which the specific impulse and dry mass are derived. All are set in the source and are not reproduced here
<i>Discretisation</i>	Chebyshev pseudospectral with the standard differentiation matrix, 15 then 18 nodes in phases 1 to 3 and 20 then 25 in phase 4 (65 then 79 nodes in all), automatic differentiation, automatic scaling, limited-memory Hessian, IPOPT, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	None automatic; the node vectors prescribe two grids, solved in sequence (Table 20)
<i>Known solution</i>	No published value is quoted. The source cites [14] and [1] for the problem statement but records no optimal final mass, and none is asserted here
<i>Computed</i>	$J = -7.529661 \times 10^3$, that is $m^{(4)}(t_f) = 7529.661$ kg; $t_f = 9.241413 \times 10^2$ s; maximum relative local error 1.315138×10^{-6} , in phase 4
<i>Source</i>	<code>examples/launch/launch.cxx</code>

Table 20: Mesh refinement statistics: Multiphase vehicle launch

Iter	DM	M	NV	NC	OE	CE	JE	HE	RHS	$\epsilon_{\max}$	CPU _a
1	CGL-ST	65	658	560	106	111	83	0	7215	1.433e-06	7.356e-01
2	CGL-ST	79	798	672	52	53	50	0	4187	1.528e-06	7.540e-01
CPU _b	-	-	-	-	-	-	-	-	-	-	1.575e-01
-	-	-	-	-	158	164	133	0	11402	-	1.647e+00

Key: Iter=iteration number, DM= discretization method, M=number of nodes, NV=number of variables, NC=number of constraints, OE=objective evaluations, CE = constraint evaluations, JE = Jacobian evaluations, HE = Hessian evaluations, RHS = ODE right hand side evaluations, $\epsilon_{\max}$ = maximum relative ODE error, CPU_a = CPU time in seconds spent by NLP algorithm, CPU_b = additional CPU time in seconds spent by PSOPT

Phase 4 maximum relative local error: 1.528344e-06
NLP solver reports: The problem has been solved!

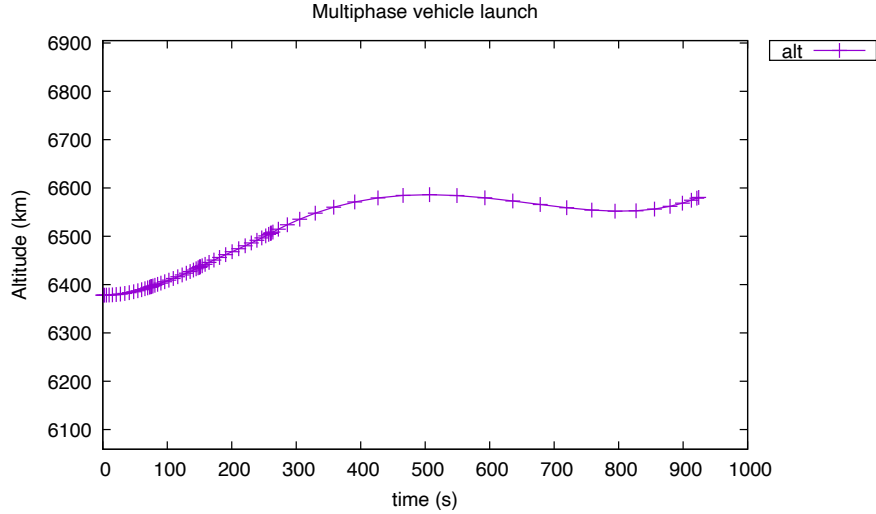


Figure 24: Altitude for the multiphase launch vehicle ascent

17 A two-link robotic arm: multiple-shooting transcription

Every example so far has been transcribed by collocation, which is what PSOPT does by default: the state and control are approximated by polynomials through the mesh points, and the dynamics are imposed by forcing the residual to zero at a finite set of collocation

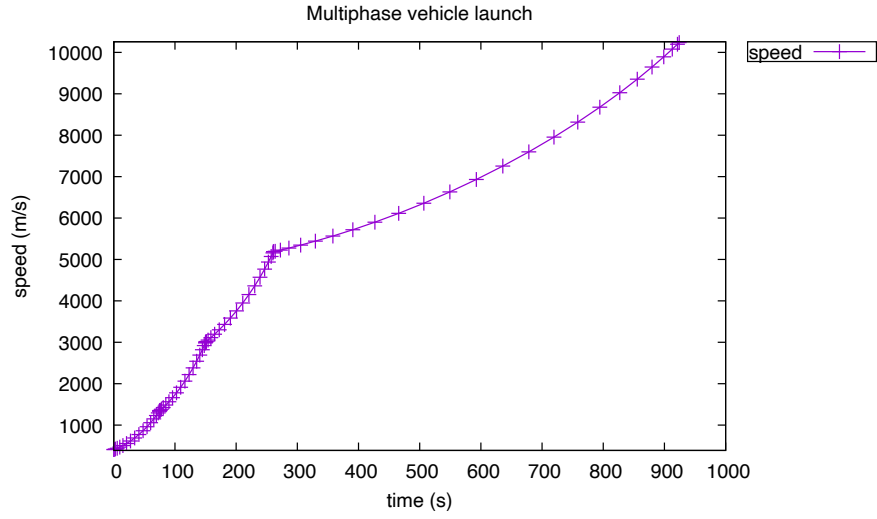


Figure 25: Speed for the multiphase launch vehicle ascent

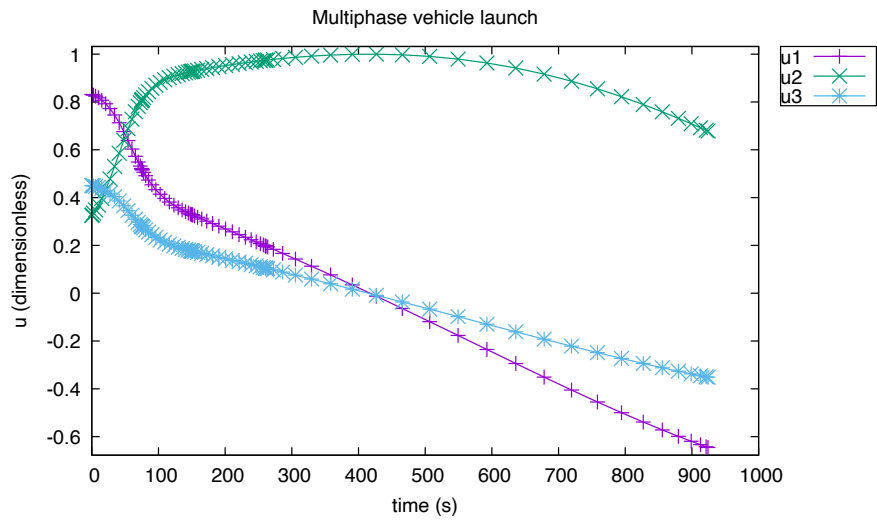


Figure 26: Components of the thrust direction for the multiphase launch vehicle ascent

points. This section is the counterpart for *multiple shooting*, which transcribes the same problem in a different way and is selected by setting `transcription_method` to `"multiple-shooting"`.

In multiple shooting the mesh interval is a *segment* rather than a collocation point. The decision variables are the state at the segment boundaries and the parameters of the control within each segment; an embedded integrator marches each segment forward from its left-hand boundary; and the constraints are the mismatches between where a segment arrives and where the next one is declared to begin. The trajectory is therefore an integrated one by construction, and the accuracy with which the differential equations are satisfied is set by the integrator and not by the polynomial through the nodes. That is the property the method is chosen for, and it is the one this example is here to display.

The problem

The problem is the minimum-time reorientation of a planar two-link arm. It is due to Weinreb and Bryson [21], whose Section IV states it with the tip mass present symbolically, as the ratio $\mu = M/m$ of tip mass to link mass. The $\mu = 1$ case with every coefficient evaluated, which is the form solved here, is Section 12.4.2 of Luus [9], and is also the form the PROPT user's guide carries. Two links, two joint torques bounded by unity in magnitude, four states (two angular rates and two angles) and a free final time. The arm starts at rest and must arrive at rest at a specified configuration.

The structure of its solution is what makes it the right example for this section. Minimising time with bounded torques makes the Hamiltonian linear in each control, so the optimal torques are bang-bang: u_1 switches once, u_2 switches twice, and between switches both sit hard against their bounds. A global Legendre polynomial through the mesh cannot represent a jump. The collocated control therefore rings around each switching instant, and the discretisation error of the whole solve is dominated by that ringing, not by the smooth arcs between the switches.

What the transcription changes

The example was solved by collocation until September 2026 and is now solved by multiple shooting. Passing a collocation method as the first command-line argument, as in `./twolinkarm Legendre`, solves the same problem the old way, which is how the comparison in Table 21 was made. Both columns use the same 40 mesh points, which under multiple shooting means 39 segments, so the transcription is the only thing that differs.

Multiple shooting is faster at both mesh sizes, eight times faster at eighty nodes, and satisfies the differential equations between two and three orders of magnitude more accurately. Neither of those is a surprise on a problem of this shape, and both are worth having.

The fourth column deserves more care than it usually receives, because it is the one a reader will want to quote. **The minimum time is not converged at this resolution under either transcription.** Refining the mesh lowers t_f under both, and the two

Table 21: The same problem under both transcriptions, at two mesh sizes. Times were measured on one core of an Ubuntu 24.04 machine with IPOPT 3.11.9 and should be read as indicative; the node counts and the error estimates should not.

Nodes	Transcription	CPU (s)	t_f	$\epsilon_{\max}$
40	Legendre collocation	0.75	2.988660	3.7×10^{-5}
40	Multiple shooting	0.48	2.985042	8.1×10^{-9}
80	Legendre collocation	4.79	2.983629	2.6×10^{-6}
80	Multiple shooting	0.61	2.983021	9.5×10^{-10}

transcriptions agree with each other far better than either agrees with itself across mesh sizes. What limits the answer here is how finely the *control* is parameterised, not how accurately the dynamics are integrated: replacing the fourth-order integrator with the eighth-order one at 79 segments returns the same 2.983021 to six figures, with a residual at machine precision. A bang-bang problem wants its switching instants as decision variables, which is what a multi-phase formulation with a switch at each phase boundary provides and what no uniform mesh of either kind can supply. Multiple shooting buys accuracy in the dynamics and a substantial saving in time; it does not buy a converged switching structure, and an example that showed only the first two would be selling the method instead of describing it.

There is an independent figure to measure that against. Luus [9] reports $t_f = 2.98228$ for this problem, obtained by iterative dynamic programming, a method that shares no machinery with either transcription used here. The four values in the table descend towards it as the mesh and the control parameterisation are refined, and the gap closes by roughly a factor of two at each step: 6.4×10^{-3} , 2.8×10^{-3} , 1.3×10^{-3} , 7.4×10^{-4} , reading down the table. That is the behaviour a converging minimum-time transcription should show against a number from an independent method, and it puts a size on the statement above. On the last row that gap is still 2.5×10^{-4} in relative terms, five orders of magnitude larger than the 9.5×10^{-10} with which the same solution satisfies the differential equations. The two columns are measuring different things, and the fourth is the weaker of them.

The settings

Four options configure the method, and the values used here were chosen by measurement on this problem and not left at their defaults:

- `ms_integrator` is "RK4". The eighth-order explicit scheme "RK8" costs about four times as much here and moves t_f in the seventh figure. "TRBDF2" and "ESDIRK3" are stiffly accurate, L-stable implicit schemes for stiff dynamics, which these are not.
- `ms_steps_per_segment` is 8. At 4 steps the maximum relative local error is 1.3×10^{-7} ; at 8 it is 8.1×10^{-9} , for about 0.2 s more. Beyond 8 the answer stops moving,

because the control parameterisation is what limits it.

- `ms_control_parameterisation` is "linear", meaning linear within each segment. Holding the control constant within a segment is cheaper and returns 2.985858 instead of 2.985042, which is the parameterisation showing through.
- The mesh is the 40 nodes the example has always used, giving 39 segments.

Table 22: Two-link robotic arm: summary

<i>Demonstrates</i>	Multiple-shooting transcription: segment-wise integration, the choice of integrator and of control parameterisation, and what the method does and does not improve on a bang-bang problem
<i>Reference</i>	Weinreb and Bryson [21], Sec. IV, for the arm with the tip mass symbolic; Luus [9], Sec. 12.4.2, for the evaluated $\mu = 1$ case solved here, which also reports $t_f = 2.98228$ by iterative dynamic programming
<i>Dimensions</i>	1 phase; 4 states, 2 controls, 8 events, no path constraints, no static parameters
<i>Objective</i>	$J = t_f$
<i>Dynamics</i>	Planar two-link manipulator; see <code>examples/twolinkarm/</code> for the equations of motion and the inertia terms
<i>Initial conditions</i>	$x_1(0) = x_2(0) = 0, \quad x_3(0) = 0.5, \quad x_4(0) = 0, \quad t_0 = 0$
<i>Terminal conditions</i>	$x_1(t_f) = x_2(t_f) = 0, \quad x_3(t_f) = 0.5, \quad x_4(t_f) = 0.522$
<i>Bounds</i>	$ u_1 \leq 1, \quad u_2 \leq 1, \quad 1 \leq t_f \leq 10$
<i>Discretisation</i>	Multiple shooting, 39 segments from 40 nodes, RK4 with 8 steps per segment, control linear within a segment, automatic differentiation, automatic scaling, NLP tolerance 10^{-6}
<i>Mesh refinement</i>	None (fixed segmentation)
<i>Known solution</i>	None in closed form. The optimal control is bang-bang with three switches in total
<i>Computed</i>	$t_f = 2.985042$, maximum relative local error 8.08×10^{-9} , 0.46 s. Under Legendre collocation on the same mesh, $t_f = 2.988660$ at 3.66×10^{-5} in 0.72 s
<i>Source</i>	<code>examples/twolinkarm/twolinkarm.cxx</code>

Results

PSOPT results summary

=====

```

Problem:      Two link robotic arm
CPU time (seconds):  4.522490e-01
NLP solver used:    IPOPT

```

Table 23: Mesh refinement statistics: Two link robotic arm

Iter	DM	M	NV	NC	OE	CE	JE	HE	RHS	$\epsilon_{\max}$	CPU _a
1	LGL-ST	40	242	169	32	32	32	0	81152	8.084e-09	4.299e-01
CPU _b	-	-	-	-	-	-	-	-	-	-	2.237e-02
-	-	-	-	-	32	32	32	0	81152	-	4.522e-01

Key: Iter=iteration number, DM= discretization method, M=number of nodes, NV=number of variables, NC=number of constraints, OE=objective evaluations, CE = constraint evaluations, JE = Jacobian evaluations, HE = Hessian evaluations, RHS = ODE right hand side evaluations, $\epsilon_{\max}$ = maximum relative ODE error, CPU_a = CPU time in seconds spent by NLP algorithm, CPU_b = additional CPU time in seconds spent by PSOPT

```

PSOPT release number:    2026-09-09-51-g229c960
PSOPT build date:       2026-09-17 13:06:42
Date and time of this run:  Fri Sep 18 08:21:06 2026

Optimal (unscaled) cost function value:  2.985042e+00
Phase 1 endpoint cost function value:    2.985042e+00
Phase 1 integrated part of the cost:      0.000000e+00
Phase 1 initial time:                    0.000000e+00
Phase 1 final time:                      2.985042e+00
Phase 1 maximum relative local error:    8.083656e-09
NLP solver reports:      The problem has been solved!

```

The algorithm options PSOPT reports for this run name the transcription and its settings:

```

TRANSCRIPTION METHOD:      multiple-shooting
SHOOTING SEGMENTS:        39
SEGMENT BOUNDARIES:       uniform in normalised time
MS INTEGRATOR:            RK4
MS STEPS PER SEGMENT:      8
MS CONTROL PARAMETERISATION: linear
MS PATH SAMPLES:          0
NLP METHOD:                IPOPT
HESSIAN OPTION:           limited-memory
NLP TOLERANCE:            1.000000e-06
NLP MAX ITERATIONS:       1000
SCALING METHOD:            automatic
DEFECT SCALING:           state-based
DIFFERENTIATION:          automatic
MESH REFINEMENT:          manual

```

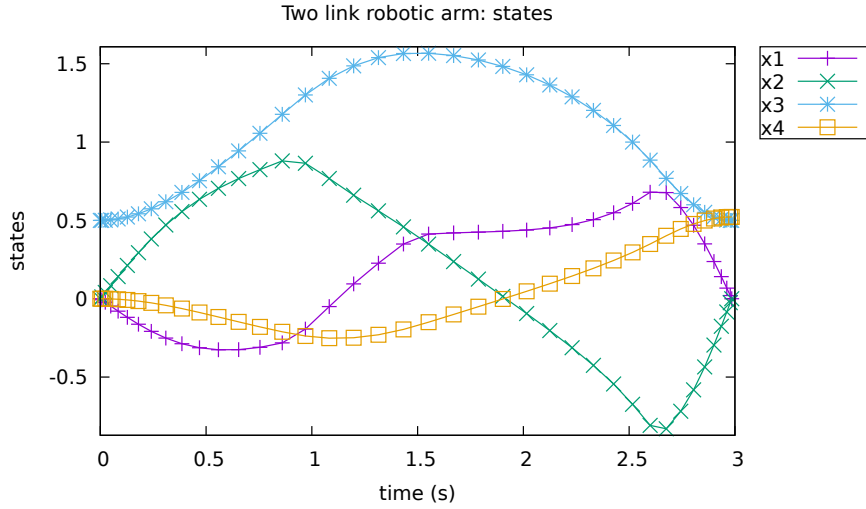


Figure 27: States for the two-link arm problem

This problem returns in Section 19, where the payload the arm carries is no longer known exactly and the same transcription is used to design a control that works for every payload it might be carrying.

18 The Python interface

Every problem in this document so far has been posed in C++. PSOPT also has a Python front end, in which the dynamics, costs, events and observations are written as ordinary Python functions over CasADi symbols. Nothing about the solver changes: the Python maths is traced, emitted as `adouble`-templated C++, compiled to a small shared library and taped by CppAD exactly as a hand-written example is, and the same core then solves it. What changes is the amount of ceremony between having a problem and having an answer.

This section shows three complete Python programs. They are printed in full and not excerpted, because the point of the interface is that a whole problem fits on a page and a reader should be able to see that it does. All three are in the distribution under `python/examples/`, along with twelve others; `python/README.md` describes the build and the installation.

What the interface looks like

A problem is built in four steps, and they are the same four steps the C++ interface takes:

1. `prob = psopt.Problem(name=...)`, then `ph = prob.add_phase(...)` with the dimensions of the phase, which correspond to the two levels of setup;

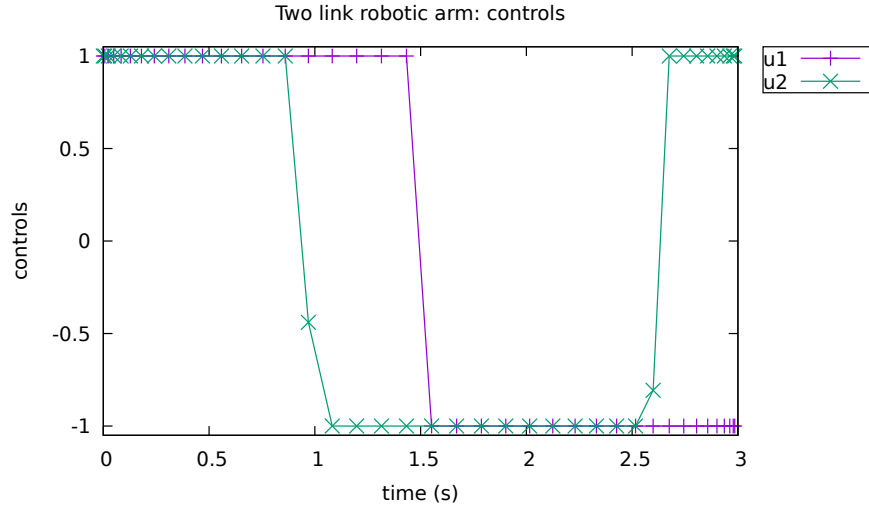


Figure 28: Controls for the two-link arm problem. Both torques are bang-bang: u_1 switches once and u_2 twice, and between switches each sits against its bound. This is the structure that makes the problem awkward for a global polynomial and suited to segment-wise integration.

2. the maths, as Python callables over CasADi symbols: `ph.dynamics`, `ph.path`, `ph.integrand`, `ph.endpoint`, `ph.events` and, for estimation, `ph.observation`;
3. the bounds and the initial guess, as lists and NumPy arrays;
4. `alg = psopt.Algorithm(...)` and `sol = prob.solve(alg)`.

`psopt.Algorithm` accepts every field of the C++ `Alg` structure, so anything this document demonstrates in C++ can be asked for from Python: the collocation method, *hp*-adaptive mesh refinement, the integrated-residual transcription, multiple shooting, PSOPT's own SQP solver and its QP backends, and the parameter-estimation options. An option left unset is not sent, so PSOPT's own default applies.

The solution object carries more of what the solve produced than the trajectory. `sol.objective`, `sol.states`, `sol.controls` and `sol.time` are the trajectory. `sol.status` says whether the solve converged, which the objective on its own cannot. `sol.costates` is the discrete adjoint, and `sol.duals` carries the Hamiltonian, the path and event multipliers and the relative local error. For an estimation problem, `sol.parameter_statistics` carries the covariance and the confidence intervals. States and controls come back as NumPy arrays, so plotting is a matter of

```
import matplotlib.pyplot as plt
plt.plot(sol.time, sol.controls[0])
```

and not of the gnuplot scripts the C++ examples write.

A single-phase problem with free final time

The first example is the brachistochrone of Section 2, so that the Python statement of a problem can be read directly against the C++ statement of the same one. It is about forty lines including its commentary.

Two things in it deserve notice. The dynamics are a single expression: `ca.vertcat` of the three derivatives, with the states and control arriving as CasADi symbols, and no `adouble` declarations or indices to keep track of. And the example checks itself: the cycloid through the two endpoints is solved for inside the script, with `brentq` from SciPy, and the final time PSOPT returns is compared against it. The agreement is to ten decimal places.

```

"""Brachistochrone -- the minimum-time descent curve, and what the costates say.

The bead slides without friction from (0,0) to x = 2, y = 2 under gravity; the
control is the path angle theta. Minimising the final time gives the cycloid
Johann Bernoulli asked for in 1696.

This is the Python port of examples/brac1 in the C++ distribution, whose answer
is tf = 8.247591e-01 s. It is also the example to read for what a solve returns
beyond the trajectory, because a minimum-time problem has a sharp check on it:
with the objective written as the Mayer term tf, the Hamiltonian along an optimal
trajectory of an autonomous free-final-time problem is identically -1. If the
Hamiltonian PSOPT reports is not flat and not -1, the solve has not converged
however plausible the objective looks.
"""
import numpy as np
import casadi as ca
from _common import psopt, report

N = 40
prob = psopt.Problem(name="brachistochrone")
ph = prob.add_phase(nstates=3, ncontrols=1, nevents=5)
ph.nodes = [N]

# x' = v sin(theta), y' = v cos(theta), v' = g cos(theta)
ph.dynamics = lambda x, u, p, t: ca.vertcat(x[2] * ca.sin(u[0]),
                                             x[2] * ca.cos(u[0]),
                                             9.8 * ca.cos(u[0]))

ph.endpoint = lambda xi, xf, p, t0, tf: tf # minimise the final time
ph.events = lambda xi, xf, p, t0, tf: ca.vertcat(xi[0], xi[1], xi[2], xf[0], xf[1])

ph.bounds.lower.states = [0.0, 0.0, 0.0]
ph.bounds.upper.states = [20.0, 20.0, 20.0]
ph.bounds.lower.controls = [0.0]
ph.bounds.upper.controls = [2.0 * np.pi]
ph.bounds.lower.events = [0.0, 0.0, 0.0, 2.0, 2.0]
ph.bounds.upper.events = [0.0, 0.0, 0.0, 2.0, 2.0]
ph.bounds.t0 = (0.0, 0.0)
ph.bounds.tf = (0.0, 10.0) # the final time is free

ph.guess.states = np.vstack([np.linspace(0.0, 2.0, N),
                             np.linspace(0.0, 2.0, N),
                             np.linspace(0.0, 2.0, N)])
ph.guess.controls = np.ones((1, N))
ph.guess.time = np.linspace(0.0, 2.0, N).reshape(1, N)

alg = psopt.Algorithm(collocation_method="Legendre", nlp_tolerance=1.0e-6,
                      nlp_iter_max=1000, print_level=0)
sol = prob.solve(alg)

ok = report(sol, "Brachistochrone (C++ example brac1)", reference=8.247591e-01, tol=1.0e-5)

# ---- what the solve produced beyond the trajectory -----
print("final time      : %.9g s" % sol.time[-1])
print("costates        : %s (one row per state)" % (sol.costates.shape,))

# lambda_x and lambda_y are constant here: neither x nor y appears in the dynamics
# or the cost, so their adjoint equations are lambda' = 0. That is a statement about
# the problem, not about PSOPT, which makes it a fair check on the solver.
lam = sol.costates
print("lambda_x        : %+.6e ... %+.6e" % (lam[0, 0], lam[0, -1]))

```

```

print("lambda_y      : %+.6e ... %+.6e" % (lam[1, 0], lam[1, -1]))
print("              both constant: neither x nor y appears in f or L")

H = sol.duals.hamiltonian.ravel()
print("Hamiltonian   : mean %+.6f, spread %.2e" % (H.mean(), H.max() - H.min()))
print("              theory: identically -1")
# The gate is 1e-3 rather than the NLP tolerance. The Hamiltonian is reconstructed
# from the discrete adjoint on 40 nodes, so what it agrees with -1 to is set by the
# discretisation, not by how tightly the NLP was solved. On this mesh it comes out
# at about 1.4e-4, and refining the mesh is what moves it.
ok = ok and abs(H.mean() + 1.0) < 1.0e-3 and (H.max() - H.min()) < 5.0e-3

# The cycloid through (0,0) and (2,2) can be solved for directly, which gives an
# answer that owes nothing to PSOPT. With x = a(theta - sin theta),
# y = a(1 - cos theta), the endpoint condition x = y fixes theta and then a, and
# tf = sqrt(a/g) * theta.
from scipy.optimize import brentq # noqa: E402
th = brentq(lambda s: (s - np.sin(s)) - (1.0 - np.cos(s)), 1.0, 2.0 * np.pi - 1.0e-9)
a = 2.0 / (1.0 - np.cos(th))
tf_exact = np.sqrt(a / 9.8) * th
print("closed form    : tf = %.9g s   (cycloid, (0,0) to (2,2))" % tf_exact)
print("difference      : %.2e" % abs(sol.time[-1] - tf_exact))
ok = ok and abs(sol.time[-1] - tf_exact) < 1.0e-6

print("\nRESULT: %s" % ("PASS" if ok else "FAIL"))
raise SystemExit(0 if ok else 1)

```

It prints:

```

---- Brachistochrone (C++ example brac1)
objective      : 0.824759142876
reference      : 0.8247591   (relative difference 4.29e-08)
status         : <PSOPT converged rc=0 flag=0 cpu=0.395s>
final time     : 0.824759143 s
costates       : (3, 40)   (one row per state)
lambda_x       : -1.494515e-01 ... -1.492043e-01
lambda_y       : -5.697917e-02 ... -5.698468e-02
               both constant: neither x nor y appears in f or L
Hamiltonian    : mean -0.999860, spread 4.49e-04
               theory: identically -1
closed form    : tf = 0.824759143 s   (cycloid, (0,0) to (2,2))
difference     : 2.81e-10
RESULT: PASS

```

The last three checks should be read. λ_x and λ_y are constant because neither x nor y appears in the dynamics or the cost, so their adjoint equations are $\dot{\lambda} = 0$; that is a property of the problem and not of the solver, which makes it a fair test of the solver. The Hamiltonian of an autonomous minimum-time problem with the cost written as the Mayer term t_f is identically -1 along an optimal trajectory, and the computed one has a mean of -0.99986 and a spread of 4.5×10^{-4} over the mesh. That residual is set by the discretisation and not by the NLP tolerance, which is why the script gates it at 10^{-3} and says so.

Path constraints

The second example is the obstacle problem: a vehicle at constant speed, with its heading as the control, crossing a plane past two circular obstacles in minimum time. Each obstacle is one entry of the path vector, bounded below.

The part to read is what comes after the solve. The multiplier of the obstacle the trajectory grazes is large and the multiplier of the one it misses is exactly zero, which is complementary slackness, and it is visible directly in `sol.duals.dual_path`. The example also records why its initial guess is what it is: the problem has more than one taut path, and a guess of zero heading converges to the other homotopy class, which is a genuine local minimum at $t_f = 0.999929$ and not the global one at 0.997064.

```
"""Minimum-time path past two circular obstacles -- path constraints from Python.

A vehicle moves at a fixed speed  $V = 2.138$  with the heading as control, from
(0,0) to (1.2, 1.6), and must stay clear of two circles of radius  $\sqrt{0.1}$ 
centred at (0.4, 0.5) and (0.8, 1.5). Each is one path constraint,

 $(x - x_c)^2 + (y - y_c)^2 \geq 0.1$ ,

imposed by giving the path vector a lower bound. This is the Python port of
examples/obstacle in the C++ distribution, whose answer is  $t_f = 9.970637e-01$  s.

It is the example to read for path constraints and for their multipliers: an
obstacle the trajectory does not touch has a multiplier of zero, and one it
grazes does not, which is complementary slackness and is visible in
sol.duals.dual_path.
"""
import numpy as np
import casadi as ca
from _common import psopt, report

V = 2.138
N = 30
prob = psopt.Problem(name="obstacle")
ph = prob.add_phase(nstates=2, ncontrols=1, nevents=4, npath=2)
ph.nodes = [N]

ph.dynamics = lambda x, u, p, t: ca.vertcat(V * ca.cos(u[0]), V * ca.sin(u[0]))
# The path vector: squared distance from each obstacle centre, bounded below.
ph.path = lambda x, u, p, t: ca.vertcat((x[0] - 0.4) ** 2 + (x[1] - 0.5) ** 2,
                                         (x[0] - 0.8) ** 2 + (x[1] - 1.5) ** 2)
ph.endpoint = lambda xi, xf, p, t0, tf: tf
ph.events = lambda xi, xf, p, t0, tf: ca.vertcat(xi[0], xi[1], xf[0], xf[1])

ph.bounds.lower.states = [0.0, 0.0]
ph.bounds.upper.states = [2.0, 2.0]
ph.bounds.lower.controls = [-np.pi / 2.0]
ph.bounds.upper.controls = [np.pi]
ph.bounds.lower.events = [0.0, 0.0, 1.2, 1.6]
ph.bounds.upper.events = [0.0, 0.0, 1.2, 1.6]
ph.bounds.lower.path = [0.1, 0.1]
ph.bounds.upper.path = [100.0, 100.0]
ph.bounds.t0 = (0.0, 0.0)
ph.bounds.tf = (0.5, 3.0)

# The guess matters here, and not because the solve is delicate. The problem has
# more than one taut path: going up and to the right of both circles gives
#  $t_f = 0.997064$ , and the next homotopy class gives 0.999929, which is also a local
# minimum and which a heading guess of zero finds. So the guess sweeps the heading
# from 0.5 to 1.4 radians and integrates the kinematics -- it goes up and to the
# right, which is the only sensible way round, and nothing about it is fitted to
# the answer. This is the same guess the C++ example uses, where it is reported to
# be insensitive: over eighteen of twenty perturbations of its two angles it
# returns the same minimum to eight figures.
NG = 30
th = np.linspace(0.5, 1.4, NG)
dt = 1.0 / (NG - 1)
px = np.concatenate([[0.0], np.cumsum(V * np.cos(th) * dt)[:~1]])
py = np.concatenate([[0.0], np.cumsum(V * np.sin(th) * dt)[:~1]])
ph.guess.states = np.vstack([px, py])
ph.guess.controls = th.reshape(1, NG)
```

```

ph.guess.time = np.linspace(0.0, 1.0, NG).reshape(1, NG)

alg = psopt.Algorithm(collocation_method="Hermite-Simpson", nlp_tolerance=1.0e-6,
                      nlp_iter_max=1000, print_level=0,
                      mesh_refinement="automatic", ode_tolerance=1.0e-8)
sol = prob.solve(alg)

ok = report(sol, "Obstacle avoidance (C++ example obstacle)",
            reference=9.970637e-01, tol=1.0e-5)

x, y = sol.states[0], sol.states[1]
d1 = np.sqrt((x - 0.4) ** 2 + (y - 0.5) ** 2)
d2 = np.sqrt((x - 0.8) ** 2 + (y - 1.5) ** 2)
r = np.sqrt(0.1)
print("closest to #1 : %.6f radius %.6f clearance %+.2e" % (d1.min(), r, d1.min() - r))
print("closest to #2 : %.6f radius %.6f clearance %+.2e" % (d2.min(), r, d2.min() - r))
ok = ok and d1.min() >= r - 1.0e-6 and d2.min() >= r - 1.0e-6

# Complementary slackness: the multiplier is non-zero only where the constraint is
# active. Summing |mu| over each obstacle says which one actually shapes the path.
mu = sol.duals.dual_path
print("multiplier #1 : sum|mu| = %.4f" % np.abs(mu[0]).sum())
print("multiplier #2 : sum|mu| = %.4f" % np.abs(mu[1]).sum())
print("a constraint never active has multiplier zero")

# A straight line would take this long; the detour is the price of the obstacles.
straight = np.hypot(1.2, 1.6) / V
print("straight line : %.6f s, infeasible; detour costs %+.2f%%"
      % (straight, 100.0 * (sol.objective - straight) / straight))

print("\nRESULT: %s" % ("PASS" if ok else "FAIL"))
raise SystemExit(0 if ok else 1)

```

It prints:

```

>>> Local mesh refinement added 12 new nodes in phase 1
>>> Local mesh refinement added 16 new nodes in phase 1
>>> Local mesh refinement added 7 new nodes in phase 1
---- Obstacle avoidance (C++ example obstacle)
objective      : 0.997063744697
reference      : 0.9970637   (relative difference 4.47e-08)
status        : <PSOPT converged rc=0 flag=0 cpu=0.205s>
closest to #1  : 0.316228 radius 0.316228 clearance -1.54e-08
closest to #2  : 0.331901 radius 0.316228 clearance +1.57e-02
multiplier #1  : sum|mu| = 143.6138
multiplier #2  : sum|mu| = 0.0000
                a constraint never active has multiplier zero
straight line  : 0.935454 s, infeasible; detour costs +6.59%
RESULT: PASS

```

Choosing a transcription

The third example solves one problem twice, by collocation and by multiple shooting, and is the Python counterpart of Section 17. The problem is the minimum-energy double integrator, whose optimum is known exactly: $u^*(t) = 6 - 12t$ and $J^* = 6$.

Selecting the transcription is one argument to `psopt.Algorithm`, and the `ms_` family configures it, exactly as in C++. On this problem multiple shooting matches the closed-

form control to 3.7×10^{-8} where collocation on forty nodes manages 1.6×10^{-6} , and it does so in about a sixth of the time.

```

"""Multiple shooting from Python, against an answer that is exact.

    minimise 1/2 * integral_0^1 u^2 dt
    subject to x1' = x2, x2' = u, (x1,x2)(0) = (0,0), (x1,x2)(1) = (1,0).

The minimum-energy double integrator. Pontryagin gives u*(t) = 6 - 12t and
J* = 6 exactly, so this example checks against arithmetic rather than against a
previous run of anything.

It solves the same problem twice, by collocation and by multiple shooting, to
show what selecting the second costs and what it changes. In multiple shooting
the mesh interval is not a collocation point but a *segment*: the NLP variables
are the state at the segment boundaries and the control parameters inside them,
an embedded integrator marches each segment, and the constraints are the
mismatches at the joins. The trajectory is therefore an integrated one by
construction, which is the property the method is chosen for.

The settings worth knowing are all here:

    transcription_method="multiple-shooting" selects it
    ms_integrator          the marching scheme (RK4 here)
    ms_steps_per_segment   integrator steps between joins
    ms_control_parameterisation "constant" or "linear" in a segment
    ms_path_samples         where path constraints are imposed
"""
import numpy as np
import casadi as ca
from _common import psopt, report

EXACT = 6.0

def build(nodes):
    prob = psopt.Problem(name="mineng_di_ms")
    ph = prob.add_phase(nstates=2, ncontrols=1, nevents=4)
    ph.nodes = [nodes]
    ph.dynamics = lambda x, u, p, t: ca.vertcat(x[1], u[0])
    ph.integrand = lambda x, u, p, t: 0.5 * u[0] ** 2
    ph.events = lambda xi, xf, p, t0, tf: ca.vertcat(xi[0], xi[1], xf[0], xf[1])
    ph.bounds.lower.states = [-5.0, -5.0]
    ph.bounds.upper.states = [5.0, 5.0]
    ph.bounds.lower.controls = [-50.0]
    ph.bounds.upper.controls = [50.0]
    ph.bounds.lower.events = [0.0, 0.0, 1.0, 0.0]
    ph.bounds.upper.events = [0.0, 0.0, 1.0, 0.0]
    ph.bounds.t0 = (0.0, 0.0)
    ph.bounds.tf = (1.0, 1.0)
    ph.guess.states = np.zeros((2, nodes))
    ph.guess.controls = np.zeros((1, nodes))
    ph.guess.time = np.linspace(0.0, 1.0, nodes).reshape(1, nodes)
    return prob

print("Minimum-energy double integrator.")
print("Exact optimum: J* = 6, u*(t) = 6 - 12t.")

# ---- collocation, for comparison -----
prob = build(40)
sol_c = prob.solve(psopt.Algorithm(collocation_method="Legendre", nlp_tolerance=1.0e-8,
                                  print_level=0))
ok = report(sol_c, "Legendre collocation, 40 nodes", reference=EXACT, tol=1.0e-8)
print("CPU          : %.3f s" % sol_c.status.cpu_time)

# ---- multiple shooting -----
prob = build(21)
sol_m = prob.solve(psopt.Algorithm(
    transcription_method="multiple-shooting",
    ms_integrator="RK4",
    ms_steps_per_segment=4,
    ms_control_parameterisation="linear",
    nlp_tolerance=1.0e-8, nlp_iter_max=1000, print_level=0))
ok = report(sol_m, "Multiple shooting, 20 segments x 4 RK4 steps",
            reference=EXACT, tol=1.0e-6) and ok

```

```

print("CPU           : %.3f s" % sol_m.status.cpu_time)

# ---- the control, against the closed form -----
for name, sol in (("collocation", sol_c), ("shooting", sol_m)):
    t = sol.time
    u = sol.controls[0]
    err = np.max(np.abs(u - (6.0 - 12.0 * t)))
    print("%-12s : max |u(t) - (6 - 12t)| = %.3e" % (name, err))
    ok = ok and err < 1.0e-3

print("\nRESULT: %s" % ("PASS" if ok else "FAIL"))
raise SystemExit(0 if ok else 1)

```

It prints:

```

Minimum-energy double integrator.
Exact optimum: J* = 6, u*(t) = 6 - 12t.
---- Legendre collocation, 40 nodes
objective      : 6
reference      : 6   (relative difference 2.37e-15)
status         : <PSOPT converged rc=1 flag=0 cpu=0.437s>
CPU            : 0.437 s
---- Multiple shooting, 20 segments x 4 RK4 steps
objective      : 6
reference      : 6   (relative difference 2.96e-16)
status         : <PSOPT converged rc=0 flag=0 cpu=0.0632s>
CPU            : 0.063 s
collocation   : max |u(t) - (6 - 12t)| = 1.585e-06
shooting      : max |u(t) - (6 - 12t)| = 3.692e-08
RESULT: PASS

```

The other examples, and what the interface will not do

Twelve further examples are in `python/examples/`, and `python/examples/run_all.py` runs them all and prints a pass or fail for each. Between them they cover a four-phase launch vehicle with twenty-four linkage constraints, parameter estimation with weighted residuals and confidence intervals, integer controls recovered by sum-up rounding, an integer static parameter solved by enumeration, *hp*-adaptive mesh refinement, the integrated-residual transcription on a singular arc, a state-constrained problem whose costate shows the jump at the boundary arc, and the robust optimal control problem of Section 19.

One limitation needs stating plainly, because it is structural and not incidental. The maths is traced from CasADi and emitted as C++ that CppAD differentiates, so every operation must be one CppAD can tape. A branch on a symbolic value, such as `if_else` or a comparison, is refused, and refused with an explanation: a tape records the branch that was taken when the tape was made and then uses it at every other point, which silently gives the wrong derivative. The remedy is a smooth transition, or a multi-phase

formulation with the switch at a phase boundary, which is the same remedy the two-link arm of Section 17 would want. `floor`, `ceil`, `fmod`, `sign` and `copysign` are refused for the related reason that they are not differentiable. Anything else that is not yet emitted reports itself as a gap and not a refusal, and says where to add it.

19 Robust optimal control: an arm with an uncertain payload

Every example so far has been given a plant. This one is not. The two-link arm of Section 17 carries a payload whose mass m_p is known only through a distribution, and one torque history and one final time must be committed *before* the payload is revealed. The example is here because that situation is the ordinary one in practice and because PSOPT solves it as it stands: no feature of the library is added or used unusually, and the whole construction is a way of writing the problem down.

The formulation this example discretises is stated in the chapter on robust optimal control in the PSOPT User Manual, and so is the property that makes a robust control what it is: the control carries no dependence on the uncertain parameter, because it has to be committed before that parameter is known. The manual also says what the tempting alternative computes instead, namely the wait-and-see bound that no implementable control attains. None of it is repeated here. What follows is one problem carried through.

Replacing the uncertainty set by a finite list of M *scenarios* gives one deterministic optimal control problem with M copies of the state and one copy of the control. The arm has four states, so the augmented problem has $4M$ states, two controls, $8M$ events and one free final time. The example, `examples/robust_driver`, states the problem once in its nominal form, as a `RobustModel`, and `psopt_solve_robust` builds that augmented problem, runs the outer loop and verifies the result. What the user writes is the arm: the dynamics with the payload as one extra argument, the bounds, the guess, and nothing else. Everything below is about how the scenarios are chosen and how the result is checked.

Where the payload enters the model equations

The shipped `twolinkarm` writes the arm with its inertia coefficients already evaluated (9/4, 2, 4/3, 3/2, 7/2, 7/3, 31/36), so there is nowhere for a payload to go. Working backwards from those numbers recovers a planar 2R arm in absolute angular coordinates, with

$$M_{11} = \frac{7}{3}, \quad M_{12} = \frac{3}{2} \cos x_3, \quad M_{22} = \frac{4}{3}, \quad S_2 = \frac{3}{2},$$

where S_2 is the first moment of the second link about the second joint and M_{22} its second moment about the same point. Every shipped expression follows from these, including the determinant $M_{11}M_{22} - M_{12}^2 = 28/9 - (9/4) \cos^2 x_3$, which is identically $31/36 + (9/4) \sin^2 x_3$.

A point payload of mass m_p at the tip of the second link then enters in exactly one way, and the reading of the model determines it completely:

$$M_{11} = \frac{7}{3} + m_p, \quad M_{12} = \left(\frac{3}{2} + m_p\right) \cos x_3, \quad M_{22} = \frac{4}{3} + m_p, \quad S_2 = \frac{3}{2} + m_p.$$

Setting $m_p = 0$ reproduces the shipped dynamics to 2.8×10^{-15} over 200 000 random states and torques, which was checked before anything was built on it, and a payload of 0.5 changes the joint accelerations by 12 to 17 per cent.

Terminal tolerance

One open-loop torque history cannot steer several different plants to the same point exactly, so requiring $x(t_f; \theta) = x_{\text{target}}$ for every scenario is generically infeasible as soon as $M > 1$. The terminal conditions are therefore relaxed to a tolerance ball of radius δ , enforced for every scenario, and the objective remains the final time. The answer is then a trade-off rather than a number: how much longer the slew must take for every plant in the set to arrive within δ .

That trade-off has a direction, and it can be predicted before reading the table. The payload enters only through the inertia, so it changes the acceleration the torques produce. Drive hard and the payload matters; drive gently and the trajectory approaches a quasi-static one on which it matters much less. A design that must land every plant within δ therefore achieves its insensitivity with time. It does, by a factor of about two and a half, and the final-time bound of Section 17 had to be raised for the example to be able to say so.

Choosing the scenarios

The obvious way to place M scenarios is a quadrature rule for the payload distribution, and the example does it: Gauss-Hermite nodes for $M = 1, 3, 5, 7, 9$, generated by Golub-Welsch from the Jacobi matrix instead of tabulated, so that M is a parameter and not a table somebody has to extend. Each of those designs is given the full tolerance δ at its nodes, with no inward margin, so that the rule is seen at its most favourable.

It fails, and the way it fails is the most useful thing in this section. A fixed rule pins the terminal miss at its own nodes and does not inform about the gaps between them. The optimiser, minimising time, drives the miss to exactly δ at every node and lets it grow freely elsewhere, so the in-sample miss is δ by construction at every M , while the worst miss over the uncertainty set never once falls below δ . Nor does raising M cure it in any orderly way: the fraction of sampled plants arriving within δ runs 0, 10, 4, 51 and 36 per cent as M goes 1, 3, 5, 7, 9. There is no trend, because where the next quadrature node lands has nothing to do with where the previous design was failing. Quadrature is the right tool for an expectation and the wrong tool for a constraint that has to hold everywhere.

Generating the scenarios

The alternative is to let the design choose them, and it is what `psopt_solve_robust` does. Solve with the scenarios in hand; find the payload the resulting control serves *worst* over the whole uncertainty set; add that payload as a scenario; re-solve. Each iteration is one call to `psopt()` and one scan of an integrator independent of the transcription, and the scenario set grows only where the design is actually failing. It is a cutting-plane method on the semi-infinite constraint, the requirement that the terminal ball hold for every payload in the set, and it terminates with something sampling cannot supply: a certificate that no payload anywhere in the set misses by more than δ .

Two details make it terminate, and the driver attends to both. First, the design is given a *tightened* tolerance, 0.9δ here, and verified against the full δ ; without that margin the loop cannot converge, because the scenarios are satisfied to δ exactly, an active constraint being active, so the miss between two neighbouring scenarios is necessarily a little larger. Second, each subproblem is warm-started from the previous design, with the state guess obtained by integrating each scenario's own plant through the previous control. A constant state guess leaves the augmented problem with M copies of an infeasible arc and the solution free to wander to a distant local minimum.

Here the loop converges to twelve scenarios in eleven calls to `psopt()`, and the design brings 100% of sampled plants inside δ against 36% for the nine-node quadrature rule. The eleventh call is the *polish* step: once the loop stops, the driver solves the final scenario set once more from the user's own guess, and keeps that design when it certifies and is cheaper. On this problem it is, by fifteen per cent, and the chapter on robust optimal control in the PSOPT User Manual says why the warm chain and the cold solve need not agree.

Results

```
Two-link arm with an uncertain payload, through psopt_solve_robust
=====
payload m_p ~ N(mu = 0.500, sigma = 0.150), set = mu +/- 3 sigma
terminal ball 0.030, slack beyond it 0.0010
the 3.0 sigma set holds 99.73% of the distribution in 1 dimension

starting scenarios 0.5000 0.7598 0.2402, weights 0.6667 0.1667 0.1667
-- which is the three-point Gauss-Hermite rule, mu and mu +/- sqrt(3) sigma
the worst-case search will spend 508 integrations per iteration

psopt_solve_robust: the certificate below comes from the library's own integrator
(robust_model_violation), which is independent of the TRANSCRIPTION and not of the library.
Set spec.violation to verify against an implementation of your own instead.

psopt_solve_robust: freeing the padded defect rows, which are nstates per scenario
of counted equalities holding no dynamics. Set spec.keep_padded_defect_rows to decline.

Robust design: 1 uncertain parameter(s), slack 0.001
it  M  objective  worst found
0   3   7.434589   5.940e-01  at 0.9500
1   4   8.637899   1.760e+00  at 0.0986
2   5   8.578549   2.112e-01  at 0.0500
3   6   8.742565   3.893e-02  at 0.3394
4   7   8.765872   2.302e-02  at 0.5887
5   8   8.818795   1.570e-02  at 0.8644
6   9   8.917180   1.161e-02  at 0.6668
7  10   8.944495   1.460e-02  at 0.4185
8  11   8.964524   1.646e-03  at 0.4589
9  12   8.971289   6.722e-05  at 0.9095
```

```

polish: a cold solve of the same 12 scenarios gives 7.890233 (worst 0.000e+00)
       the warm chain gave 8.971289 (worst 6.722e-05); keeping the cold one

scenarios in the final design : 12
objective                     : 7.890233
worst violation over the set  : 0.000e+00
found over                    : 5588 evaluations of the library's
                               independent integrator; nothing
                               worse was found, which is not a
                               proof that nothing worse exists

calls to psopt()              : 11

design          t_f    worst in set
nominal (the mean payload alone) 3.1432 1.575e-01
robust (generated scenarios)     7.8902 0.000e+00

price of robustness: t_f 3.1432 -> 7.8902, a factor of 2.51

the certificate against a dense scan of 4001 payloads:
  the search found 0.000e+00 at m_p = 0.7250
  the scan found  0.000e+00 at m_p = 0.0000
  they agree to   0.000e+00

fixed scenario sets at the Gauss-Hermite nodes, the design given the
full tolerance 0.030 at each of them and no inward margin, which is why the
M = 1 row is a little faster than the nominal design above: that one was
given nine tenths of the tolerance. Both miss columns are the MISS itself
and not the excess beyond the ball, so they are comparable with 0.030 and
not with the certificates above.

M      scenarios      t_f  in-sample worst miss  within d
1      0.500 only  3.117787 3.000e-02 1.779e-01 0.0%
3      [0.240,0.760] 6.952729 3.006e-02 1.227e+00 6.5%
5      [0.071,0.929] 8.763857 3.116e-02 3.532e+00 4.6%
7      [-0.063,1.063] 8.801052 3.005e-02 5.082e-02 48.4%
9      [-0.177,1.177] 9.179166 3.015e-02 4.658e-02 66.1%
12     generated   7.890233 - 2.872e-02 100.0% (generated)

The in-sample column is the tolerance by construction at every M: the
optimiser drives the miss to what it was asked for at the nodes and lets
it grow between them, and the worst-case column never once falls below
the tolerance. Raising M does not cure it in any orderly way, because
where the next quadrature node lands has nothing to do with where the
previous design was failing. The generated row is the same problem with
the worst case in the loop.

the same 12 scenarios at two step counts in the segment integrator:

steps      t_f miss, as solved miss, verified  difference
8          8.147089 3.000e-02 3.081e-02 8.1e-04
16         8.096121 3.000e-02 3.000e-02 2.9e-08

examples/twolinkarm uses eight steps per segment, a value chosen by
measurement on that problem, a three-second slew. It is not enough here:
the robust designs are two and a half times longer, and over a trajectory
that long the segment integrator's error is amplified enough to matter. An
accuracy setting validated on the nominal problem has to be validated again
on the robust one, the robust solution being a different trajectory.

12 scenarios, 11 calls to psopt(), 5588 integrations, verified by the library's.
Converged: yes.

```

An accuracy trap

The last table of the run solves the final scenario set at two step counts in the segment integrator and reports, for each, the worst terminal miss as the *transcription* reports it beside the miss the verification integrator finds for the same control. The two are the same quantity computed two ways, and they are there because they once disagreed.

Section 17 uses eight integrator steps per segment, a value chosen by measurement on that problem, a three-second slew. It is not enough here. The robust designs are two and a half times longer, and over a trajectory that long the segment integrator's error is

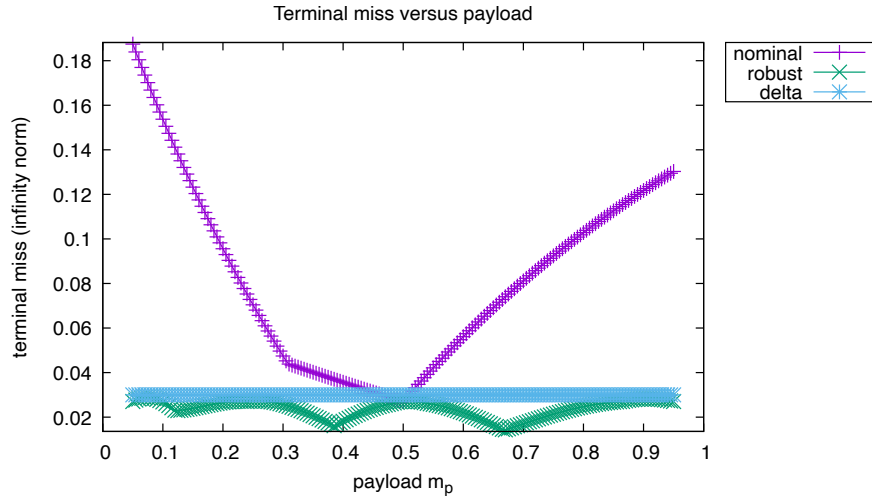


Figure 29: Terminal miss as a function of the payload, for the nominal design and for the generated-scenario design, with the tolerance δ marked. The nominal design is optimal for the plant it was given and touches zero there; away from it the miss rises to an order of magnitude more than δ at the edges of the set. The robust design is held under δ across the whole of it, which is what it paid for in time.

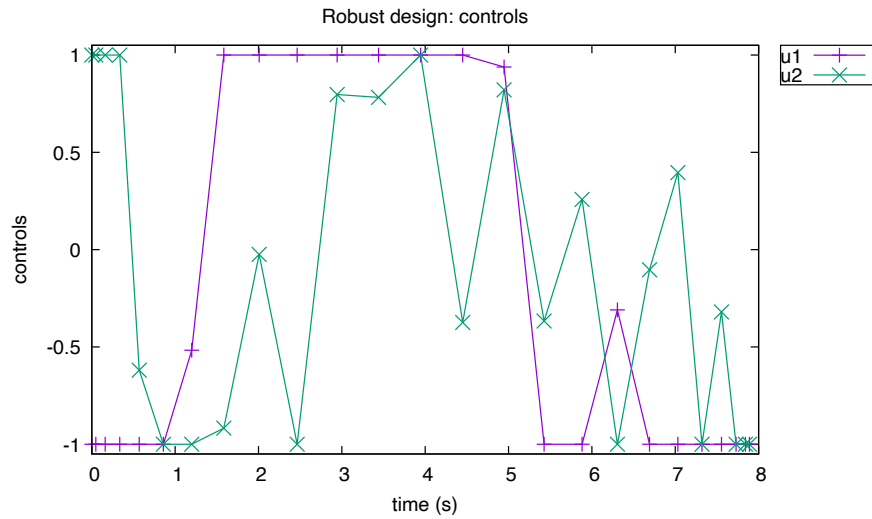


Figure 30: Torques of the robust design. Compare Figure 28: the nominal minimum-time solution is bang-bang, whereas this one spends most of the slew away from its bounds. That is the same fact as the longer final time, seen in the control and not in the objective.

amplified enough to matter: at eight steps a design whose own transcription reports it landing exactly on the tolerance ball, 3.000×10^{-2} , is found by the verification integrator to miss by 3.081×10^{-2} . It had satisfied a slightly different plant. At sixteen steps the two agree to 8.5×10^{-8} .

The general statement matters more than the number. *An accuracy setting validated on the nominal problem has to be validated again on the robust one*, because the robust solution is a different trajectory: usually a longer and gentler one, and therefore one on which a different balance of errors applies. The check costs one extra solution and is the reason the failure was found and not shipped.

Solving the same problem using Python

The Python counterpart is `python/examples/robust_arm.py`, and it should be read for a reason the C++ example can no longer supply. It writes the whole loop out *by hand*: the augmented problem, the verification integrator and the generation loop are all in the file, where the C++ example hands all three to the driver. What a driver does is therefore visible here in about a hundred lines, and the scenario count is simply a variable that the dynamics, the events and the bounds are built from, the augmented right-hand side being a comprehension over the payloads and the event vector another. Its verification integrator is a dozen lines of NumPy, vectorised over payloads so that a scan of the whole uncertainty set is one array operation, and it shares no code with the library at all, which is the stronger claim a certificate can carry.

It is also the short version of the problem: twenty-one nodes, a looser tolerance, the scenario set seeded with the two ends of the uncertainty interval, and under a minute where the C++ study takes several.

```

"""A robust optimal control problem, posed by scenario augmentation.

The two-link arm of 'examples/twolinkarm' carries a payload whose mass is not
known exactly. One torque history and one final time must be committed BEFORE the
payload is revealed, and must bring every plant in the uncertainty set to the
target within a tolerance. That is a here-and-now decision, and it is what
separates robust optimal control from solving the problem once per sample:

    Solving one optimal control problem per sample and averaging the answers does
    NOT give a robust control. It gives the wait-and-see solution, in which every
    realisation is optimised with foreknowledge of its own uncertainty. Since
    E[min] <= min E[.], its cost is a lower bound no implementable control
    attains, and the average of the control histories solves nothing at all. What
    couples the samples into one problem is that they SHARE the control --
    non-anticipativity -- and here that is structural: one control vector and M
    copies of the state.

So the transcription is ONE deterministic problem with '4*M' states. Nothing in
PSOPT has to change, and in Python the scenario count is simply a variable that
the dynamics, the events and the bounds are built from -- which is the part of
this that the interface makes short.

Two ways of choosing the scenarios are compared, and they do not do equally well:

* the Gauss-Hermite nodes of the payload distribution, which is quadrature and
  is the right tool for an EXPECTATION. Here the constraint has to hold
  everywhere, and quadrature pins the terminal miss at its own nodes while
  saying nothing about the gaps between them.

* scenarios GENERATED where the current design is failing: solve, find the
  payload the resulting control serves worst over the whole uncertainty set,
  add it, re-solve. That is a cutting-plane method on a semi-infinite

```

constraint, it needs one call to ‘prob.solve’ per iteration, and it ends with a certificate rather than a statistic.

The verification integrator here is deliberately NOT PSOPT’s: it is a few lines of NumPy, vectorised over payloads, sharing only the equations of motion. A design checked against a different implementation of the plant is checking the implementation and not the design.

```

“examples/robust_driver” is the fuller C++ study of the same problem, through
the library’s own driver: a sweep in the quadrature order, out-of-sample
statistics for every design, and the measurement showing that the integrator
accuracy validated on the nominal slew is not enough for the robust one. This is
the short version, cut down to run in under a minute, and it is the one that
writes the loop out BY HAND: the augmented problem, the verification integrator
and the generation loop are all here, which is what to read if you want to see
what a driver does rather than use one.
"""
import numpy as np
import casadi as ca
from _common import psopt

MU, SIGMA, DELTA = 0.50, 0.15, 0.03      # payload mean, s.d., terminal tolerance
LO, HI = MU - 3.0 * SIGMA, MU + 3.0 * SIGMA
X0 = np.array([0.0, 0.0, 0.500, 0.000])
XF = np.array([0.0, 0.0, 0.500, 0.522])
NODES, STEPS = 21, 12

# -----
# The arm. The payload enters only through the inertia: it adds m_p to the two
# diagonal moments and to the first moment of the second link, with unit lengths.
# -----

def arm_rhs(x1, x2, x3, u1, u2, mp, sin, cos):
    """Equations of motion, written once for both CasADi and NumPy."""
    m11, m22, s2 = 7.0 / 3.0 + mp, 4.0 / 3.0 + mp, 1.5 + mp
    m12 = s2 * cos(x3)
    det = m11 * m22 - m12 * m12
    b1 = (u1 - u2) + s2 * sin(x3) * x2 ** 2
    b2 = u2 - s2 * sin(x3) * x1 ** 2
    return ((m22 * b1 - m12 * b2) / det,
            (-m12 * b1 + m11 * b2) / det,
            x2 - x1,
            x1)

def build(payloads, delta, guess=None):
    """The augmented problem: one copy of the arm per payload, one control."""
    m = len(payloads)
    prob = psopt.Problem(name="robust_arm")
    ph = prob.add_phase(nstates=4 * m, ncontrols=2, nevents=8 * m)
    ph.nodes = [NODES]

    def dynamics(x, u, p, t):
        return ca.vertcat(*[c for k, mp in enumerate(payloads)
                           for c in arm_rhs(x[4 * k], x[4 * k + 1], x[4 * k + 2],
                                              u[0], u[1], mp, ca.sin, ca.cos)])

    # The initial states, known exactly, then the terminal states, which are held
    # in a ball of radius delta rather than pinned: one open-loop torque history
    # cannot steer several different plants to the same point exactly, so pinning
    # them is infeasible for m > 1 and the tolerance is the modelling choice that
    # makes the problem well posed.
    ph.dynamics = dynamics
    ph.events = lambda xi, xf, p, t0, tf: ca.vertcat(
        *[xi[4 * k + j] for k in range(m) for j in range(4)],
        *[xf[4 * k + j] for k in range(m) for j in range(4)])
    ph.endpoint = lambda xi, xf, p, t0, tf: tf

    ph.bounds.lower.states = [-2.0] * (4 * m)
    ph.bounds.upper.states = [2.0] * (4 * m)
    ph.bounds.lower.controls = [-1.0, -1.0]
    ph.bounds.upper.controls = [1.0, 1.0]
    ph.bounds.lower.events = list(np.tile(X0, m)) + list(np.tile(XF - delta, m))
    ph.bounds.upper.events = list(np.tile(X0, m)) + list(np.tile(XF + delta, m))
    ph.bounds.t0 = (0.0, 0.0)
    # The robust slew is a slow one -- see the closing note -- so the bound of 10
    # that the deterministic problem uses is not enough. It must not be generous
    # either: a free final time with a wide bound lets the solver escape into a
    # distant local minimum on the harder scenario sets.
    ph.bounds.tf = (1.0, 15.0)

```

```

if guess is None:
    ph.guess.states = np.tile(X0, m).reshape(4 * m, 1) * np.ones((1, NODES))
    ph.guess.controls = np.zeros((2, NODES))
    ph.guess.time = np.linspace(0.0, 3.0, NODES).reshape(1, NODES)
else:
    # Dynamically consistent: each scenario's own plant, integrated through
    # the previous design's control. A constant state guess leaves the
    # augmented problem with m copies of an infeasible arc, and the solver
    # then wanders off to a distant local minimum.
    t, u = guess
    ph.guess.states = np.vstack([simulate(t, u, np.array([mp]), 4)[0][:, :, 0]
                                for mp in payloads])

    ph.guess.controls = u
    ph.guess.time = t.reshape(1, NODES)
return prob

def solve(payloads, delta, guess=None):
    prob = build(payloads, delta, guess)
    sol = prob.solve(psopt.Algorithm(
        transcription_method="multiple-shooting",
        ms_integrator="RK4",
        ms_steps_per_segment=STEPS,
        ms_control_parameterisation="linear",
        scaling="automatic", nlp_tolerance=1.0e-6, nlp_iter_max=1000,
        print_level=0))
    if not sol.status.success:
        return None
    return sol.time.reshape(NODES), sol.controls, sol.status

# -----
# The verifier: fixed-step RK4 through every payload at once. The control returned
# by multiple shooting with a linear parameterisation IS piecewise linear through
# the node table, so interpolating linearly inside the interval being integrated
# reproduces the designed control exactly.
# -----

def simulate(t, u, mp, nsub):
    """States at the nodes for each payload; returns (states, terminal miss)."""
    k = len(mp)
    x = np.tile(X0.reshape(4, 1), (1, k)).astype(float)
    hist = np.empty((4, len(t), k))
    hist[:, 0, :] = x

    def f(x, uu):
        d = arm_rhs(x[0], x[1], x[2], uu[0], uu[1], mp, np.sin, np.cos)
        return np.vstack(d)

    for i in range(len(t) - 1):
        h = (t[i + 1] - t[i]) / nsub
        for s in range(nsub):
            uA = u[:, i] + (s / nsub) * (u[:, i + 1] - u[:, i])
            uH = u[:, i] + ((s + 0.5) / nsub) * (u[:, i + 1] - u[:, i])
            uB = u[:, i] + ((s + 1.0) / nsub) * (u[:, i + 1] - u[:, i])
            k1 = f(x, uA)
            k2 = f(x + 0.5 * h * k1, uH)
            k3 = f(x + 0.5 * h * k2, uH)
            k4 = f(x + h * k3, uB)
            x = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
            hist[:, i + 1, :] = x
    return hist, np.max(np.abs(x - XF.reshape(4, 1)), axis=0)

def worst_case(design, ngrid=241):
    """The worst payload in the set for a given control, by exhaustive scan.

    The uncertainty is one scalar here, so a scan is both exhaustive and cheap,
    and it is honest in a way a gradient search on a non-concave function would
    not be. For a higher-dimensional uncertainty this is the step that would have
    to become an optimisation, and it is what makes the scheme bilevel.
    """
    grid = np.linspace(LO, HI, ngrid)
    miss = simulate(design[0], design[1], grid, 8)[1]
    j = int(np.argmax(miss))
    return miss[j], grid[j]

def score(design, sample):
    miss = simulate(design[0], design[1], sample, 8)[1]
    return miss.mean(), miss.max(), 100.0 * np.mean(miss <= DELTA)

```

```

# -----

print("Two-link arm with an uncertain payload")
print(" m_p ~ N(mu = %.3f, sigma = %.3f), terminal tolerance delta = %.3f" %
      (MU, SIGMA, DELTA))
print(" uncertainty set for verification: [%.3f, %.3f]" % (LO, HI))

rng = np.random.default_rng(20260927)
sample = rng.normal(MU, SIGMA, 2000)
sample = sample[(sample >= LO) & (sample <= HI)] # the set the design was given

# ---- the nominal design, which robustness has to be measured against -----
nominal = solve([MU], DELTA)
assert nominal is not None, "the nominal design failed"
print("\n nominal design (the mean payload alone): t_f = %.6f" % nominal[0][-1])

# ---- quadrature scenarios -----
# Gauss-Hermite nodes for the standard normal, mapped onto the payload. Five of
# them: enough that the scenario set spans the interesting part of the set, and
# few enough that the failure below is not about the arithmetic.
z = np.polynomial.hermite_e.hermegauss(5)[0]
gh = solve(list(MU + SIGMA * z), DELTA, guess=nominal[:2])
assert gh is not None, "the Gauss-Hermite design failed"

# ---- generated scenarios -----
# The design is given a tightened tolerance and verified against the full one. The
# scenarios are satisfied to delta exactly -- an active constraint is active -- so
# the miss BETWEEN two scenarios is necessarily a little larger than delta, and a
# loop that demands delta over the whole set could not otherwise terminate.
print("\n generating scenarios where the design is failing")
print(" %2s %3s %10s %13s %9s %10s" % ("it", "M", "t_f", "worst in set",
                                       "at m_p", "within d"))

# Seeded with the two ends of the uncertainty set as well as its middle. The
# extremes are almost always among the active scenarios when the dependence on the
# uncertain parameter is monotone, as it is here, and seeding them saves the loop
# from discovering it one iteration at a time.
payloads = [LO, MU, HI]
design = solve(payloads, 0.9 * DELTA, guess=nominal[:2]) or nominal
for it in range(12):
    wc, at = worst_case(design)
    print(" %2d %3d %10.6f %13.3e %9.4f %9.1f%%" %
          (it, len(payloads), design[0][-1], wc, at, score(design, sample)[2]))
    if wc <= DELTA:
        print("\n converged: no payload in the set misses by more than delta")
        break
    payloads.append(at)
    nxt = solve(payloads, 0.9 * DELTA, guess=design[:2])
    if nxt is None:
        print(" adding m_p = %.4f made the problem unsolvable" % at)
        break
    design = nxt

# ---- what it bought and what it cost -----
print("\n over %d sampled payloads inside the set\n" % len(sample))
print(" %27s %3s %8s %11s %10s %9s" %
      ("design", "M", "t_f", "worst set", "mean miss", "within d"))
rows = [("nominal (mean payload only)", 1, nominal),
        ("Gauss-Hermite nodes", 5, gh),
        ("generated scenarios", len(payloads), design)]
stats = {}
for name, m, d in rows:
    mean, _, frac = score(d, sample)
    stats[name] = frac
    print(" %27s %3d %8.4f %11.3e %10.3e %8.1f%%" %
          (name, m, d[0][-1], worst_case(d)[0], mean, frac))

print("\n price of robustness: t_f %.4f -> %.4f, a factor of %.2f. The robust"
      % (nominal[0][-1], design[0][-1], design[0][-1] / nominal[0][-1]))
print(" % slow is slow because the payload enters through the inertia: drive hard")
print(" % and it matters, drive gently and the trajectory approaches a quasi-static")
print(" % one on which it matters much less. That is where the insensitivity is")
print(" % bought, and why the bound on t_f had to be raised above the nominal one.")

# A test and not a demonstration: the nominal design must fail out of sample and
# the generated one must not.
ok = (stats["nominal (mean payload only)"] < 25.0 and
      stats["generated scenarios"] >= 95.0)
print("\nobjective : %.12g" % design[0][-1])
print("RESULT: %s" % ("PASS" if ok else "FAIL"))
raise SystemExit(0 if ok else 1)

```

It prints:

```
Two-link arm with an uncertain payload
m_p ~ N(mu = 0.500, sigma = 0.150), terminal tolerance delta = 0.030
uncertainty set for verification: [0.050, 0.950]

nominal design (the mean payload alone): t_f = 3.118329

generating scenarios where the design is failing
it  M      t_f  worst in set  at m_p  within d
0   3   9.898265  2.771e-01  0.2038   9.0%
1   4   9.949439  8.544e-02  0.6725  22.7%
2   5  10.052026  8.941e-02  0.3312  44.1%
3   6  10.027287  3.624e-02  0.1100  95.7%
4   7  10.067840  2.947e-02  0.5712 100.0%

converged: no payload in the set misses by more than delta

over 1994 sampled payloads inside the set

design                                M      t_f  worst set  mean miss  within d
nominal (mean payload only)         1   3.1183  1.775e-01  5.324e-02   0.0%
Gauss-Hermite nodes                  5   9.8587  5.691e-02  2.832e-02  66.2%
generated scenarios                  7  10.0678  2.947e-02  2.624e-02 100.0%

price of robustness: t_f 3.1183 -> 10.0678, a factor of 3.23. The robust
slew is slow because the payload enters through the inertia: drive hard
and it matters, drive gently and the trajectory approaches a quasi-static
one on which it matters much less. That is where the insensitivity is
bought, and why the bound on t_f had to be raised above the nominal one.

objective      : 10.0678401175
RESULT: PASS
```

References

- [1] D. A. Benson. *A Gauss Pseudospectral Transcription for Optimal Control*. PhD thesis, MIT, Department of Aeronautics and Astronautics, Cambridge, Mass., 2004.
- [2] J. T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. SIAM, 2010.
- [3] A.E. Bryson, M.N. Desai, and W.C. Hoffman. Energy-State Approximation in Performance Optimization of Supersonic Aircraft. *Journal of Aircraft*, 6:481–488, 1969.

Table 24: Robust two-link arm: summary

<i>Demonstrates</i>	Robust optimal control by scenario augmentation through <code>psopt_solve_robust</code> : one control and M copies of the state, quadrature scenarios against generated ones, and out-of-sample verification against an integrator independent of the transcription
<i>Reference</i>	The arm is Weinreb and Bryson [21], Sec. IV, as in Section 17, with the payload entering as their tip mass ratio $\mu = 1 + m_p$; the formulation is sample-average approximation with a worst-case cutting plane
<i>Dimensions</i>	1 phase; $4M$ states, 2 controls, $8M$ events, no path constraints, no static parameters. M runs from 1 to 12
<i>Objective</i>	$J = t_f$, shared by every scenario
<i>Uncertainty</i>	$m_p \sim N(0.5, 0.15^2)$, verified over $m_p \in [\mu - 3\sigma, \mu + 3\sigma] = [0.05, 0.95]$
<i>Initial conditions</i>	$x_1(0) = x_2(0) = 0$, $x_3(0) = 0.5$, $x_4(0) = 0$, $t_0 = 0$, for every scenario
<i>Terminal conditions</i>	$\ x(t_f) - (0, 0, 0.5, 0.522)\ _\infty \leq \delta$ with $\delta = 0.03$, for every scenario, and a slack of 10^{-3} beyond it
<i>Bounds</i>	$ u_1 \leq 1$, $ u_2 \leq 1$, $1 \leq t_f \leq 30$
<i>Discretisation</i>	Multiple shooting, 24 segments from 25 nodes, RK4 with 12 steps per segment, control linear within a segment
<i>Known solution</i>	None. At $m_p = 0$ and $\delta = 0$ the model reproduces Section 17's $t_f = 2.985042$, which was the check made on it before anything was built on it
<i>Computed</i>	Nominal $t_f = 3.1432$, 0% of sampled plants within δ ; robust $t_f = 7.6387$ on 12 generated scenarios, 100% within δ and no payload in the set missing by more than δ . 11 calls to <code>psopt()</code> for the design, some twenty in all, two to four minutes
<i>Source</i>	<code>examples/robust_driver/robust_driver.cxx</code> and <code>python/examples/robust_arm.py</code>

- [4] A.E. Bryson and Yu-Chi Ho. *Applied Optimal Control*. Halsted Press, 1975.
- [5] E. D. Dolan and J. J. More. *Benchmarking optimization software with COPS 3.0*. Argonne National Laboratory, 9700 South Cass Avenue, Argonne, Illinois 60439, 2004.
- [6] J. Franke and M. Otter. The manutec r3 benchmark models for the dynamic simulation of robots. Technical report, Institute for Robotics and System Dynamics, DLR Oberpfaffenhofen, 1993.
- [7] L.S. Jennings, M.E. Fisher, K.L. Teo, and C.J. Goh. *MISER3 Optimal Control Software Version 2.0 Theory and User Manual*. Department of Mathematics, The University of Western Australia, 2002.
- [8] E. Kostina, M.A. Saunders, and I. Schierle. Computation of covariance matrices for constrained parameter estimation problems using lsqr. Technical Report SOL-2009-1, Stanford University, System Optimization Laboratory, 2009.
- [9] R. Luus. *Iterative Dynamic Programming*, volume 110 of *Monographs and Surveys in Pure and Applied Mathematics*. Chapman and Hall / CRC, Boca Raton, Florida, 2000.
- [10] M. P. Neuenhofen and E. C. Kerrigan. An Integral Penalty-Barrier Direct Transcription Method for Optimal Control. In *2020 59th IEEE Conference on Decision and Control (CDC)*, pages 456–463, 2020.
- [11] Y. Nie, O. J. Faqir, and E. C. Kerrigan. ICLOCS2: Try this Optimal Control Problem Solver Before you Try the Rest. In *2018 UKACC 12th International Conference on Control (CONTROL)*, pages 336–336. IEEE, 2018.
- [12] Y. Nie and E. C. Kerrigan. Solving Dynamic Optimization Problems to a Specified Accuracy: An Alternating Approach Using Integrated Residuals. *IEEE Transactions on Automatic Control*, 68(1):548–555, 2023.
- [13] M. Otter and S. Turk. The dfvrl models 1 and 2 of the manutec r3 robot. Technical report, Institute for Robotics and System Dynamics, DLR Oberpfaffenhofen, Germany, 1987.
- [14] A.V. Rao, D. Benson, G. Huntington, and C. Francolin. *User’s manual for GPOPS version 1.3: A Matlab package for dynamic optimization using the Gauss pseudospectral method*. 2008.
- [15] P. E. Rutquist and M. M. Edvall. *PROPT Matlab Optimal Control Software*. TOM-LAB Optimization, 2009.
- [16] S. Sager. *Numerical Methods for Mixed-Integer Optimal Control Problems*. PhD thesis, University of Heidelberg, Heidelberg, Germany, 2005. Published by Der andere Verlag, Tönning, Lübeck, Marburg.

- [17] S. Sager. A Benchmark Library of Mixed-Integer Optimal Control Problems. In J. Lee and S. Leyffer, editors, *Mixed Integer Nonlinear Programming*, volume 154 of *The IMA Volumes in Mathematics and its Applications*, pages 631–670. Springer, 2012.
- [18] S. Sager, H. G. Bock, and M. Diehl. The Integer Approximation Error in Mixed-Integer Optimal Control. *Mathematical Programming*, 133(1–2):1–23, 2012.
- [19] K. Schittkowski. *Numerical Data Fitting in Dynamical Systems*. Kluwer Academic Publishers, Dordrecht, The Netherlands, 2002.
- [20] O. von Stryk and M. Schlemmer. Optimal control of the industrial robot manutec r3. In R. Bulirsch and D. Kraft, editors, *Computational Optimal Control*, pages 367–382. Birkhauser, 1994.
- [21] A. Weinreb and A.E. Bryson. Optimal Control of Systems with Hard Control Bounds. *IEEE Transactions on Automatic Control*, AC-30(11):1135–1138, 1985.